KALWINGS

PLATFORM MANUAL

Run it. Support it.
Build on it.

Staff guide for admins and support, and developer reference for the team that maintains app.kalwings.com.

Version 1.0

Date 15 October 2026

Audience Admins, support staff, developers

Product app.kalwings.com · Robokalam Technologies

Contents

Staff guide 1

A1 Read this first 2
A2 Quick reference: where do I do that? 2
A3 Signing in and becoming an admin 2
A4 A tour of the admin workspace 2
A5 The Overview page and verification queues 2
A6 Companies: add, verify, edit, post for them 2
A7 Colleges 2
A8 Mentors 2
A9 Users: add people, change roles, suspend 2
A10 Search, filters and notifications 2
A11 Bulk upload from Excel or CSV 2
A12 Employer plans: rename, reprice, add 2
A13 Reviews: listings, credentials, reports, inbox 2
A14 Site settings: banner, support email, sign-ups 2
A15 Launch readiness 2
A16 Insights, audit log and server errors 2
A17 Seeing what your users see 2
A18 Troubleshooting for staff 2
A19 Glossary for staff 2

Developer reference 2

B1 What you are working on 2
B2 Set up your machine (step by step) 2
B3 Configuration reference 2
B4 Authentication, sessions and roles 2
B5 The data model 2
B6 Database migrations (the careful part) 2
B7 Server actions: the pattern to copy 2
B8 How-to recipes 2
B9 The admin console, file by file 2
B10 Bulk import internals 2
B11 Email 2
B12 Uploads and files 2
B13 UI conventions and the design system 2
B14 Security rules for contributors 2
B15 Testing 2
B16 CI/CD, deploy, health and backups 2
B17 Working with Git and pull requests 2
B18 Troubleshooting for developers 2
B19 Glossary 2
B20 Appendices (generated from the source tree) 2

Figures

Figure 1. The admin workspace after you sign in 3
Figure 2. Sign-in page, step 1: ask for a code 3
Figure 3. Sign-in page, step 2: enter the code 3
Figure 4. Create-account page: people choose their own role 3
Figure 5. The Companies page 3
Figure 6. A company page 3
Figure 7. Posting a listing and managing listings 3
Figure 8. The Colleges page 3
Figure 9. The Mentors page 3
Figure 10. What the mentor sees in Mentor profile 3
Figure 11. The Users page 3
Figure 12. What the user does: Settings → Change my role 3
Figure 13. The Bulk upload page 3
Figure 14. Preview before import 3
Figure 15. After pressing Import 3
Figure 16. Editing a plan 3
Figure 17. Adding a plan 3
Figure 18. Assigning plans 3
Figure 19. Listing moderation 3
Figure 20. Profile verification 3
Figure 21. Abuse reports 3
Figure 22. Contact inbox 3
Figure 23. Site settings 3
Figure 24. Launch readiness (demo data; banner showing) 3
Figure 25. Insights 3
Figure 26. Audit log 3
Figure 27. Server errors 3
Figure 28. Public opportunities page 3
Figure 29. Public mentors page 3
Figure 30. Student home 3
Figure 31. Student profile 3
Figure 32. Student browsing opportunities 3
Figure 33. Applications tracker 3
Figure 34. Career copilot 3
Figure 35. Settings (every role has this page) 3
Figure 36. Employer: Company 3
Figure 37. Employer: Roles 3
Figure 38. Employer: Pipeline 3
Figure 39. Employer: Talent search 3
Figure 40. College dashboard 3
Figure 41. Placement drives 3
Figure 42. Student home on a phone (390 px wide) 3

PART A

Staff guide

For platform admins and support staff. No coding needed. Every task is step by step with a labelled screenshot. All screenshots use clearly marked demo data, never real people.

A1 Read this first

Kalwings is where freshers show real work, employers see it, and colleges and mentors help in between. As staff you keep that fair and safe: you verify companies, colleges and mentors, review listings, answer questions, and keep the numbers honest.

How to read the screenshots

A red box with a red number marks the thing you should look at. The numbered list under the picture says what each number is. Pictures show **demo data** (names like “Demo Company Pvt Ltd” or “Sample Co. A”). Your live site will show your real records.

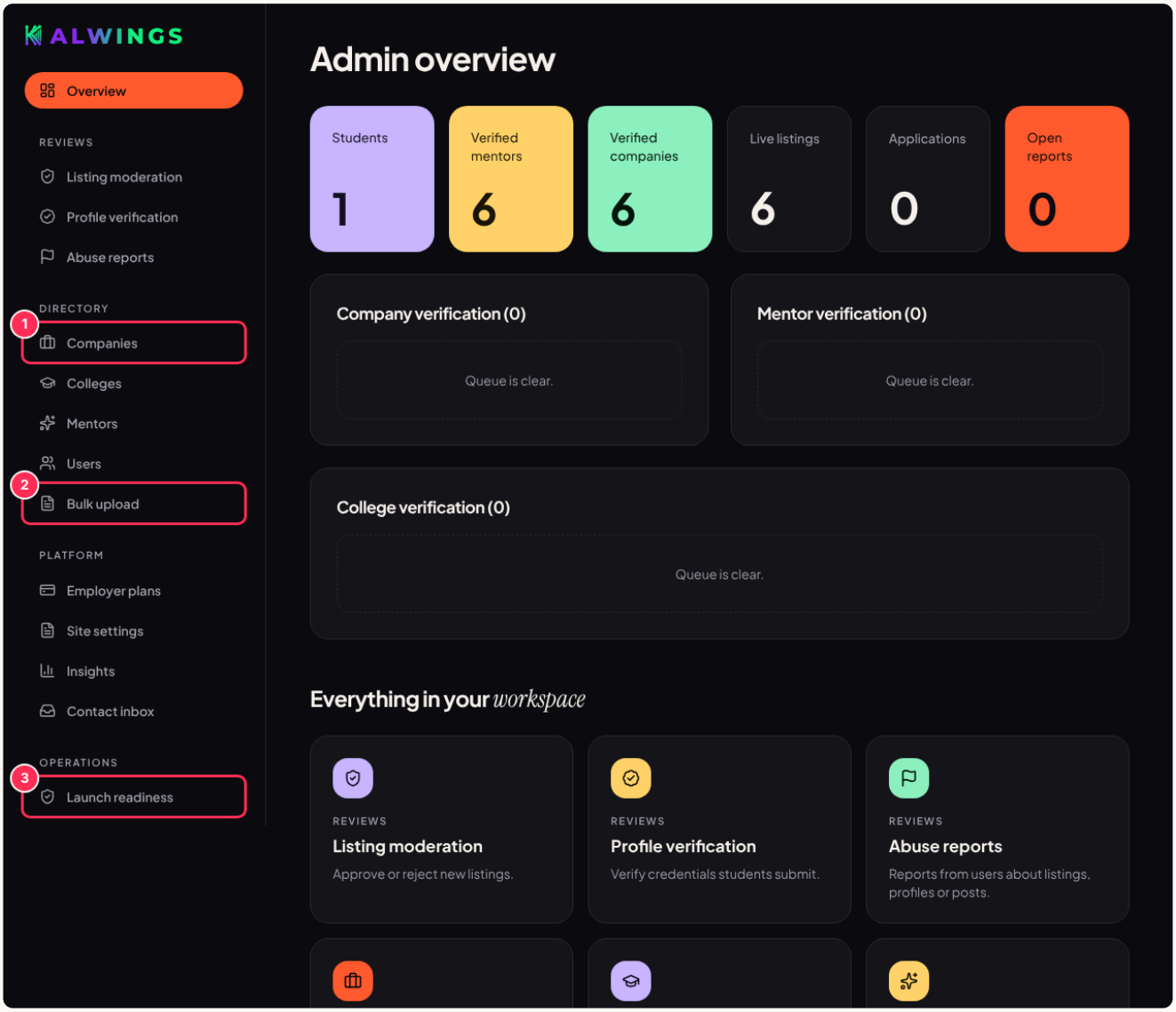


Figure 1. The admin workspace after you sign in

1. **Companies** : add, verify and manage companies (section A6)
2. **Bulk upload** : add many students, companies or mentors from a spreadsheet (section A11)
3. **Launch readiness** : a checklist of what is still missing (section A15)

Five rules that never change

> **Platform rule. 1. Real people only, with consent.** Never add a student, mentor or alumnus unless they agreed to be listed. Never copy a listing from another site; only post roles the employer has agreed to.

Platform rule. 2. Pay, work mode, location and eligibility are always shown on every listing. If a role is unpaid it must say so.

Platform rule. 3. Student contact details stay private unless the student chooses to share them.

Platform rule. 4. No invented numbers, logos, awards or testimonials. If you do not have a real figure, leave it out.

Platform rule. 5. Nobody is auto-rejected, and scores are advice. A person decides about every application. No "guaranteed job" wording anywhere.

Who is who

Role	What they do	How they get in
Student	Builds a profile with projects, applies to roles, takes challenges, books mentors	Joins by email code, or you add them (A11)
Employer (HR / recruiter)	Posts roles and challenges, reviews candidates, runs a pipeline	Joins and waits for verification, or you add them (A6, A9)
Mentor	Offers office-hour slots, gives endorsements	Joins and waits for verification, or you add them (A8)
College admin	Sees consenting students, runs placement drives and reports	Registers a college and waits for verification, or you add them (A7)
Platform admin (you)	Verifies, moderates, adds people, sets plans and settings	Email listed in the server setting ADMIN_EMAILS (A3)

A2 Quick reference: where do I do that?

I want to…	Go to	Section
Verify a new company, college or mentor	Overview (the verification queues) or Companies / Colleges / Mentors	A6 to A8
Approve or reject a new listing	Listing moderation	A13
Add a company, HR person, college or mentor myself	Companies, Colleges, Mentors, Users	A6 to A9
Post a listing for a company	Companies, then open the company	A6
Add 100 students or companies at once	Bulk upload	A11
Change a plan name, price or limits	Employer plans	A12
Put up a banner, change the support email, close sign-ups	Site settings	A14
Change someone's role	Users, then "Change role…"	A9
Decide a role-change request	Users, "Role requests" at the top	A9
Suspend or restore an account	Users, "Suspend…" / "Restore"	A9
See what is still missing before launch	Launch readiness	A15
See who did what	Audit log	A16
Handle a report about a post, listing or person	Abuse reports	A13

I want to…	Go to	Section
Read a message from the contact form	Contact inbox	A13

A3 Signing in and becoming an admin

Kalwings has no passwords by default. You type your email, we send a six-digit code, you type the code. That is the whole sign-in.

Sign in step by step

1. Open **app.kalwings.com/login**.
2. Type your email address in the email box (1 in Figure 2) and press **Send one-time code** (2).
3. Open your inbox. The email arrives within a minute. Copy the six-digit code (it works for 10 minutes and allows 5 tries).
4. Type the code in the code box (1 in Figure 3) and press **Log in** (2).
5. You land on your workspace. As an admin you see the menu shown in Figure 1.

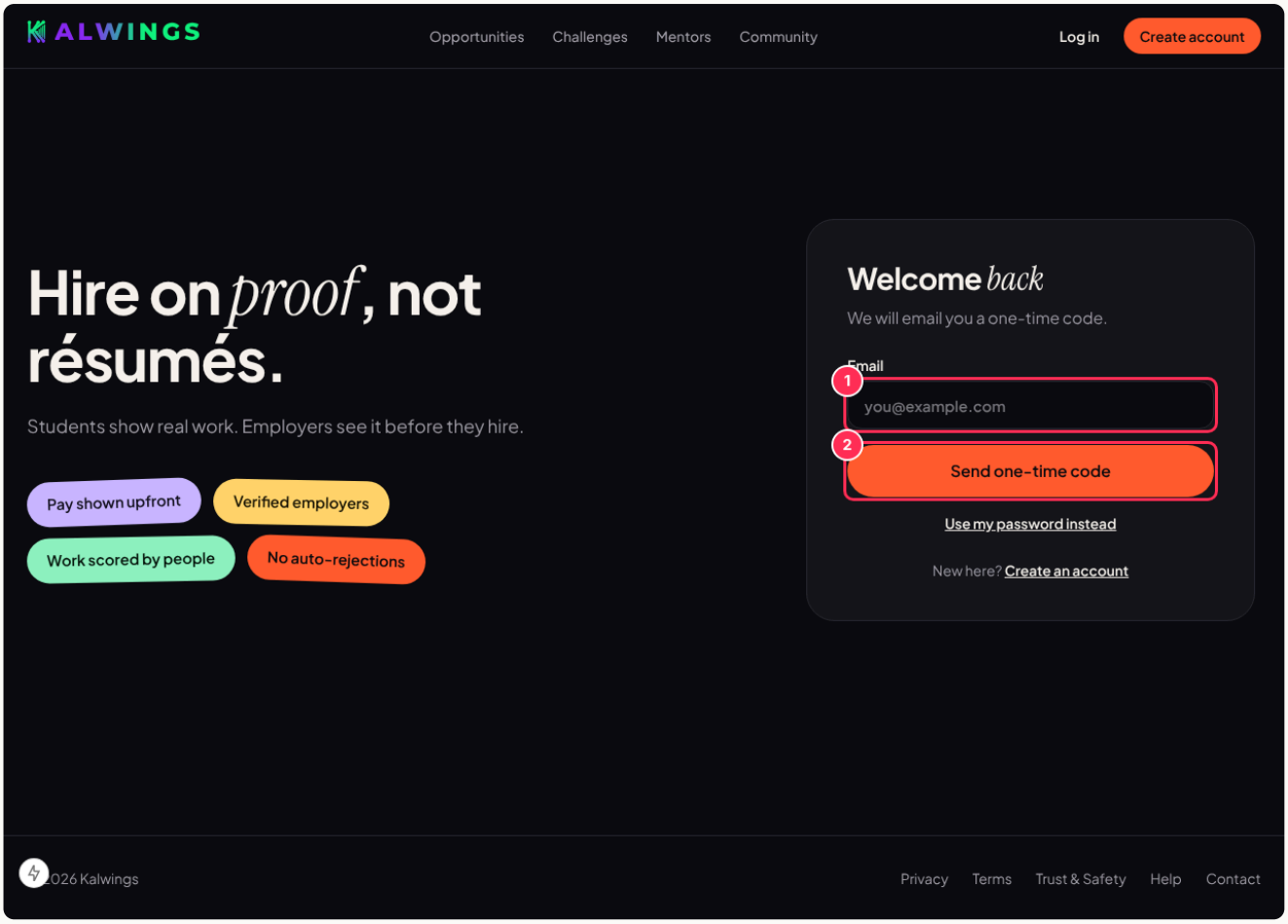


Figure 2. Sign-in page, step 1: ask for a code

1. Your email address
2. Send one-time code

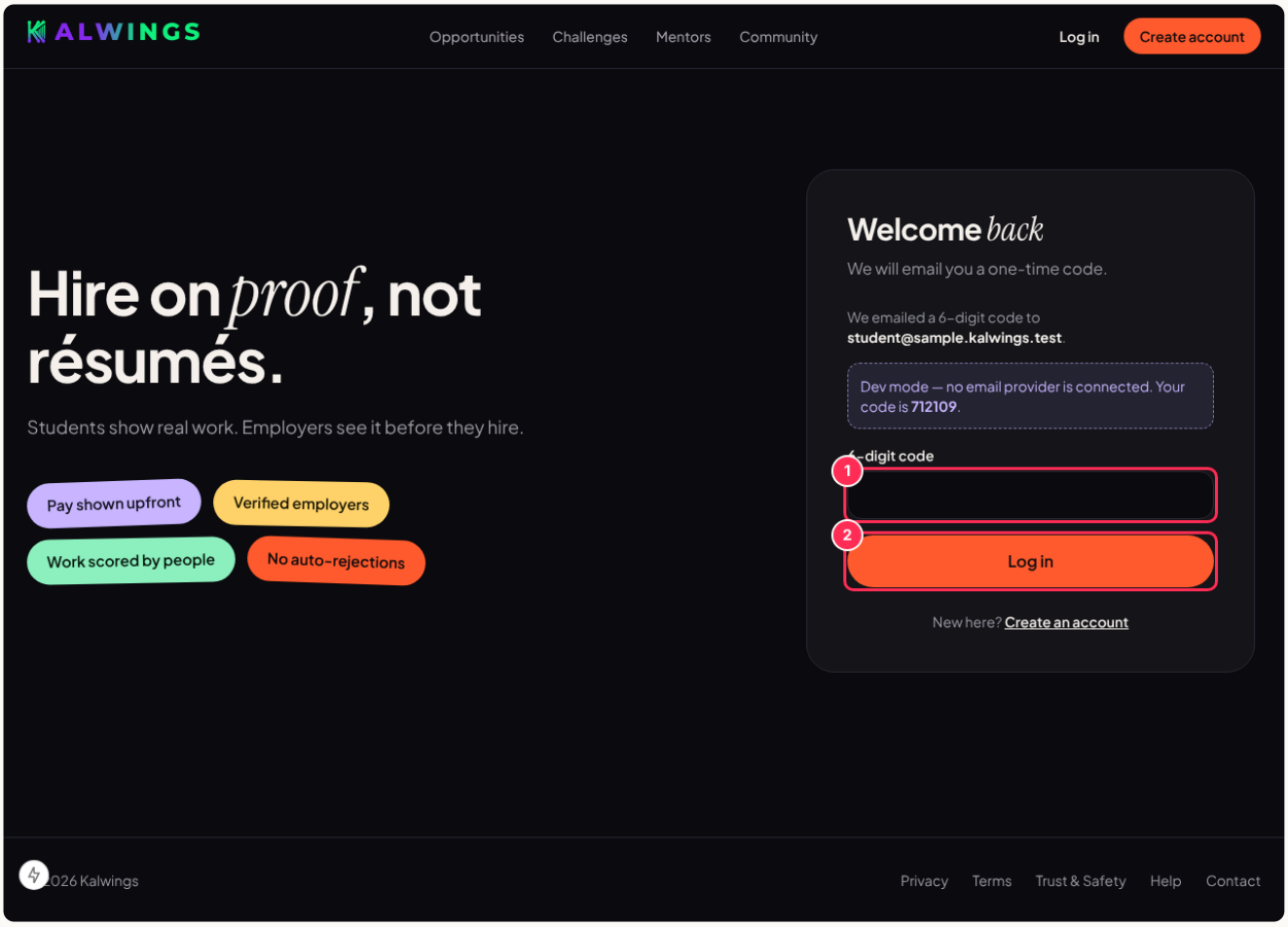


Figure 3. Sign-in page, step 2: enter the code

1. The six-digit code from your email
2. Log in

> **Note.** The screenshot shows a development message (“Dev mode: no email provider is connected”). On the live site no code is shown on screen; it only arrives by email.

If the code never arrives

- Check spam. Sender is the Kalwings mailbox set up on the server.
- Wait one minute and request again. Too many requests lock you out for a while (a safety limit): wait 15 minutes.
- Open **Launch readiness** (if you can still sign in). If “Sign-in emails can be sent” is not ticked, email is not set up on the server and nobody can receive codes. Tell the developer (Part B, chapter B3).

How someone becomes a platform admin

- The very first admin: the developer puts your email in the server setting `ADMIN_EMAILS` and you sign in. You are promoted automatically.
- Later admins: add the person from **Users** (section A9) with the role “Platform admin”, or change an existing user’s role (A9). You will be asked to confirm because admins can see and change everything.
- An admin cannot change their own role, and the last admin cannot be demoted. This stops anyone locking the team out.

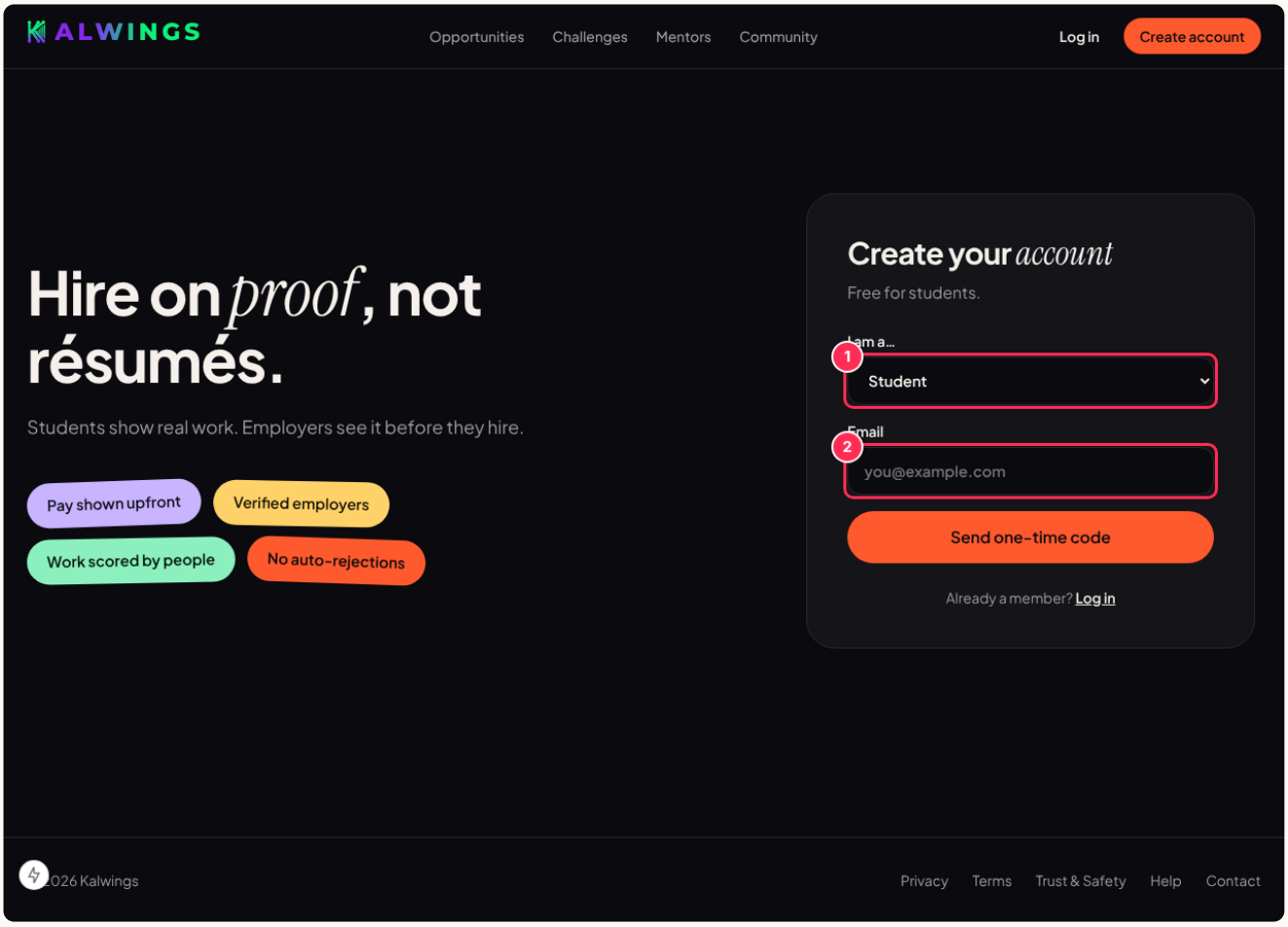


Figure 4. Create-account page: people choose their own role

1. Role the person is joining as (Student, Mentor, Employer, College)
2. Email address

Students are active immediately. New employers, mentors and colleges join a **verification queue** and cannot post, appear in search or run drives until you verify them.

A4 A tour of the admin workspace

The left menu is grouped. It is generated from one list in the code, so what you see is always what exists.

Group	Item	What it is for
Reviews	Listing moderation	Approve or reject new listings before students see them
Reviews	Profile verification	Check evidence links students attach to their profile entries
Reviews	Abuse reports	Reports from users about listings, profiles or community posts
Directory	Companies	Add, verify, edit companies; set plan; add HR; post listings for them
Directory	Colleges	Add and verify the colleges students can choose from
Directory	Mentors	Add mentors with every detail; edit; upload photo
Directory	Users	Add people, change roles, decide role requests, suspend or restore
Directory	Bulk upload	Add many students, companies or mentors from Excel or CSV
Platform	Employer plans	Rename plans, change prices and limits, assign plans to companies
Platform	Site settings	Announcement banner, support email, who can sign up

Group	Item	What it is for
Platform	Insights	Real counts and trends across the platform
Platform	Contact inbox	Messages from the website contact form
Operations	Launch readiness	What still needs doing, read live from your data
Operations	Audit log	Who did what and when
Operations	Server errors	Errors recorded by the server (for the developer)

At the bottom of the menu: **Notifications** (alerts for you), **Settings** (your own account: appearance, password login, role request, data download) and **Log out**.

A5 The Overview page and verification queues

Overview is your inbox. Counters at the top show students, verified mentors, verified companies, live listings, applications and open reports. Below are the queues: Company verification, Mentor verification and College verification. Each waiting item has **Verify** and **Reject** buttons.

How to verify a company from the queue

1. Open **Overview**.
2. Read the company name and website shown in **Company verification**. Open the website in another tab: is it a real business and does the person who registered work there? (Check the email domain matches the website.)
3. Press **Verify** if it is genuine, or **Reject** if not. Verification takes effect at once; the company can then post and search. Both actions are written to the audit log.

> **Careful.** Never verify a company just because the form looks complete. Verification is the trust promise of the platform. If unsure, leave it in the queue and email them first.

A6 Companies: add, verify, edit, post for them

Open **Directory → Companies**. You can add a company yourself, which is useful when an employer has agreed by phone or email and you want them live today.

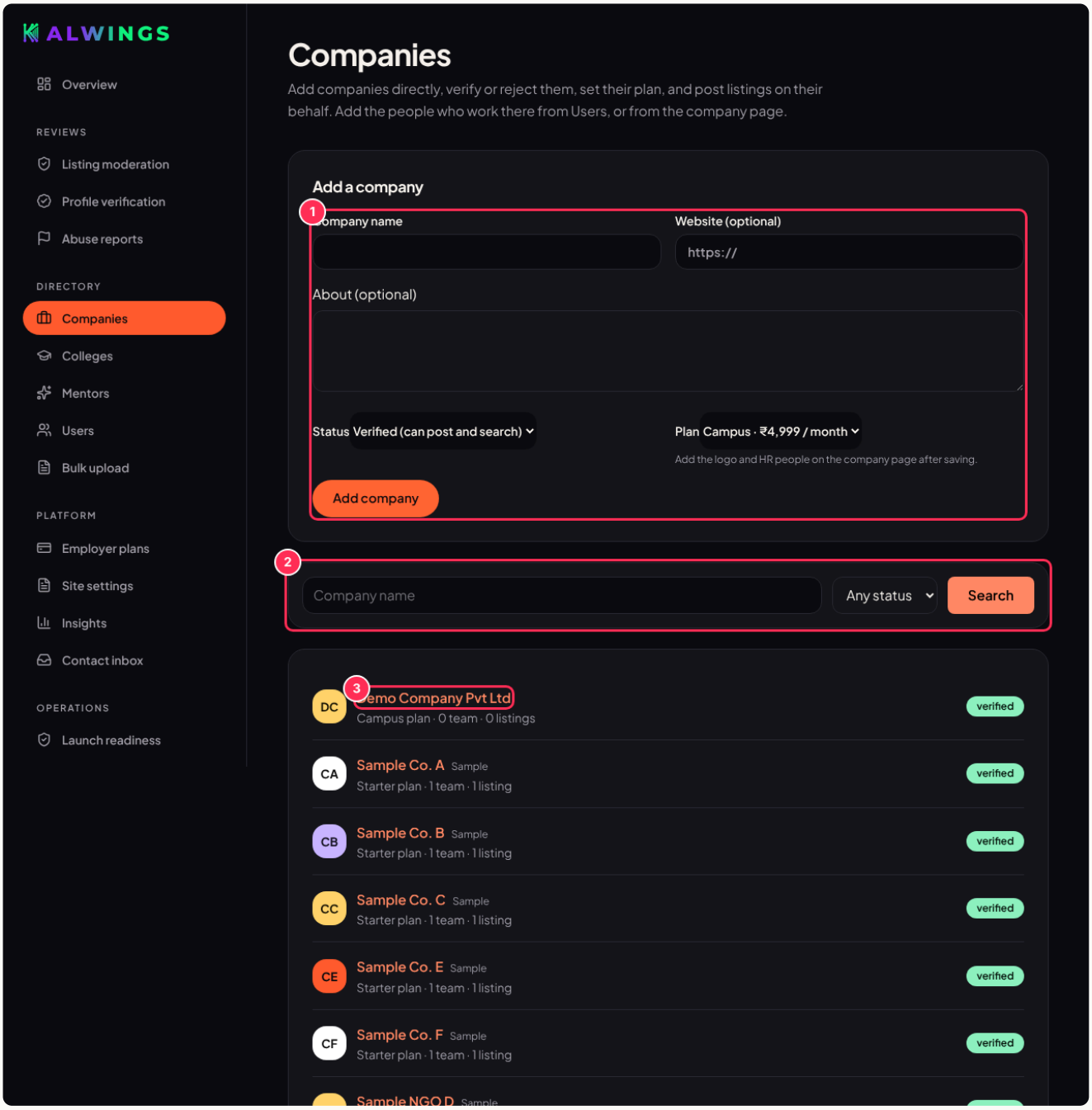


Figure 5. The Companies page

1. **Add a company** form
2. Search by name and filter by status
3. A company: click the name to open it

Add a company

1. Fill **Company name** (required). Names must be unique. If it already exists you will be told.
2. Optional: **Website** (must start with http:// or https://) and **About**.
3. Choose **Status**. “Verified” lets them post and search at once. Use “Pending” if you have not checked them yet.
4. Choose their **Plan** (A12). The price shown helps you pick the right one.
5. Press **Add company**. A green confirmation appears.

Open a company

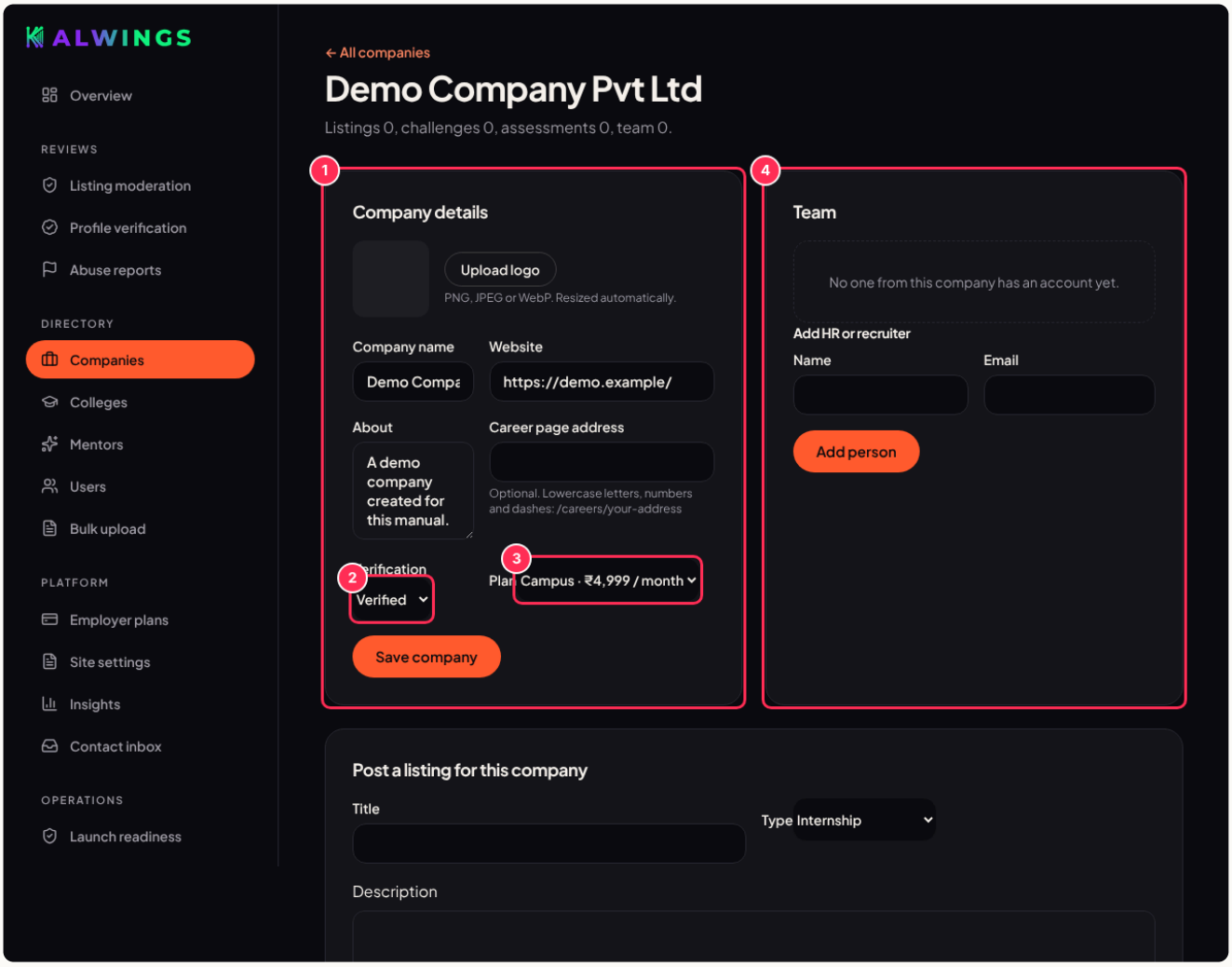


Figure 6. A company page

1. **Company details** : logo, name, website, about, career-page address, verification, plan
2. **Verification** : Verified, Pending or Rejected
3. **Plan** : which plan limits apply
4. **Team** : who works there, and a form to add HR

- **Logo:** press “Upload logo”, choose a square PNG, JPEG or WebP (it is resized automatically; the limit is 1 MB). The logo then shows on listings, the career page and candidate applications.
- **Career page address:** lowercase letters, numbers and dashes. The company’s public page becomes /careers/that-address.
- **Save company** applies every change at once.
- **Add HR or recruiter:** type their name and email and press **Add person**. They are added to this company as a verified recruiter. They sign in with an emailed code. Nothing else needs doing.

Post a listing for a company

Use this when the company has agreed to a role and asked you to post it. It is available only for **verified** companies (a listing from an unverified company would never be visible).

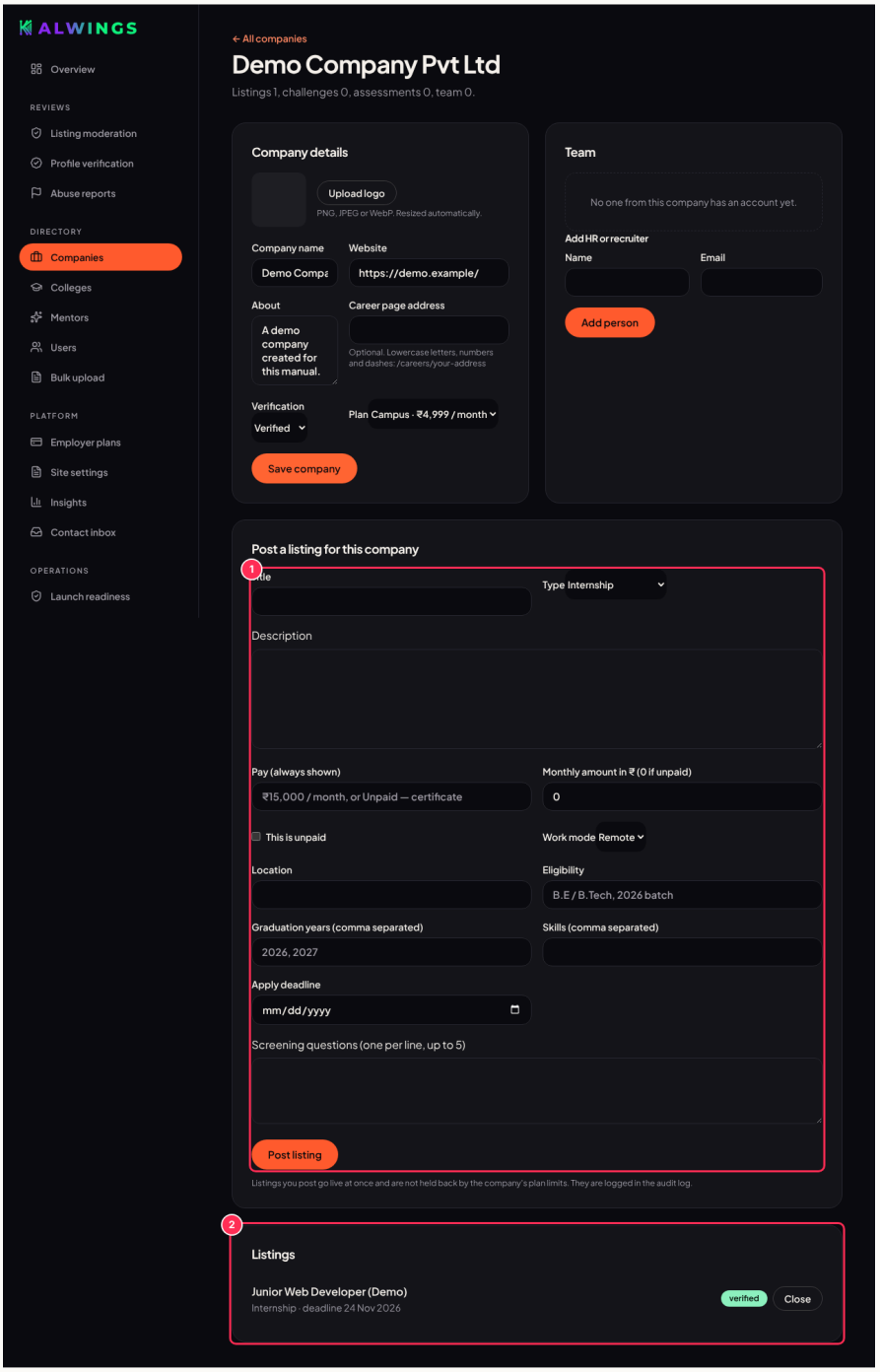


Figure 7. Posting a listing and managing listings

1. **Post a listing** form
2. **Listings** already posted, with Close or Reopen

1. Open the company and scroll to **Post a listing for this company**.
2. Fill every field: Title, Type (Internship, Fresher job, Graduate trainee, Apprenticeship, Campus hiring, Freelance), Description (at least 30 characters: describe the work, the team and how selection works), Pay, Monthly amount, Work mode, Location, Eligibility, Apply deadline.
3. **Pay is mandatory.** If the role is unpaid tick “This is unpaid” and say what the student gets instead (for example “Certificate only”). Paid roles must state an amount with a number in it, e.g. “₹15,000 / month”.
4. Optional: graduation years (2026, 2027), skills, and up to five screening questions (one per line).

5. Press **Post listing**. It goes live immediately (you are the moderator) and does not count against the company's plan limits. The action is logged.

> **Tip.** To take a role down, open the company, find the listing and press **Close**. **Reopen** brings it back. Challenges are not posted from here: they need a brief and a rubric, so the employer creates them under Challenges.

A7 Colleges

Students pick their college from the list of **verified** colleges. Open **Directory → Colleges**.

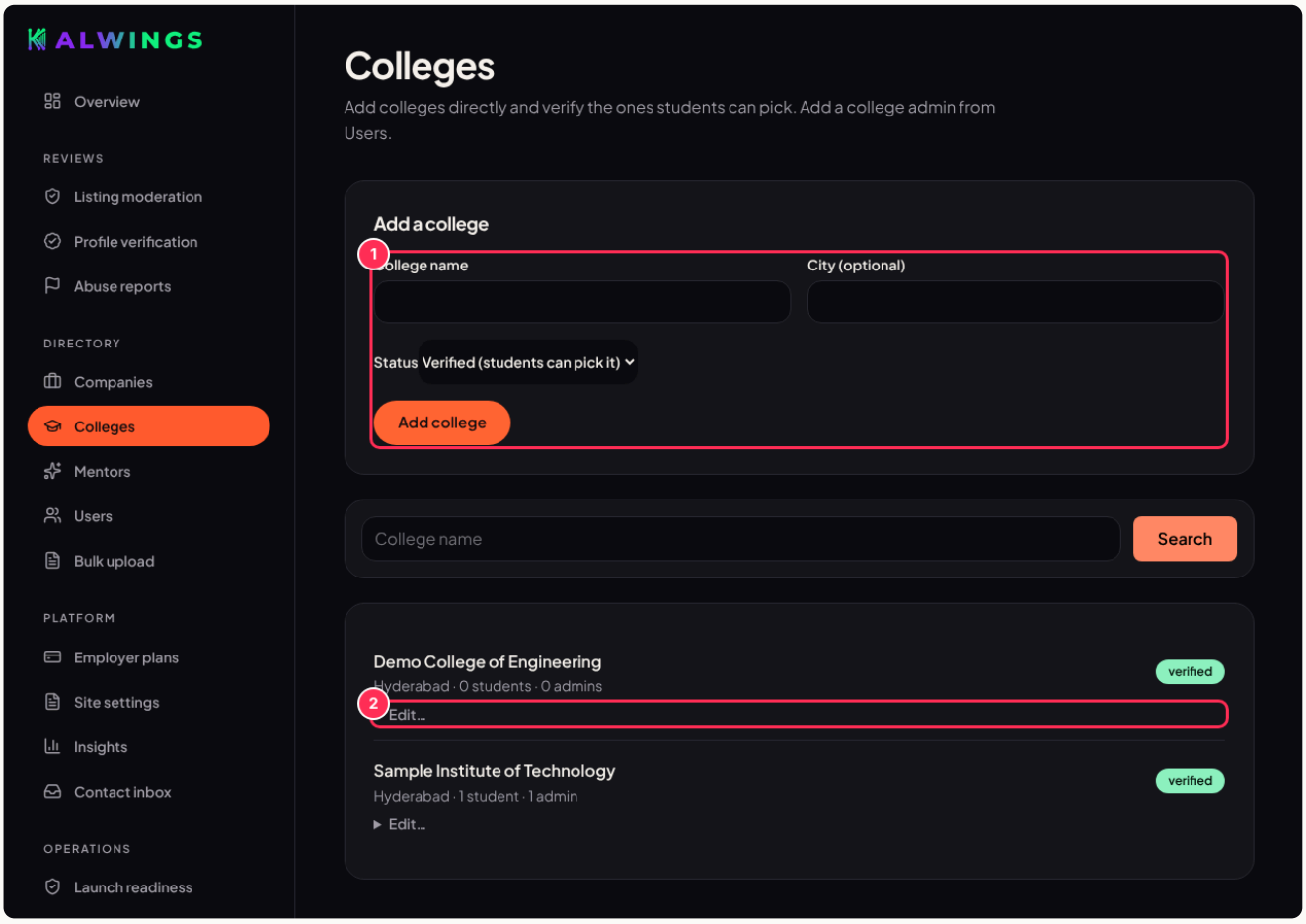


Figure 8. The Colleges page

1. **Add a college** : name, city, status
2. **Edit...** opens the edit form for that college

1. Type the **College name** exactly as students know it (names must be unique) and the **City**.
2. Leave **Status** on "Verified" so students can select it, or choose "Pending".
3. Press **Add college**.

To edit, press **Edit...** beside a college, change the fields and press **Save college**. The row also shows how many students and admins belong to it. To give a college an administrator, add them from **Users** with the role "College admin".

> **Note.** College admins only ever see students who chose to share their progress with the college. Students decide that themselves in their profile.

A8 Mentors

A mentor is a working professional who offers office-hour slots. Only add people who agreed. Open **Directory → Mentors**.

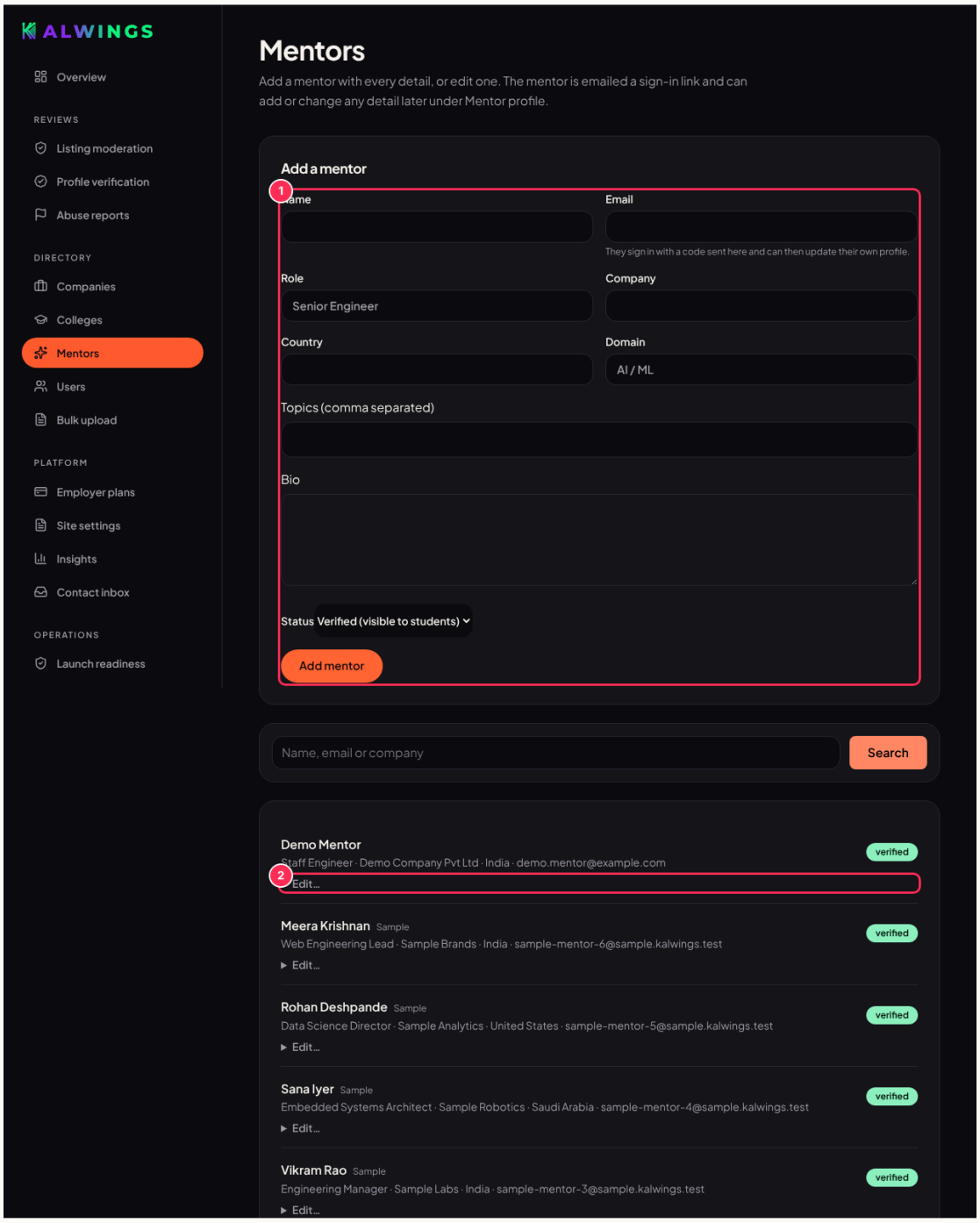


Figure 9. The Mentors page

1. **Add a mentor** form with every profile detail
2. **Edit…** : change any detail or upload a photo

Add a mentor with every detail

1. Fill Name, Email (this becomes their sign-in), Role (for example “Staff Engineer”), Company, Country and Domain (for example “Backend” or “AI / ML”).
2. Optional but recommended: Topics (comma separated) and Bio.

3. Choose **Status**: “Verified” makes them visible to students at once.
4. Press **Add mentor**. The mentor is emailed how to sign in (if email is configured; otherwise tell them yourself).

To change a mentor later, press **Edit...**, update the fields (the email cannot be changed here) and press **Save mentor**. In the same place you can **Upload photo** on their behalf.

What the mentor does next

The mentor signs in with their email, opens **Mentor profile** and can add or change any detail themselves: role, company, country, domain, topics, bio and photo, and add office-hour slots. They cannot change their own verification status; only you can.

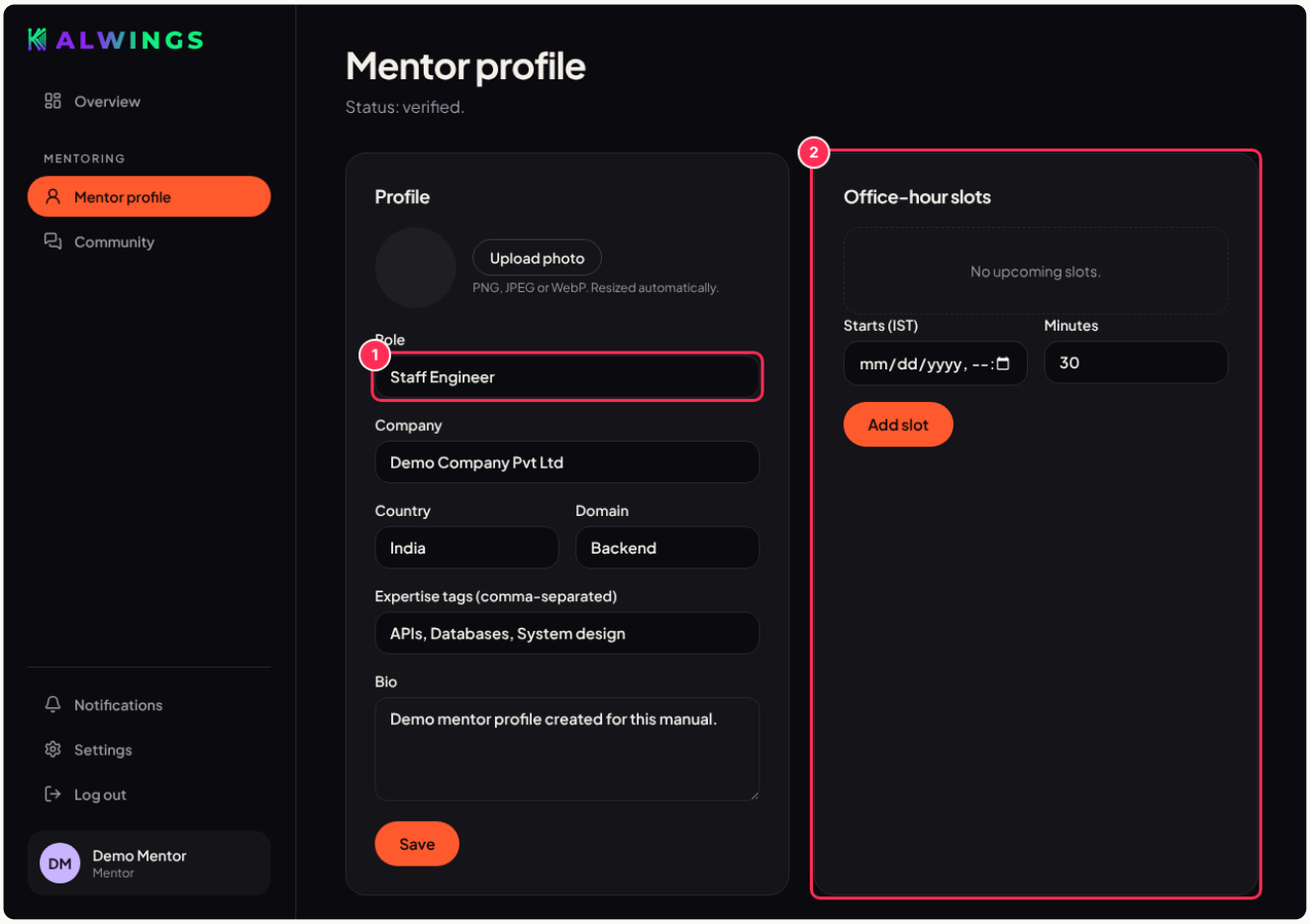


Figure 10. What the mentor sees in Mentor profile

1. The mentor can edit every profile detail
2. Office-hour slots: when students can book time

A9 Users: add people, change roles, suspend

Open **Directory → Users**. This is where you add any kind of account, move people between roles, answer role requests and handle accounts that break the rules.

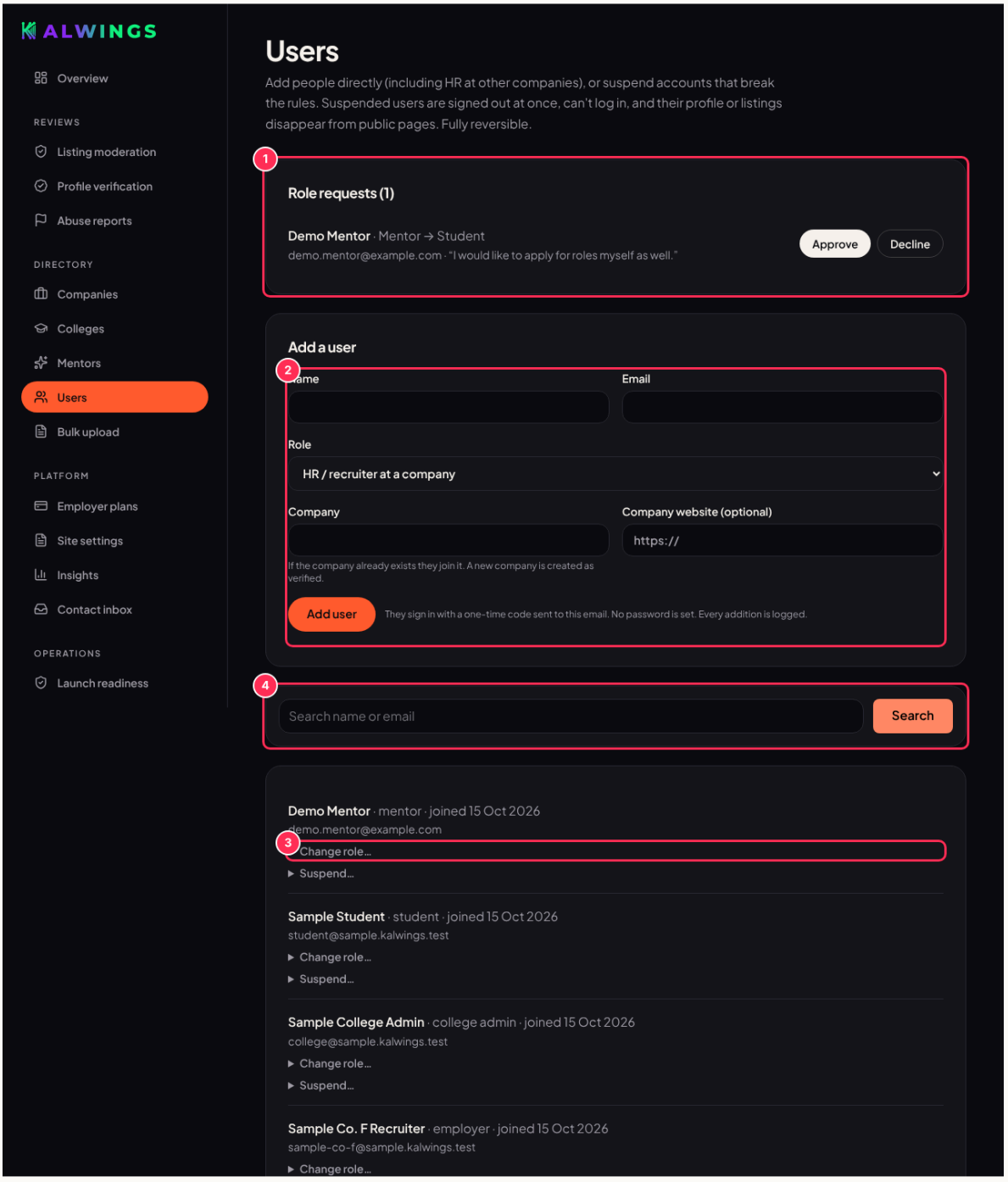


Figure 11. The Users page

1. **Role requests** : people asking to change role, with Approve and Decline
2. **Add a user** : any role, including HR at another company
3. **Change role…** on any account
4. Search by name or email

Add a user

1. Enter Name and Email.
2. Choose the Role: Student, Mentor, HR / recruiter at a company, College admin or Platform admin.
3. For HR: type the Company (an existing company is joined; a new one is created as verified) and optionally its website. For a college admin: type the College and optionally the city (created as verified if new).
4. Press **Add user**. The person receives an email saying how to sign in. There is no password.

Careful. Choosing “Platform admin” asks you to confirm. Admins can see and change everything. Add only people on your own team.

Change someone’s role

1. Search for the person, or find them in the list.
2. Press **Change role…** under their name, pick the new role and press **Change role**.
3. They get a notification. Their account and all their data stay exactly as they were; only what they can open changes. (If someone becomes a student and never had a student profile, an empty one is created for them.)

Note. You cannot change your own role, and the last remaining admin cannot be demoted. Ask another admin if you need your own role changed.

Role-change requests from users

Any signed-in person can ask for a different role from their own Settings page (shown in section A17). Their request appears at the top of **Users** under “Role requests” with their reason.

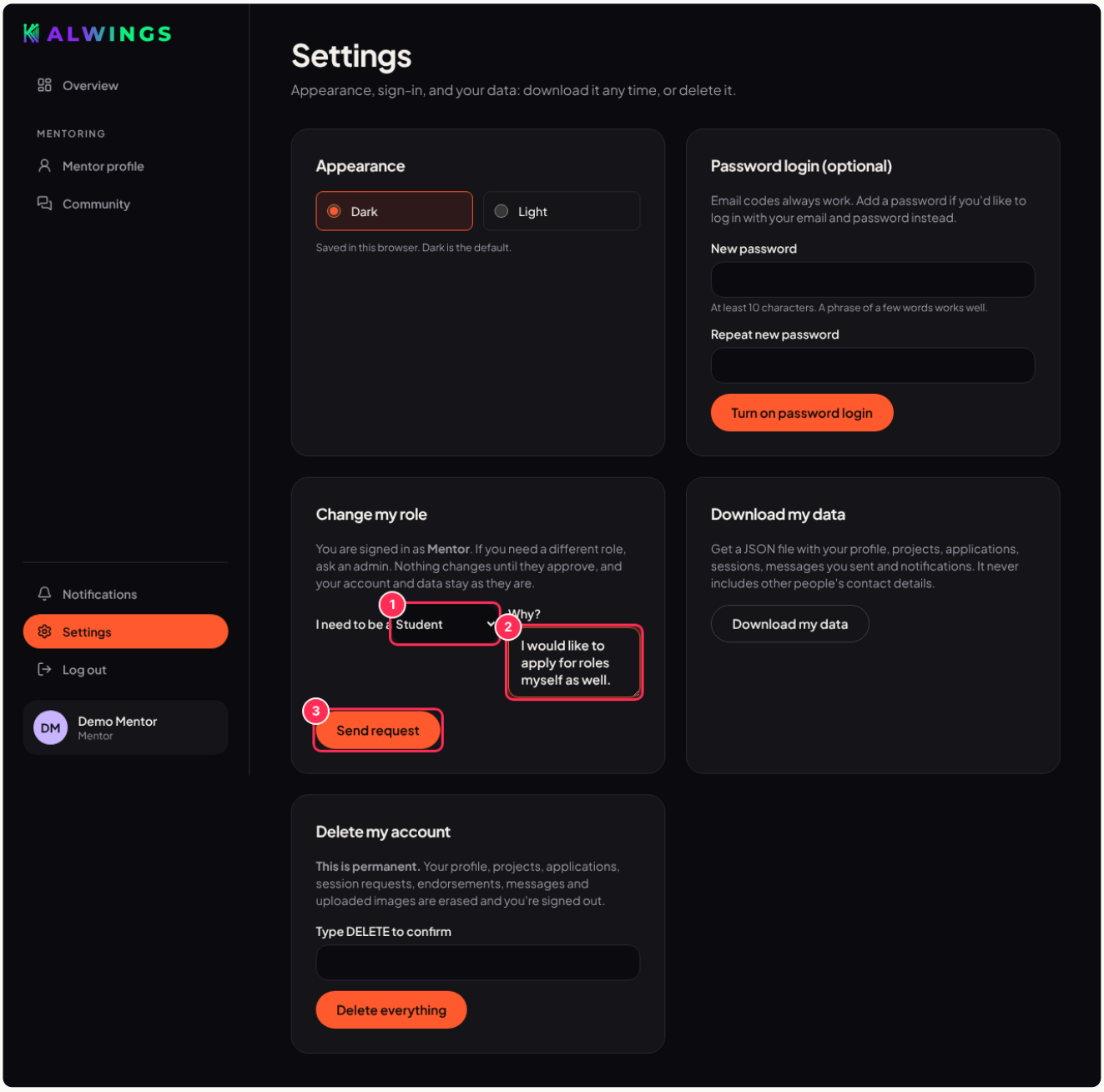


Figure 12. What the user does: Settings → Change my role

1. Choose the role they need
2. Explain why (at least 10 characters)
3. Send request

1. Read the person, the old and new role and the reason.
2. Press **Approve** to apply the change at once, or **Decline**. Approving “Platform admin” asks you to confirm.
3. They are notified of the outcome. Nothing changes until you approve. A user can have only one waiting request and can withdraw it.

Suspend and restore

Use this for abuse, fake accounts or legal requests. Suspending signs the person out everywhere at once, blocks sign-in, and hides their profile and listings from public pages. It does **not** delete anything and is fully reversible.

1. Find the person. Press **Suspend...**.

2. Write a clear **reason** (at least 5 characters). The person is sent this reason.
3. Press **Suspend account**. Platform admins cannot be suspended here.
4. To undo, find the person again and press **Restore**.

> **Platform rule.** Admin screens never delete a person. If someone asks to be erased, they use **Settings → Delete my account** themselves, which removes their profile and data.

A10 Search, filters and notifications

Every Directory page has a search box. Companies also filter by status. Searching never changes anything. **Notifications** (bottom of the menu) lists alerts: new role requests, new verification requests, and replies. Unread items are counted on the menu.

A11 Bulk upload from Excel or CSV

Use bulk upload when you have many records: a college's batch of students, a list of partner companies, a group of mentors. Open **Directory → Bulk upload**. There is one panel each for Students, Companies and Mentors.

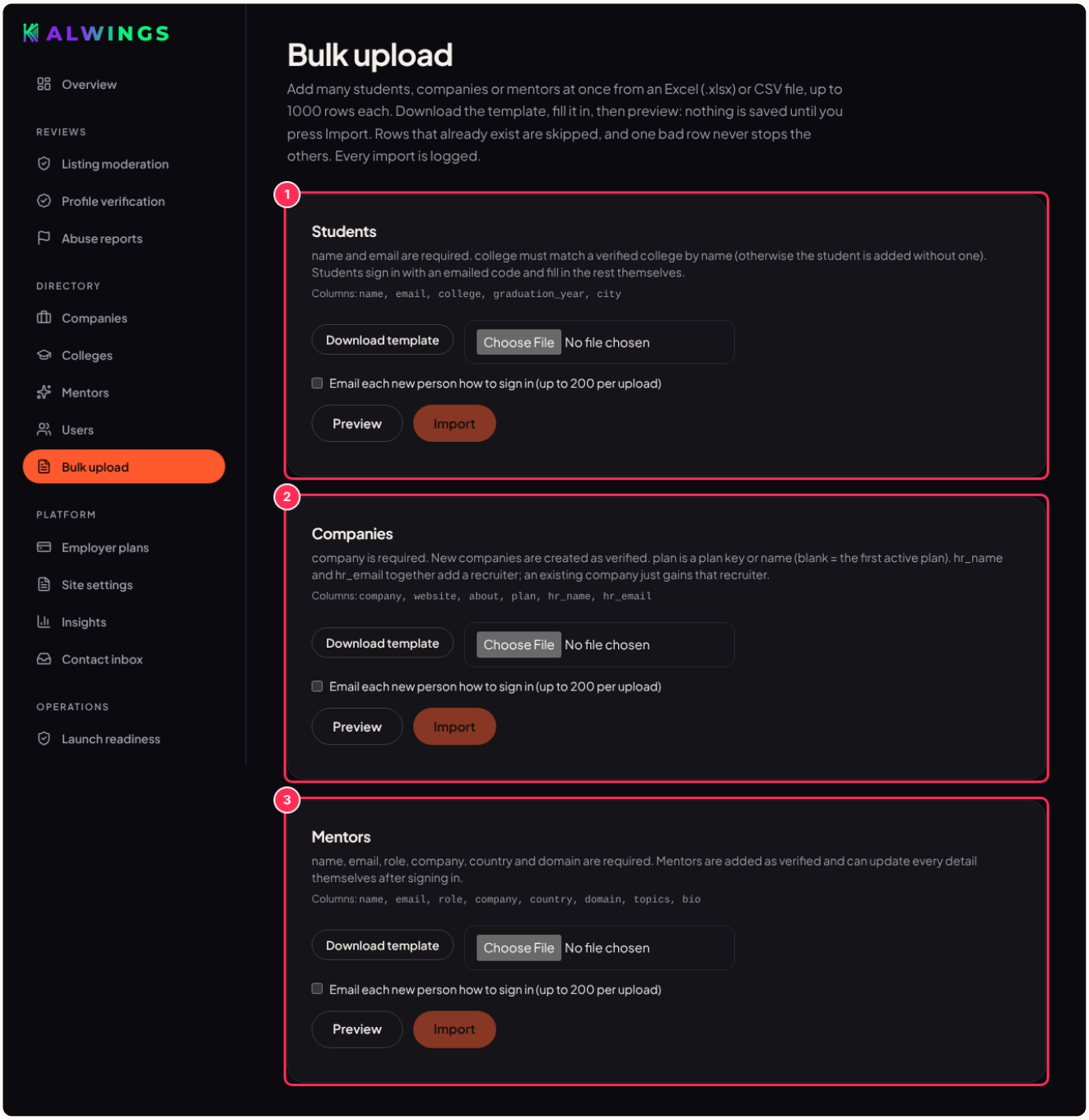


Figure 13. The Bulk upload page

1. Students panel
2. Companies panel
3. Mentors panel

The safe workflow

1. In the right panel press **Download template**. It opens in Excel: the first row holds the column names. Keep that row exactly as it is.
2. Fill one person or company per row. Save as **.xlsx** or **.csv** (the first sheet is used; up to 1,000 rows and 2 MB per file; split larger lists).
3. Press **Choose File** and pick your file.
4. Press **Preview**. Nothing is saved yet. You see every row marked ✓ ready, – skipped, or ✕ error with the reason.

5. Fix any errors in your file and preview again, or leave them out.
6. Press **Import**. Only the ready rows are added. You get a summary and every import is written to the audit log.

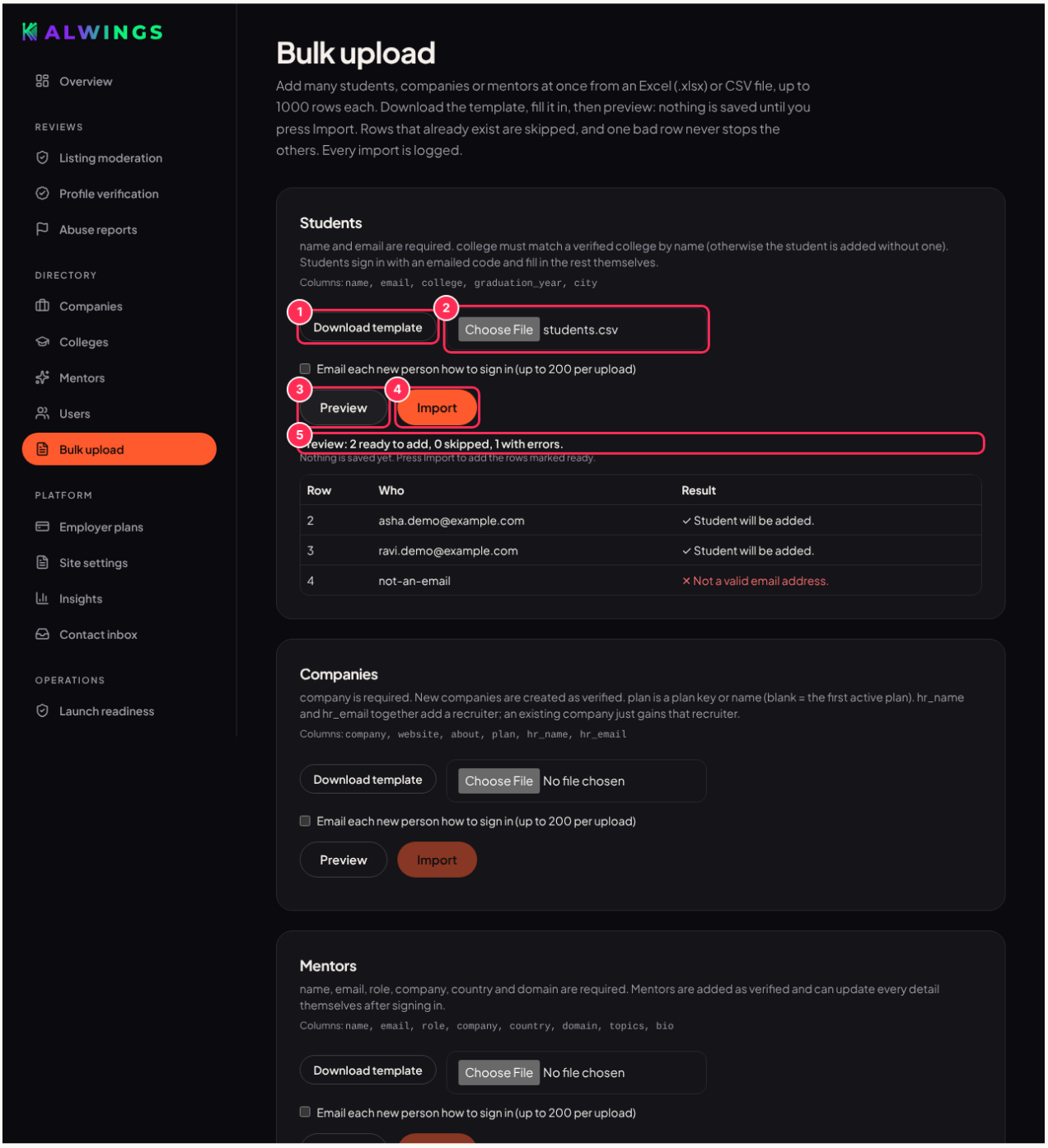


Figure 14. Preview before import

1. Download the template for this kind of record
2. Choose the file
3. Preview: shows what would happen, saves nothing
4. Import: adds the ready rows
5. Summary: how many ready, skipped and with errors

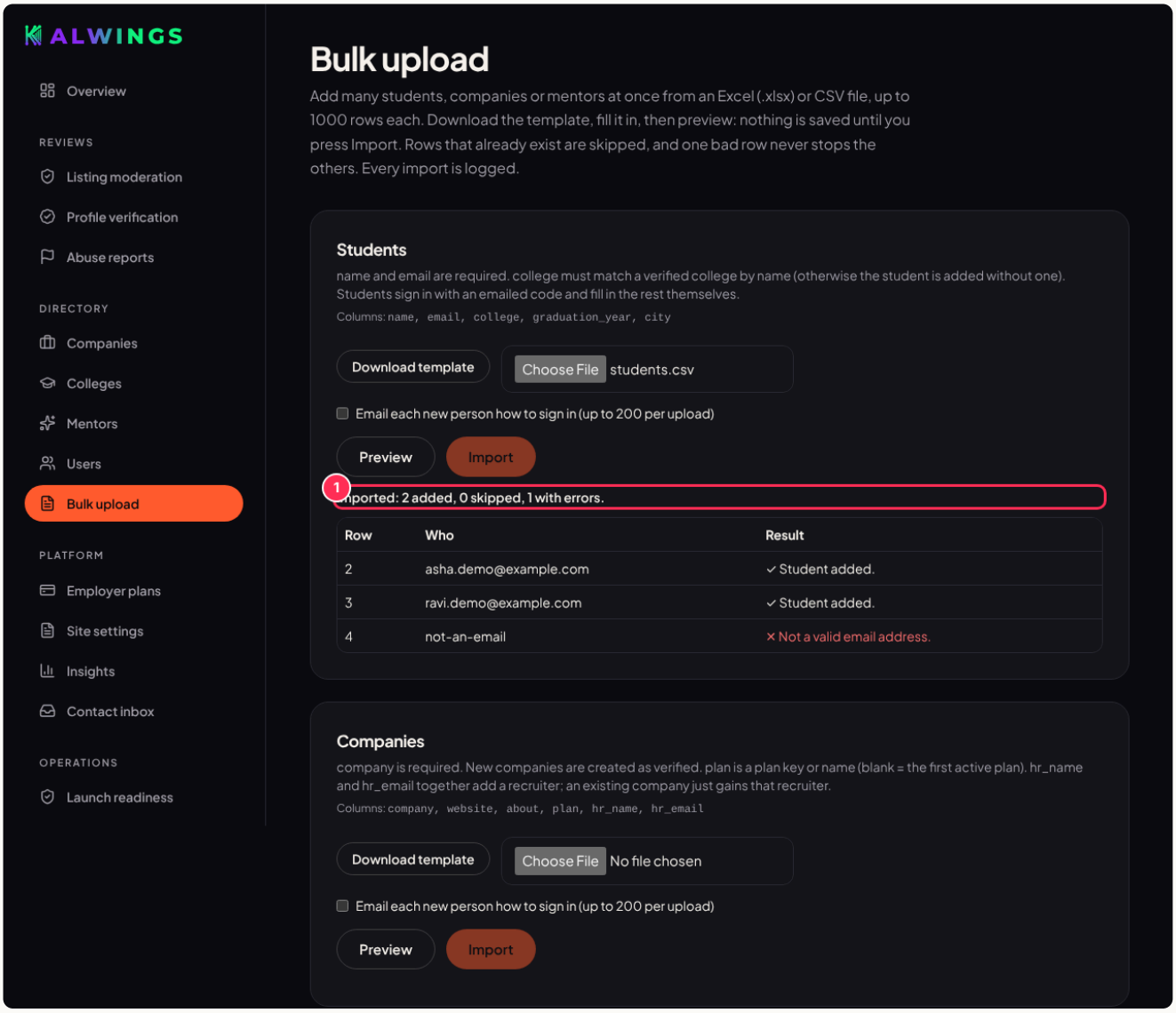


Figure 15. After pressing Import

1. The summary now says “Imported”. The table lists every row’s result

What each template needs

Panel	Columns	Required	Rules
Students	`name, email, college, graduation_year, city`	name, email	college must match a **verified** college by name (otherwise the student is added without a college and the row says so). graduation_year like 2026. Students sign in with an emailed code and fill in the rest.
Companies	`company, website, about, plan, hr_name, hr_email`	company	New companies are created **verified**. plan is a plan key or name; blank = the first active plan. hr_name and hr_email go together: they add a recruiter. If the company already exists the recruiter is added to it.
Mentors	`name, email, role, company, country, domain, topics, bio`	name, email, role, company, country, domain	Added as **verified**. topics is comma separated. Mentors edit every detail themselves after signing in.

What “skipped” and “error” mean

- **Skipped:** the person or company already exists, or appears twice in your file. Nothing is changed for that row. Uploading the same file again adds nobody twice.

- **Error:** something in that row is wrong, with the reason shown (not a valid email, name too short, unknown plan, website without http://). Fix it and upload again. One bad row never blocks the others.

Emailing new people

Tick “Email each new person how to sign in” before importing if you want everyone told straight away (up to 200 emails per upload). Leave it unticked to import quietly and tell people yourself, for example if you want to announce on a given day. With it unticked nobody is emailed.

> **Careful.** Only upload people who agreed to be listed. Bulk upload creates real accounts. The file is not stored after the preview or import; keep your own copy.

A12 Employer plans: rename, reprice, add

A plan limits how much an employer can have at once: active listings, open challenges, active assessments and team members. There is no online payment yet, so you set each company’s plan by hand. Open **Platform → Employer plans**.

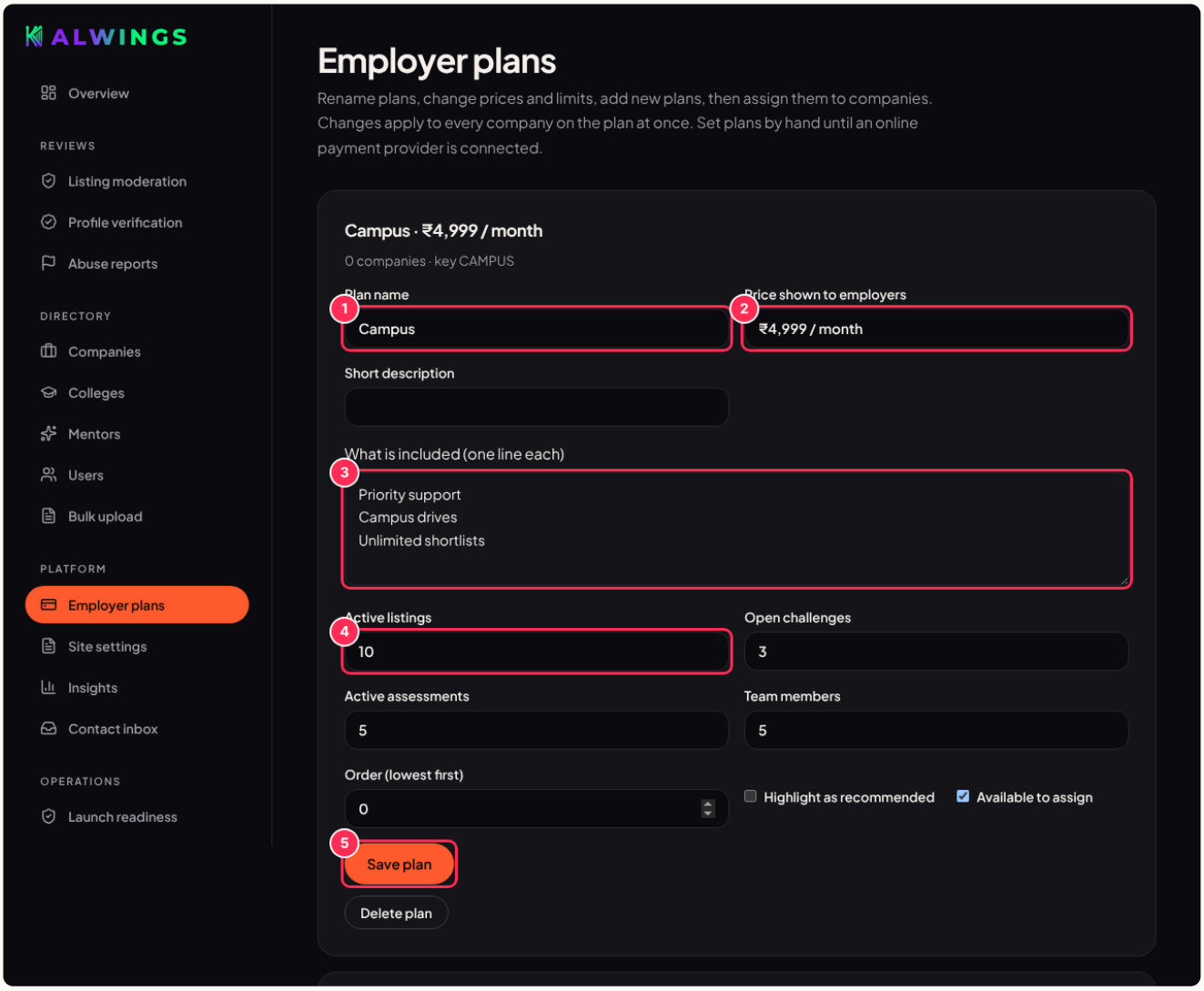


Figure 16. Editing a plan

1. **Plan name** : what employers see
2. **Price shown to employers** : free text such as “₹2,999 / month”, “Free” or “Contact us”
3. **What is included** : one line per feature
4. The four limits
5. **Save plan** applies at once to every company on the plan

Change an existing plan

1. Find the plan panel (the heading shows name, price and how many companies use it).
2. Edit the name, price text, description, features or limits.
3. Tick **Highlight as recommended** to mark the plan employers should pick. Untick **Available to assign** to retire a plan without deleting it (at least one plan must stay available).
4. Press **Save plan**. Companies already on the plan see the new name and limits immediately.

Add a new plan

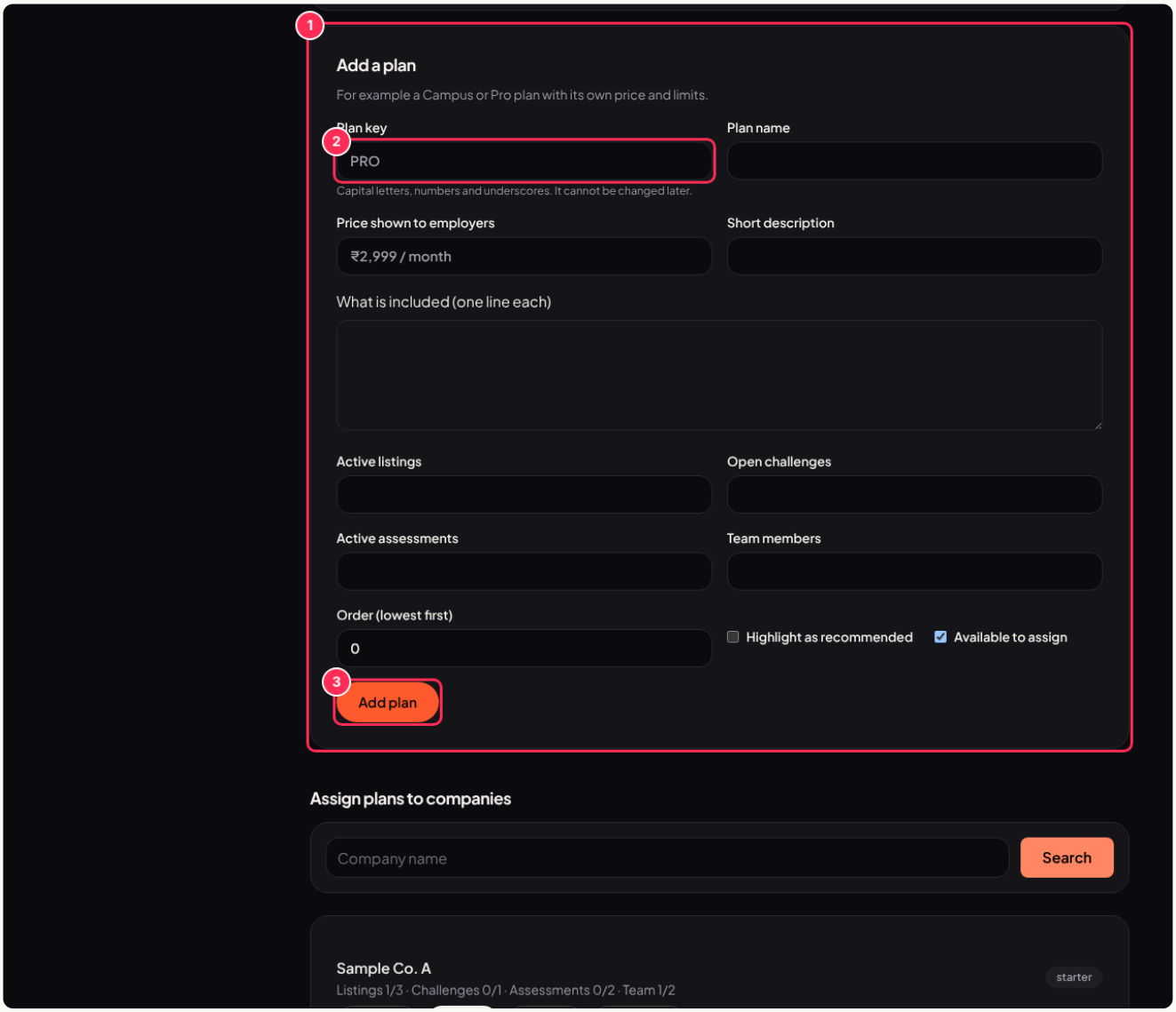


Figure 17. Adding a plan

1. The **Add a plan** panel
2. **Plan key** : 2 to 40 capital letters, numbers or underscores (for example CAMPUS). It cannot be changed later
3. Add plan

1. Choose a short capital-letter **Plan key** that is not already used.
2. Fill name, price text, features and limits. Order controls the sequence (lowest first).
3. Press **Add plan**.

Assign a plan to a company

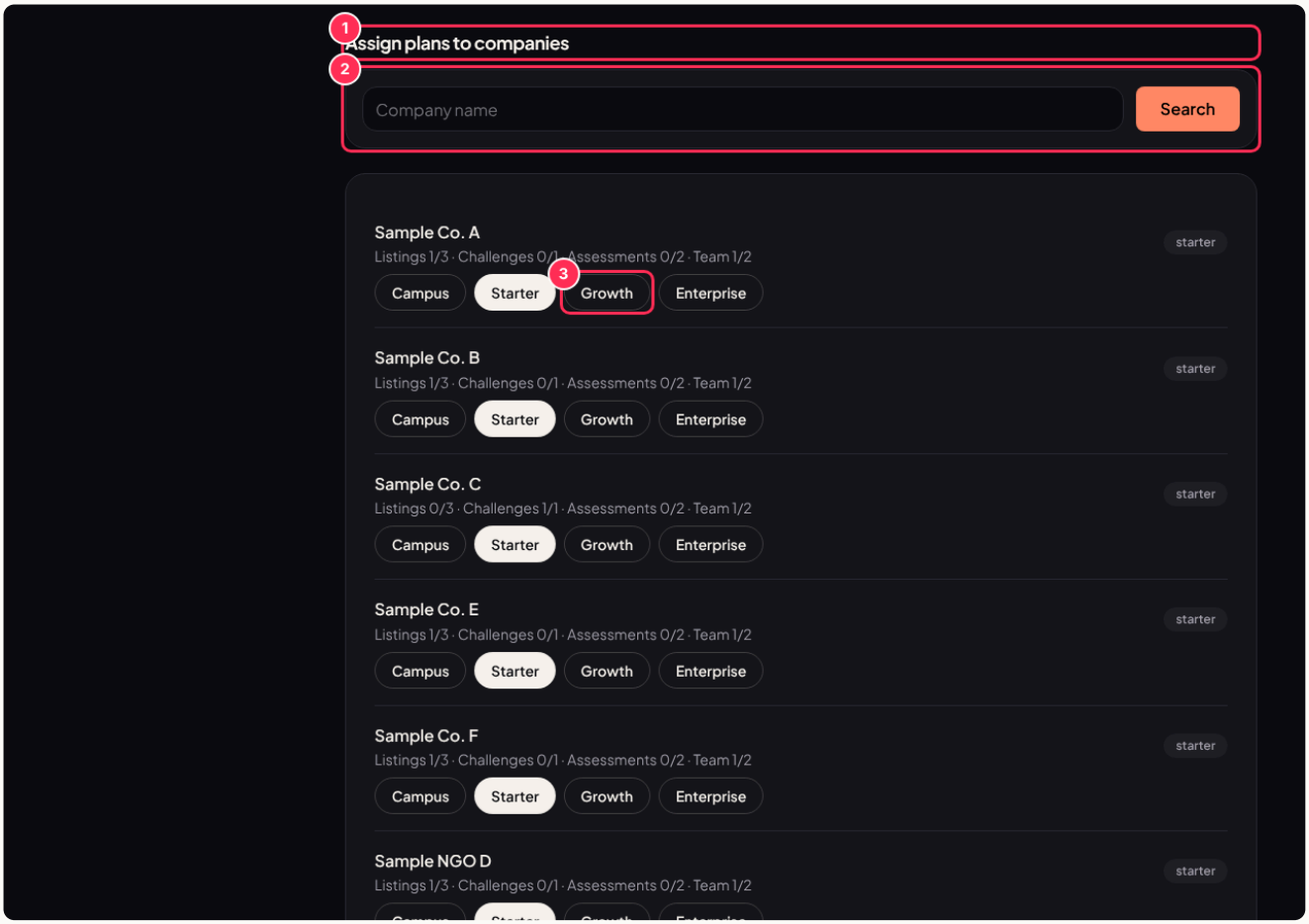


Figure 18. Assigning plans

1. The assign section
2. Search a company by name
3. One button per plan: the dark one is the current plan; press another to switch

Each company shows its usage (for example “Listings 1/3”). Pressing a plan button switches that company at once and notifies its team. You can also change the plan on the company page (A6).

> **Tip. Delete plan** works only when no company is on the plan and at least one other plan exists; otherwise switch companies first or just un-tick “Available to assign”.

A13 Reviews: listings, credentials, reports, inbox

Listing moderation

Listings posted by employers wait here. Check them against the rules: pay, mode, location and eligibility are present and honest; no “guaranteed job”; no fee asked from students; nothing misleading. Press **Approve** to publish or **Reject** with a reason.

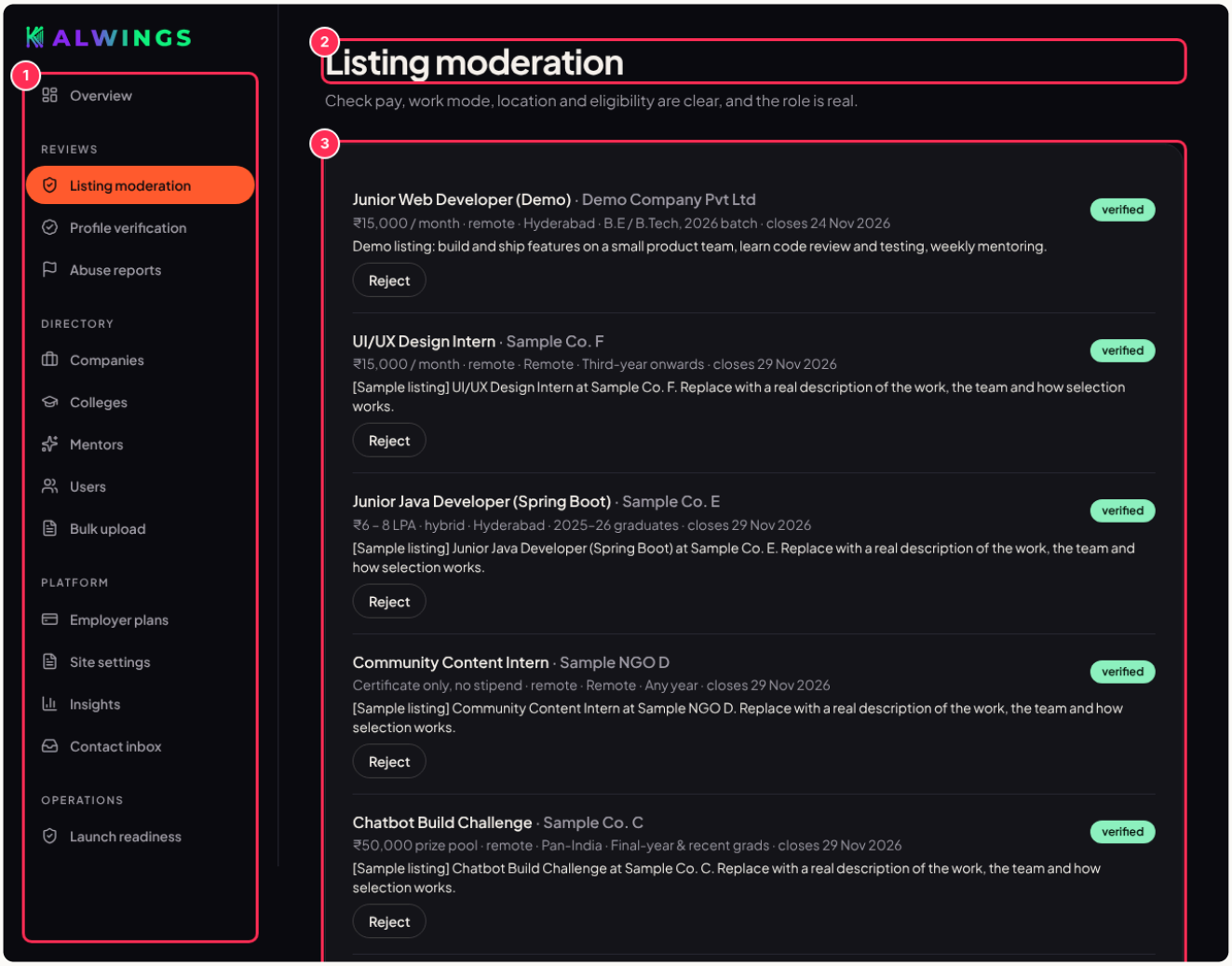


Figure 19. Listing moderation

1. Navigation
2. Page title
3. The listings, each with Approve / Reject

Profile verification

Students attach evidence links (a certificate, a project, a repository) to profile entries and can request verification. Open the link, confirm it proves what the student claims, then press **Verify**. Only you can mark an entry as verified; the student's own badge never says "verified" without this step.

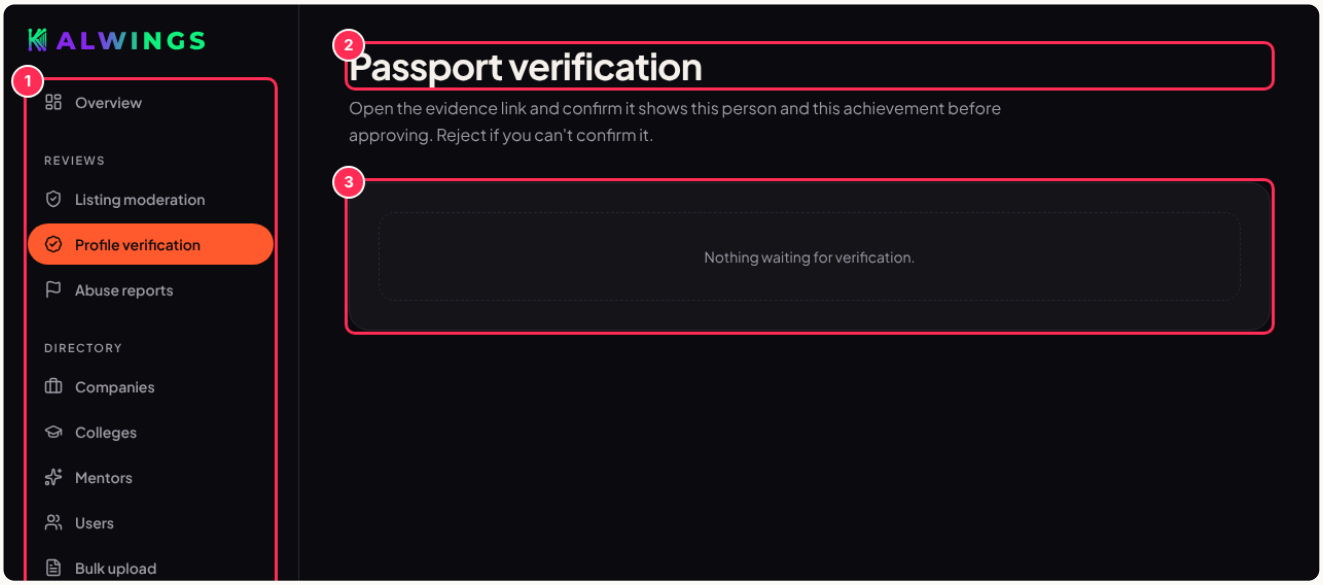


Figure 20. Profile verification

1. Navigation
2. Page title
3. Entries waiting for a decision

Abuse reports

Anyone can report a post, listing or profile. Open the report and look at the content. For community posts and comments press **Hide content** if it breaks the rules (hidden content disappears for other members). Then press **Mark resolved** to close the report. Use **Users → Suspend** for repeat offenders.

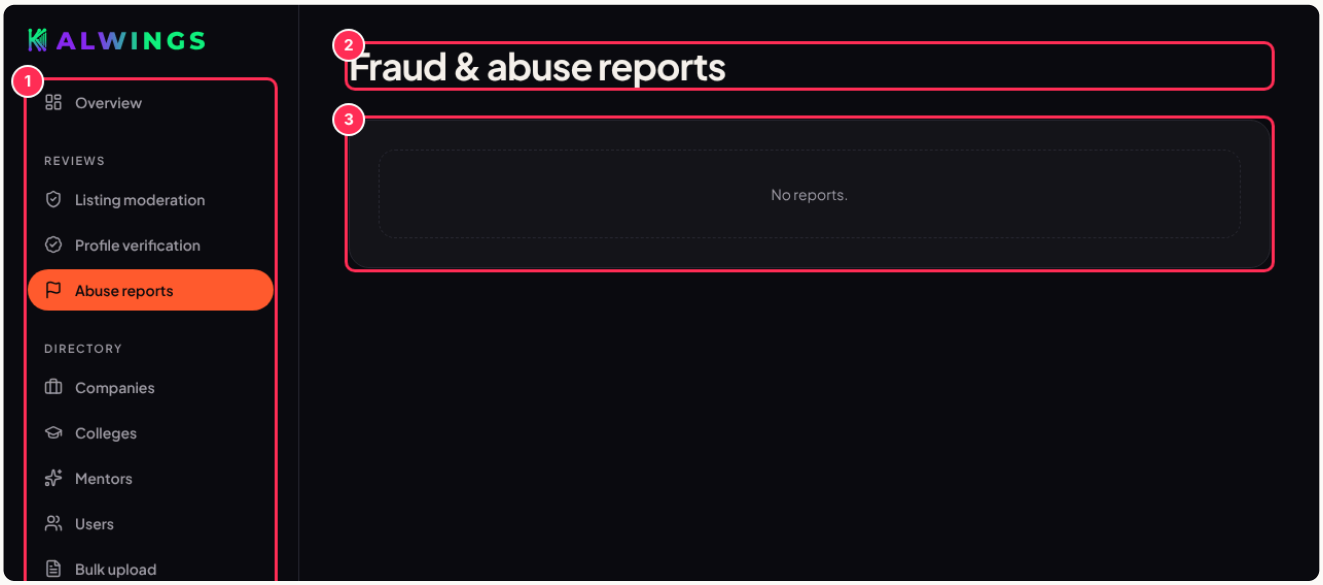


Figure 21. Abuse reports

1. Navigation
2. Page title
3. Open reports

Contact inbox

Messages from the website contact form arrive here (and also by email if configured). Reply from your own mailbox, then mark the message handled so the team knows.

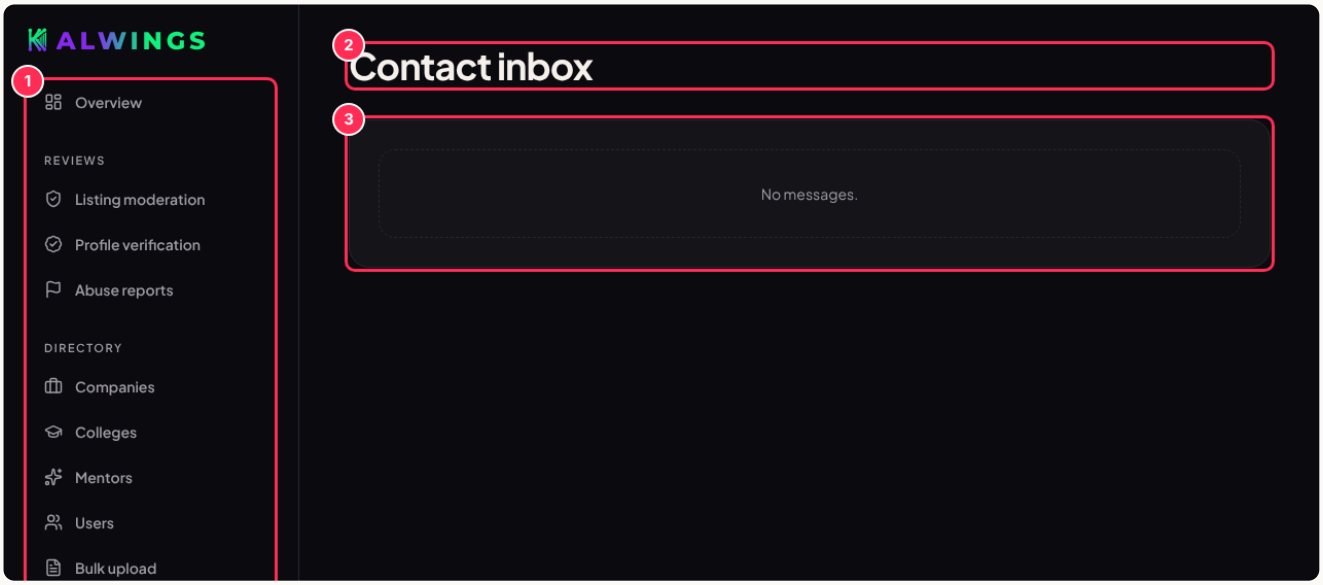


Figure 22. Contact inbox

1. Navigation
2. Page title
3. Messages

A14 Site settings: banner, support email, sign-ups

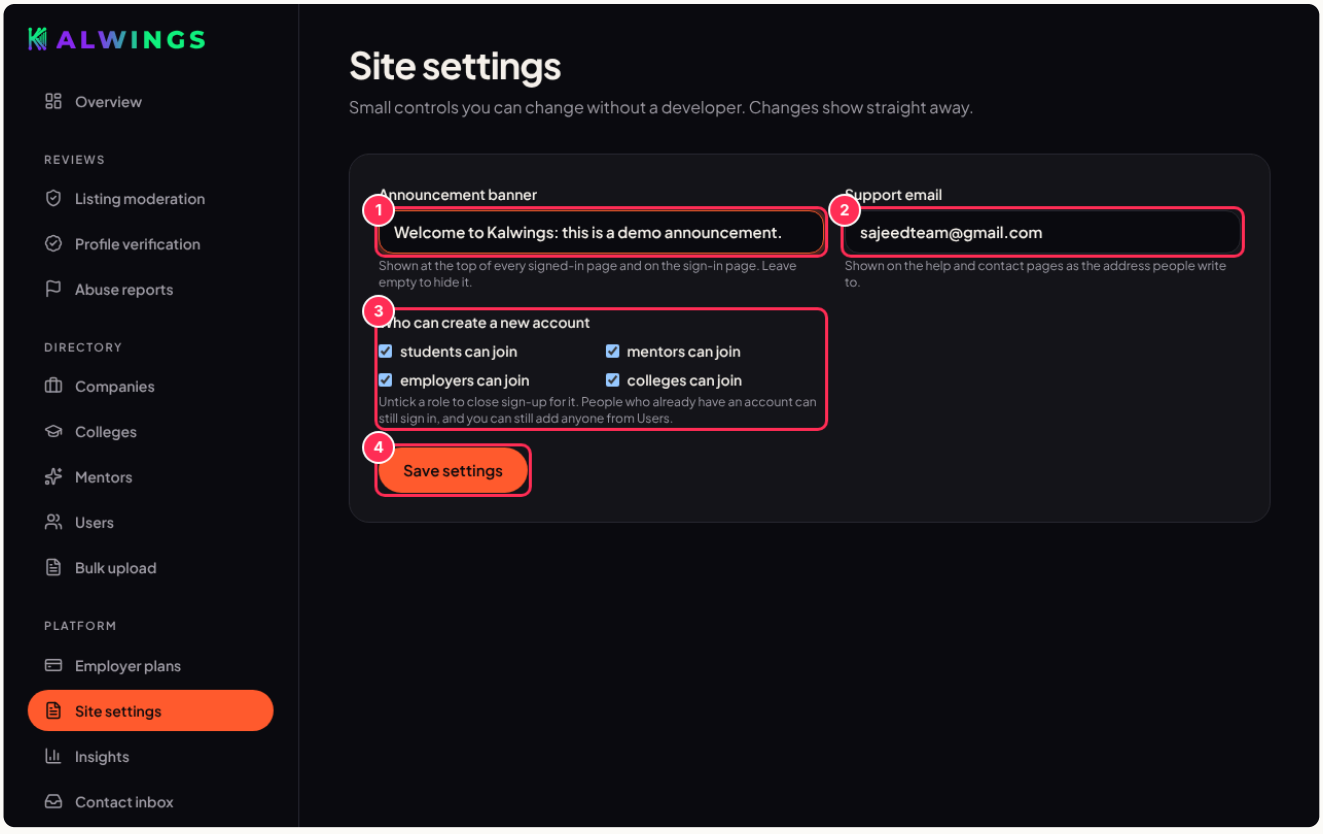


Figure 23. Site settings

1. **Announcement banner** : one sentence shown at the top of every signed-in page and the sign-in page
2. **Support email** : the address shown on Help and Contact
3. **Who can create a new account** : untick a role to close sign-up for it
4. Save settings

1. Type the banner text (up to 300 characters) or clear it to hide the banner.
2. Set the support email people should write to (blank = the default address).
3. Untick a role to stop new accounts of that type (for example while you review a backlog). People who already have an account can still sign in, and you can still add anyone from Users or Bulk upload.
4. Press **Save settings**. Changes show straight away.

> **Note.** When sign-up is closed, a person trying to join sees a clear message and no code is emailed to them.

A15 Launch readiness

A single page that tells you what still needs doing, **read live from your database and server**. Nothing is guessed or invented. Open **Operations → Launch readiness** before launch and after every big upload.

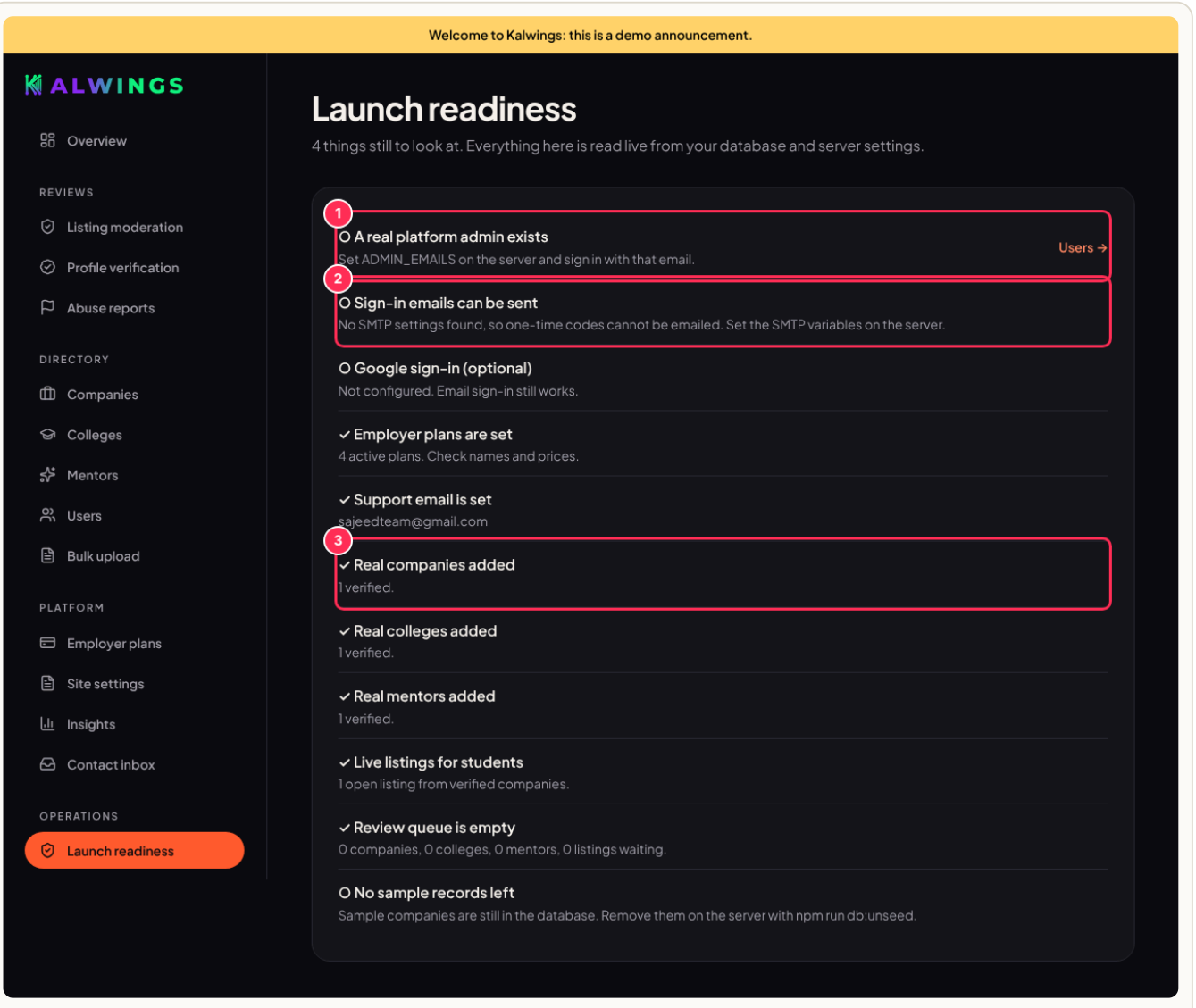


Figure 24. Launch readiness (demo data; banner showing)

1. **A real platform admin exists** : ticked when at least one non-sample admin account exists
2. **Sign-in emails can be sent** : red means nobody can receive codes: fix first
3. **Real companies added** : with a link to Companies when empty

Line	Ticked when	If not ticked
A real platform admin exists	At least one active admin that is not a sample account	Set ADMIN_EMAILS on the server and sign in (A3)
Sign-in emails can be sent	Email settings (SMTP) are present on the server	Ask the developer to set the SMTP variables (B3)
Google sign-in (optional)	Google keys are present	Optional; email sign-in still works
Employer plans are set	At least one plan is active	A12
Support email is set	A support address exists	A14
Real companies / colleges / mentors added	At least one verified, non-sample record each	A6, A7, A8, A11
Live listings for students	At least one open listing from a verified company	Post real, agreed roles (A6)
Review queue is empty	Nothing pending for companies, colleges, mentors, listings	A5, A13
No sample records left	No sample companies remain	Ask the developer to run the unseed command (B2, B3)

A16 Insights, audit log and server errors

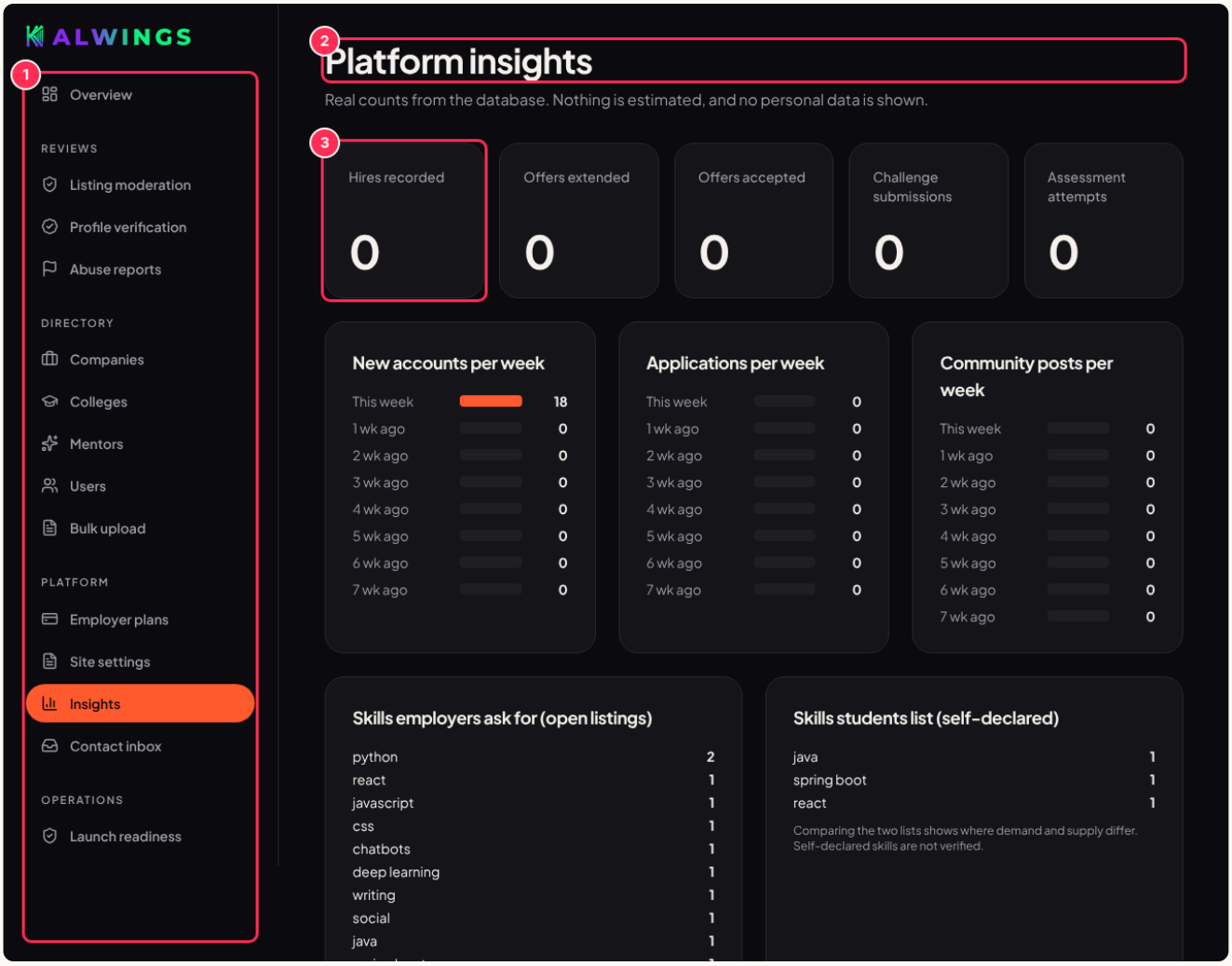


Figure 25. Insights

1. Navigation
2. Page title
3. Real counts: nothing is estimated

Insights shows real weekly counts (new accounts, applications, community posts), the skills employers ask for versus the skills students list, applications by stage and live listings by type. If a number is small, it is small; never round it up in public.

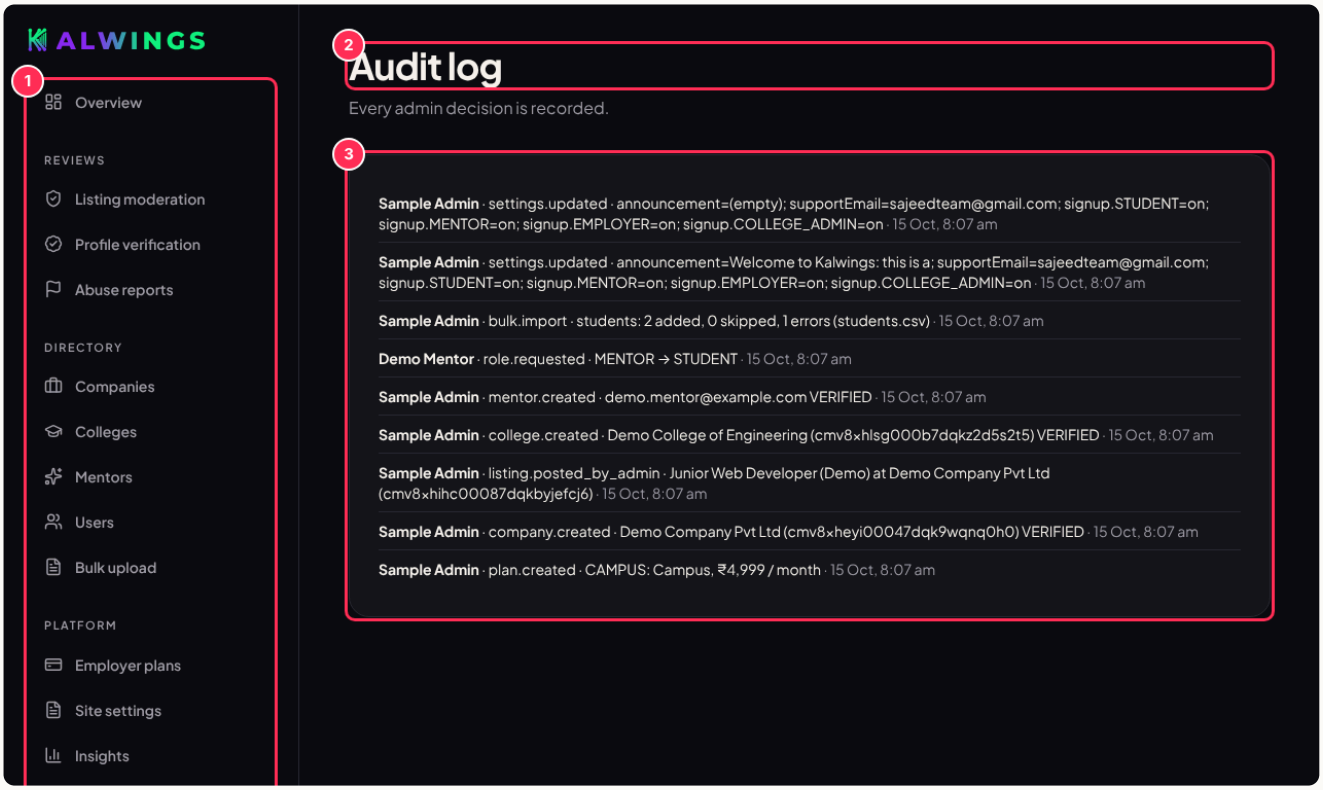


Figure 26. Audit log

1. Navigation
2. Page title
3. Every sensitive action: who, what, when

The **Audit log** records every admin decision (verify, reject, plan change, role change, bulk import, settings change, suspension). Use it to answer “who changed this?” Entries cannot be edited. Keep at least 12 months.

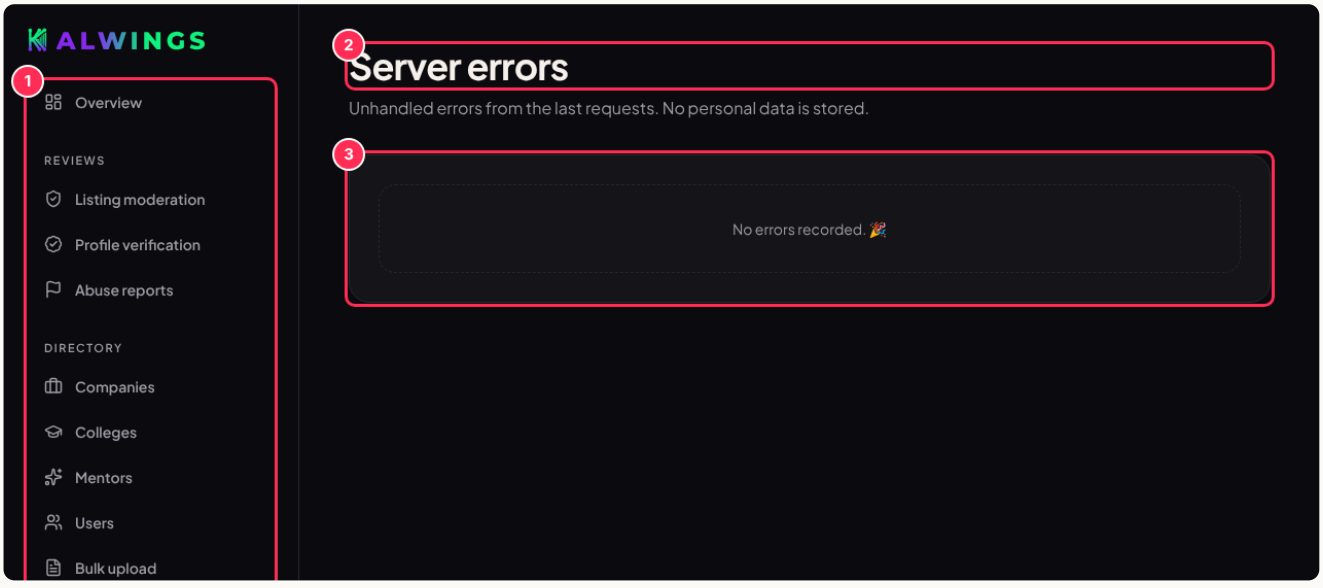


Figure 27. Server errors

1. Navigation
2. Page title
3. Errors with no personal data: for the developer

Server errors lists unhandled errors from the server. Staff do not need to act on them; if the list grows, tell the developer and include the time you saw it.

A17 Seeing what your users see

Support staff need to know the other roles' screens. Use a different browser or a private window with a test account, never your admin session, when you reproduce a problem.

Public pages (no sign-in)

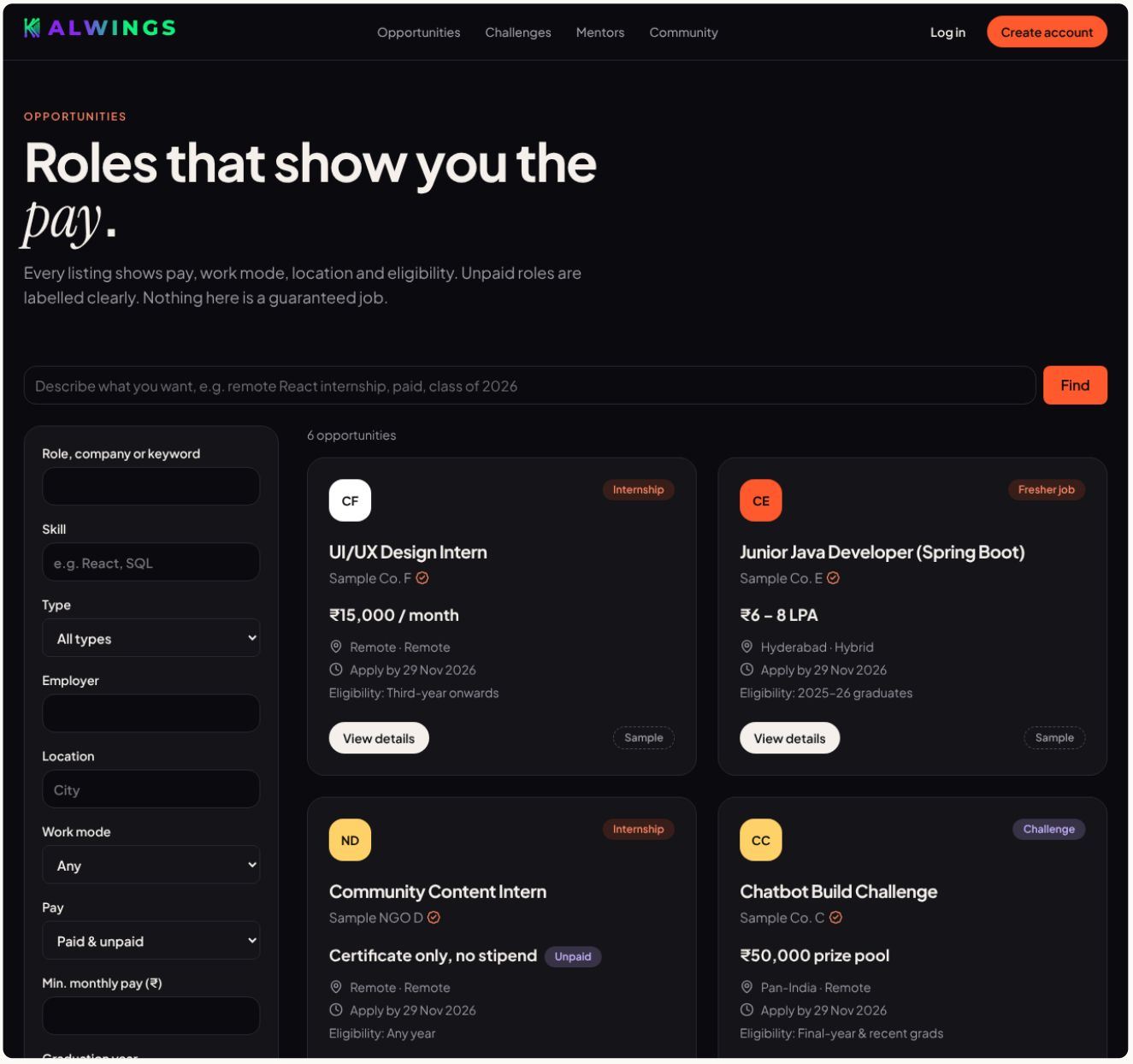


Figure 28. Public opportunities page

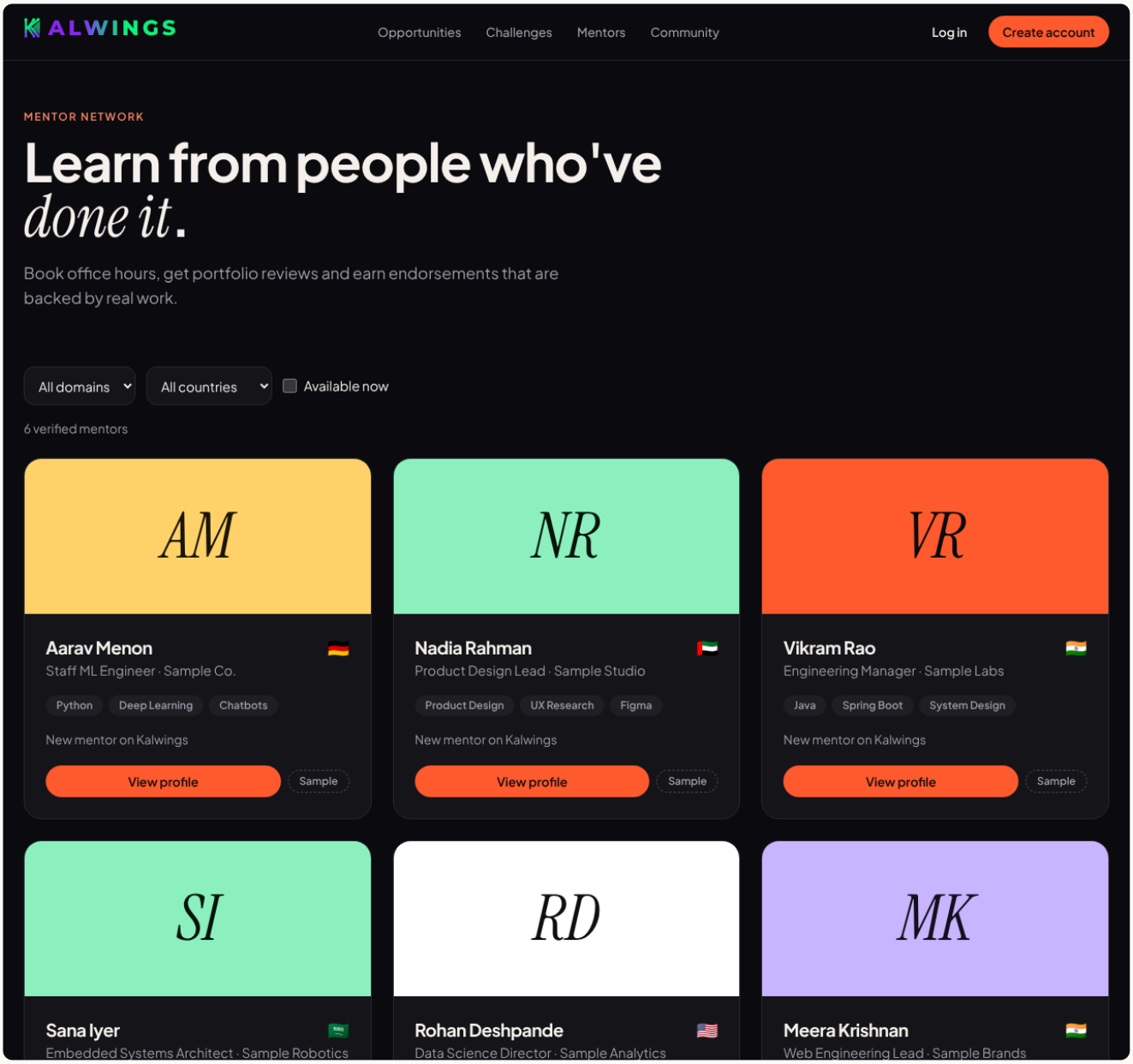


Figure 29. Public mentors page

Student

A student's menu: Find work (Opportunities, Challenges, Campus drives, Saved roles), Applications, Profile, Grow (Career copilot, Mentor sessions, Community). Their profile score is a guide, not a ranking.

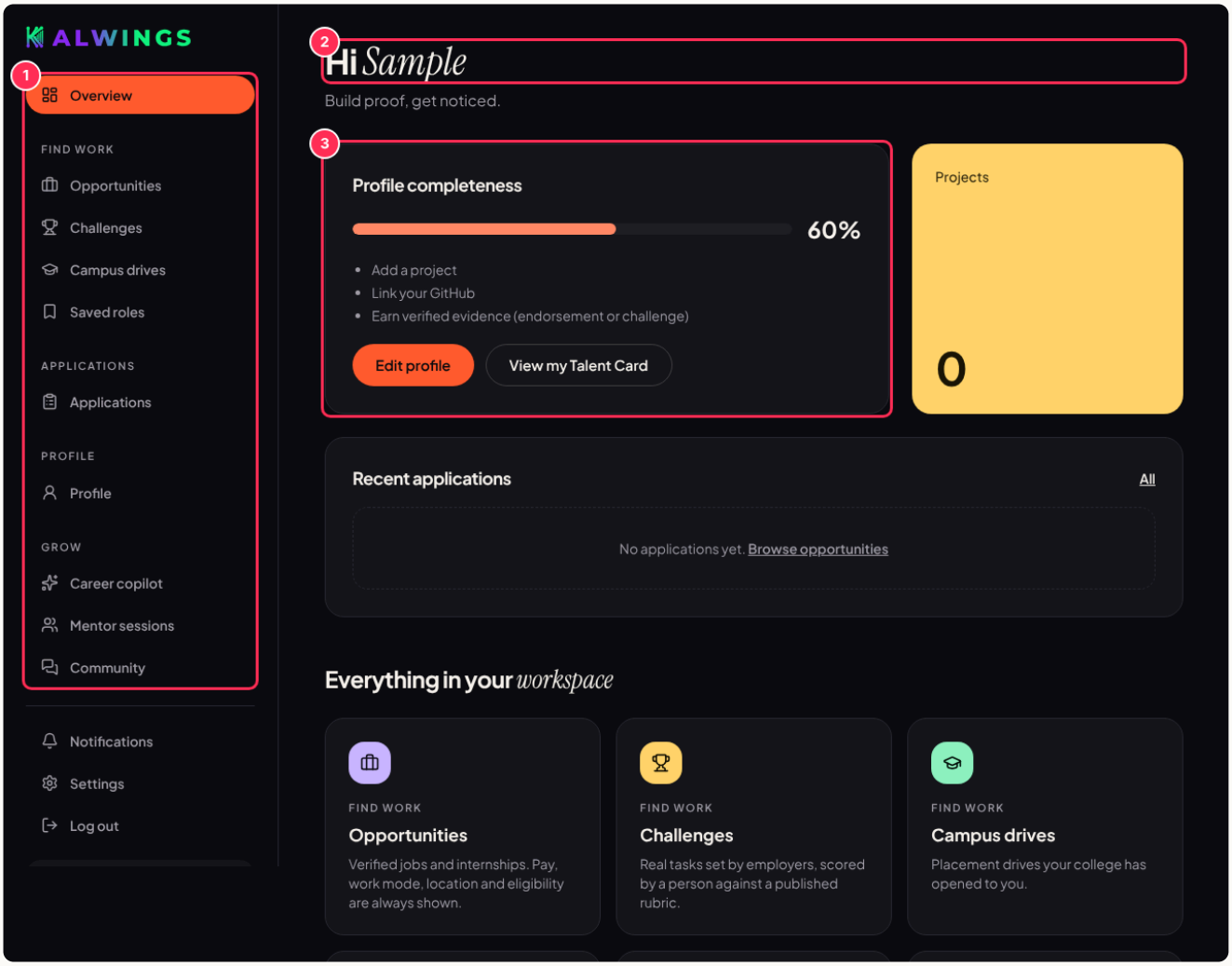


Figure 30. Student home

1. Navigation: the student's areas
2. Greeting
3. Profile completeness and recent applications

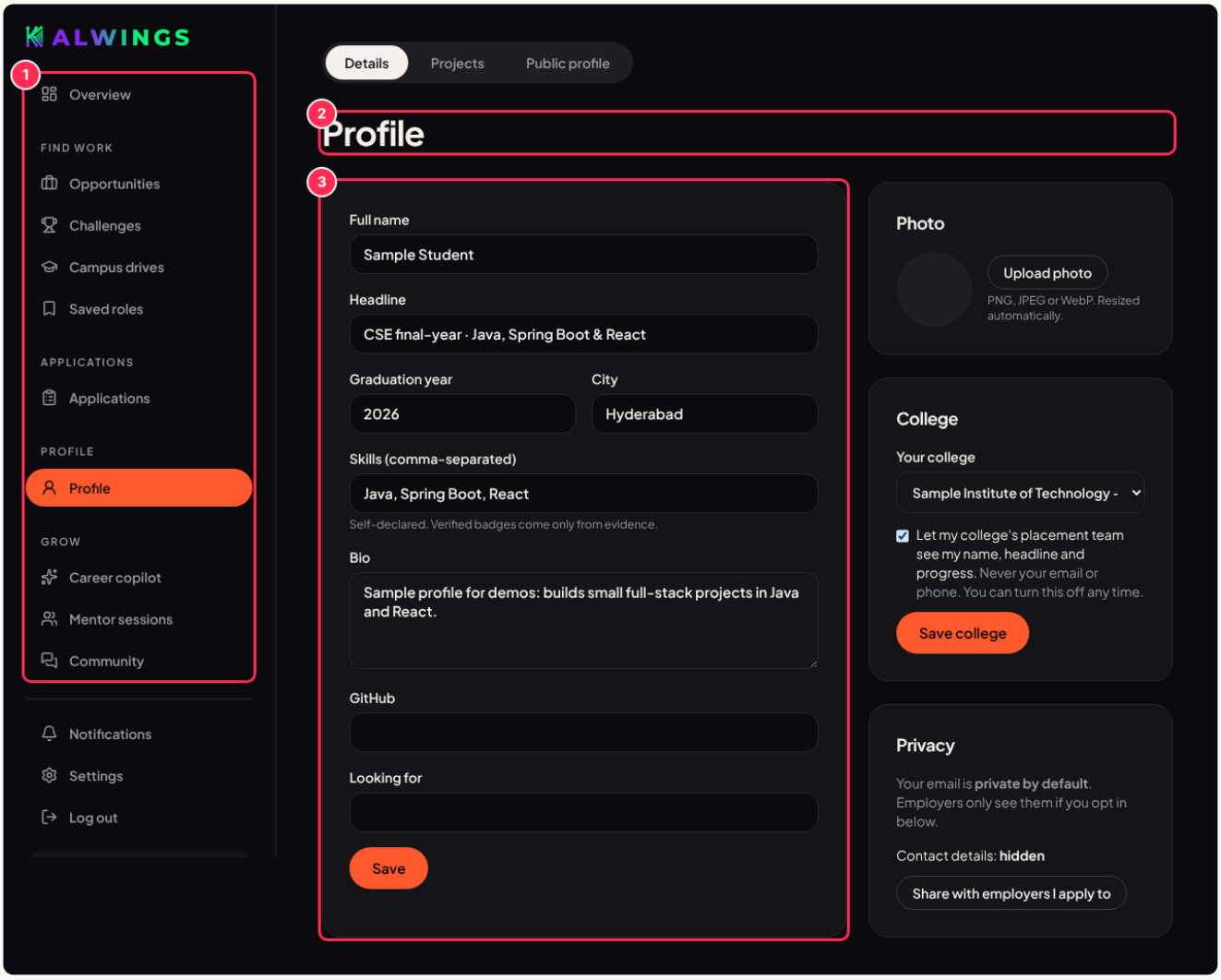


Figure 31. Student profile

1. Navigation
2. Page title
3. Details, skills, bio, college and privacy: the student controls what employers see

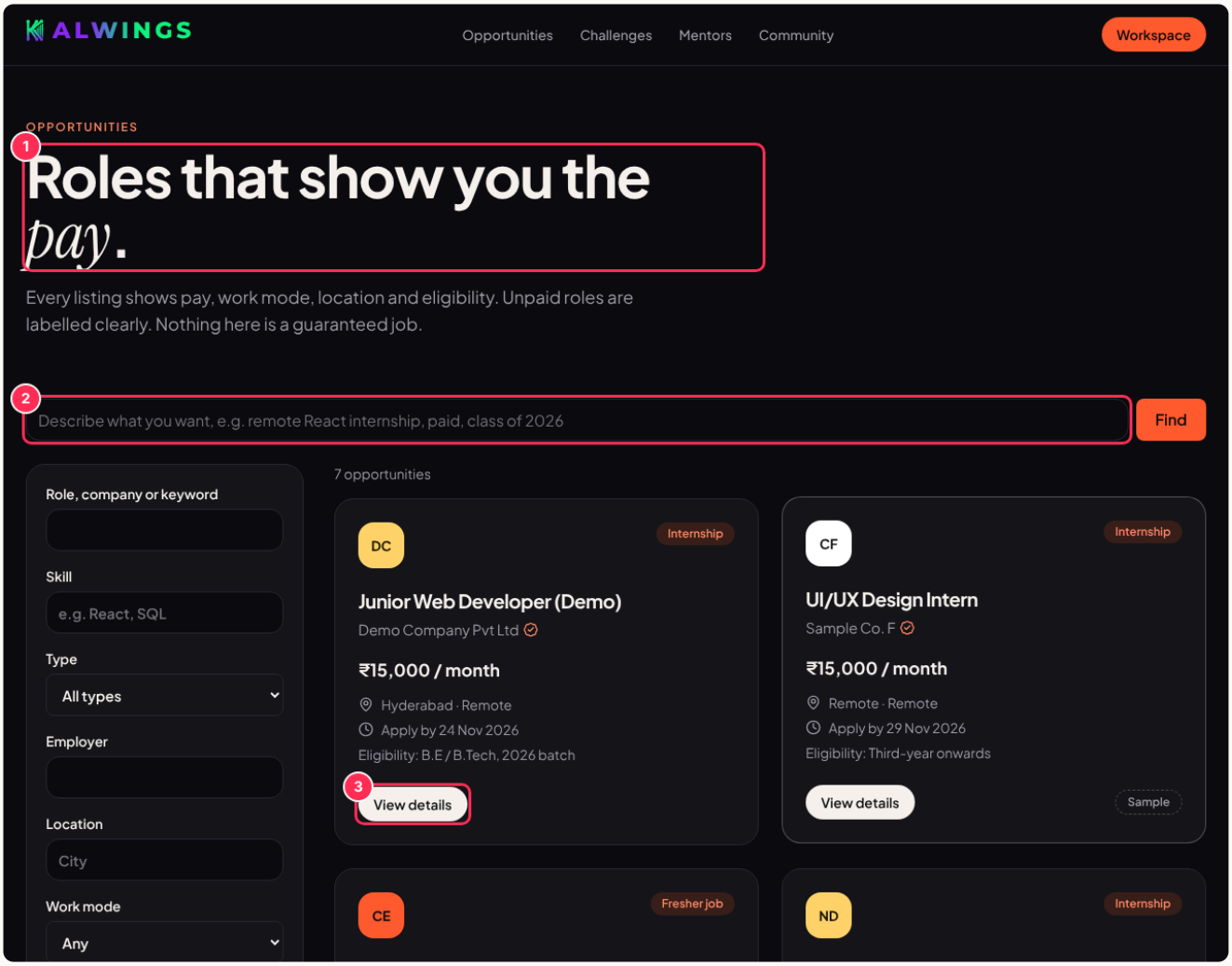


Figure 32. Student browsing opportunities

1. Page title
2. Describe what you want in plain words
3. Open a listing

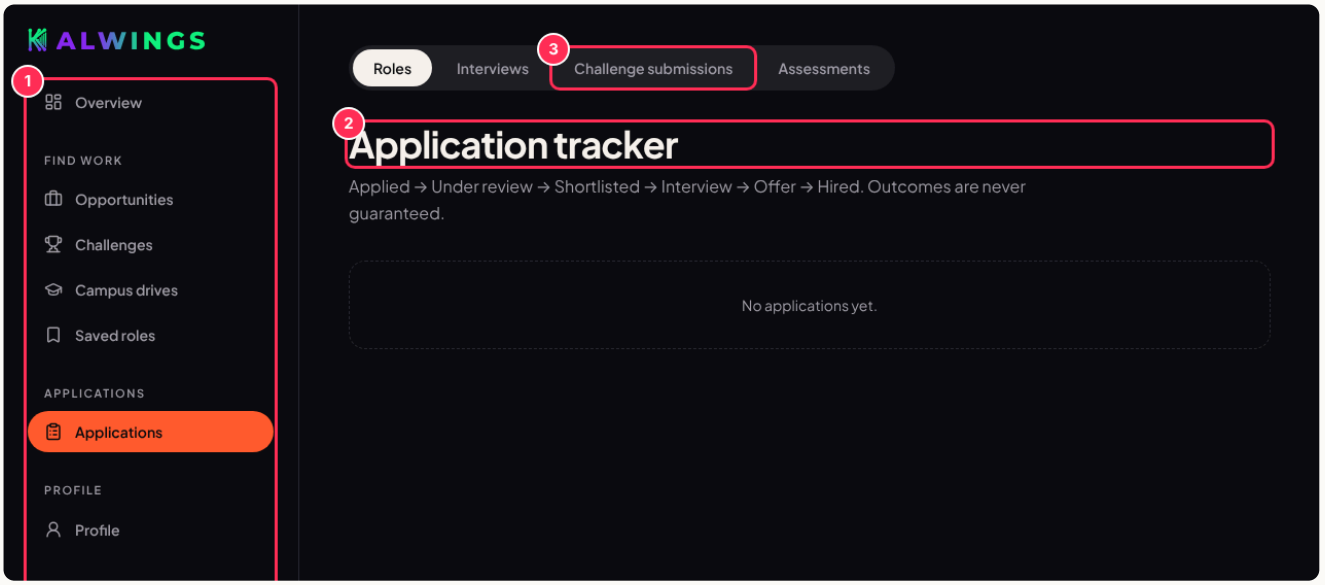


Figure 33. Applications tracker

1. Navigation
2. Page title
3. Tabs: Applications, Interviews, Challenge submissions, Assessments

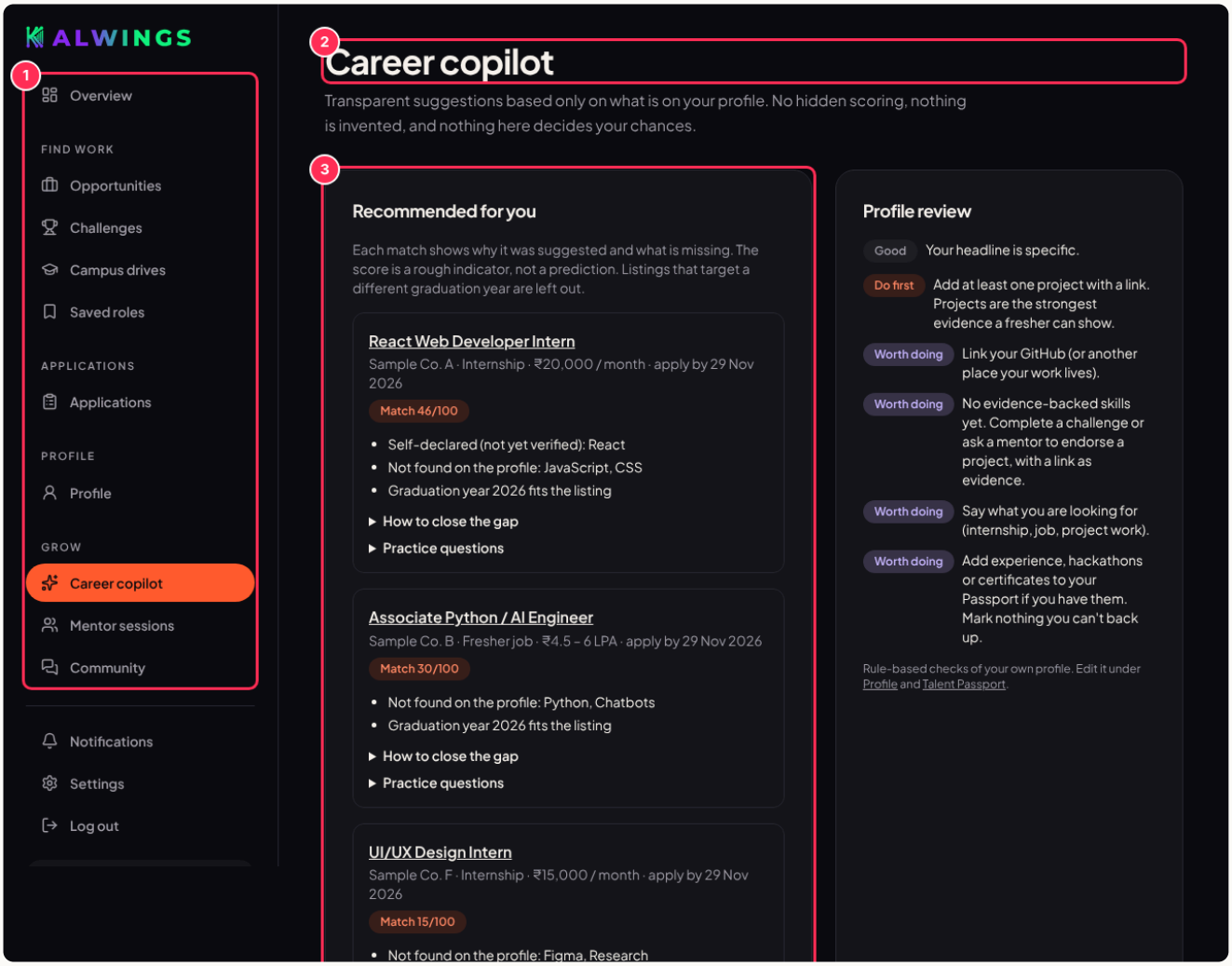


Figure 34. Career copilot

1. Navigation
2. Page title
3. Explained matches and profile review: rule-based, no hidden scoring

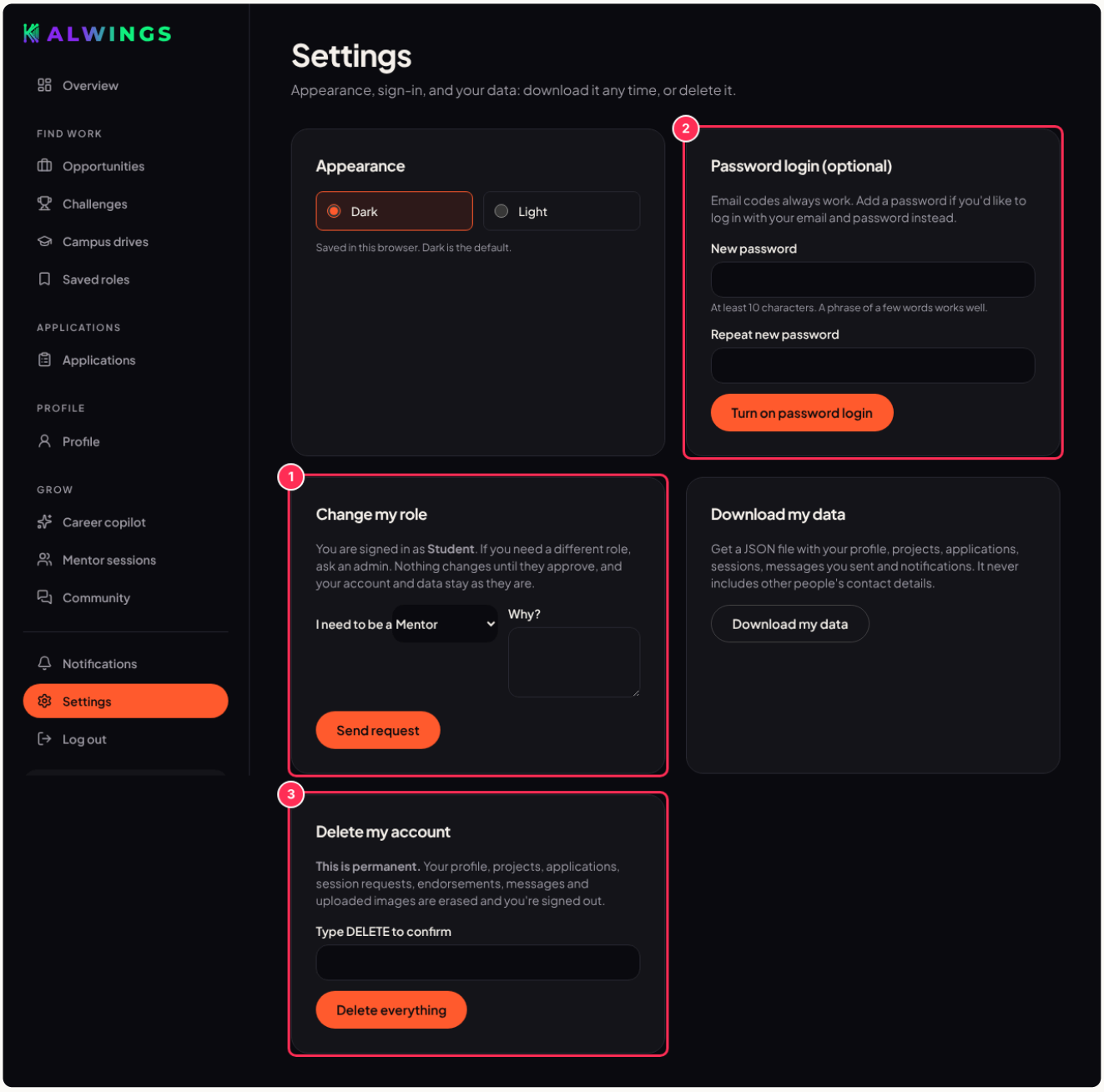


Figure 35. Settings (every role has this page)

1. **Change my role** : asks an admin
2. **Password login** : optional, email codes always work
3. **Delete my account** : permanent, done by the user, not by staff

Employer

An employer's menu: Company, Roles (and challenges, assessments, description check), Talent search, Pipeline, Analytics, Campus drives.

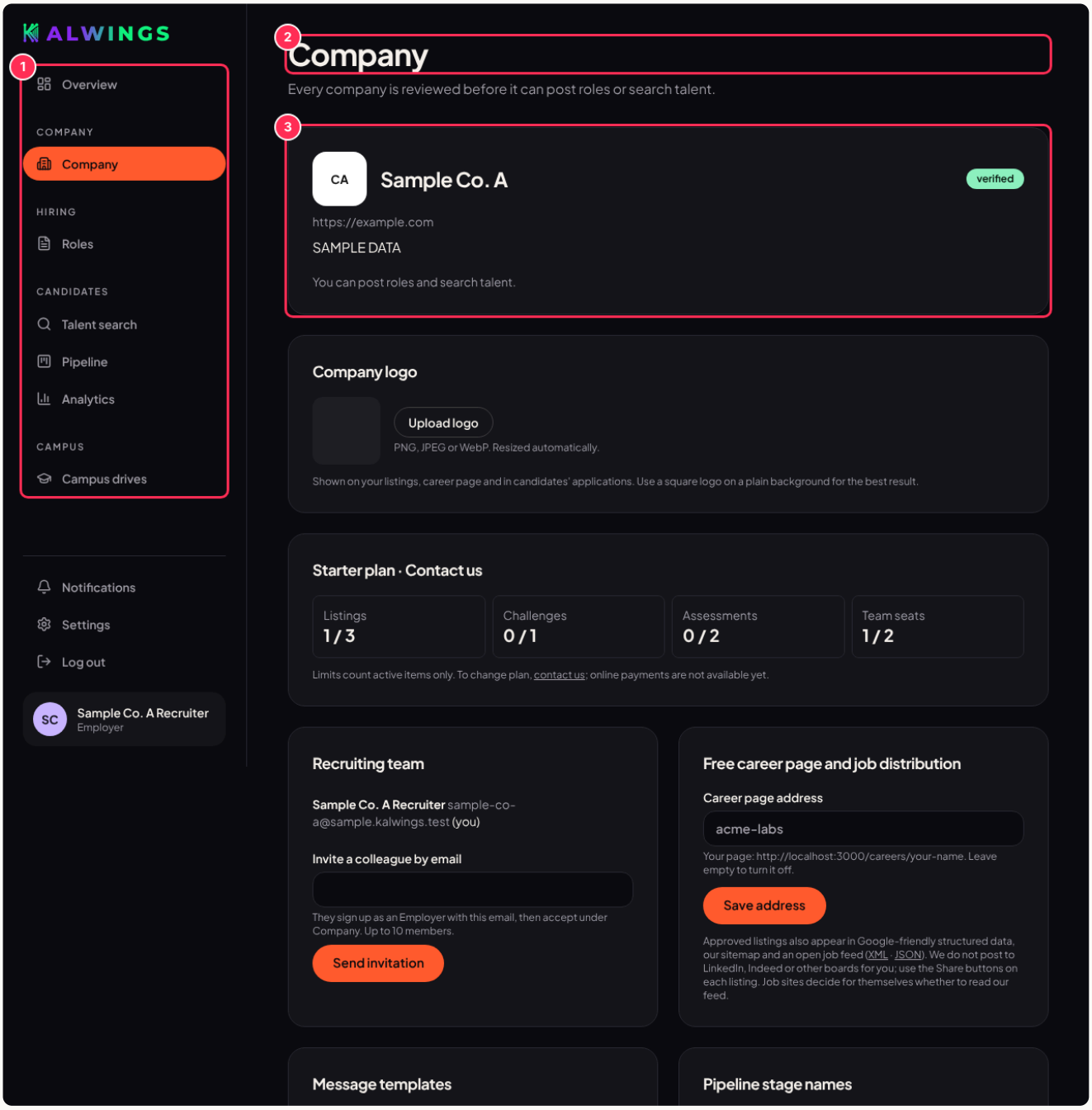


Figure 36. Employer: Company

1. Navigation
2. Page title
3. Company status, plan usage, team, career page, message templates and stage names

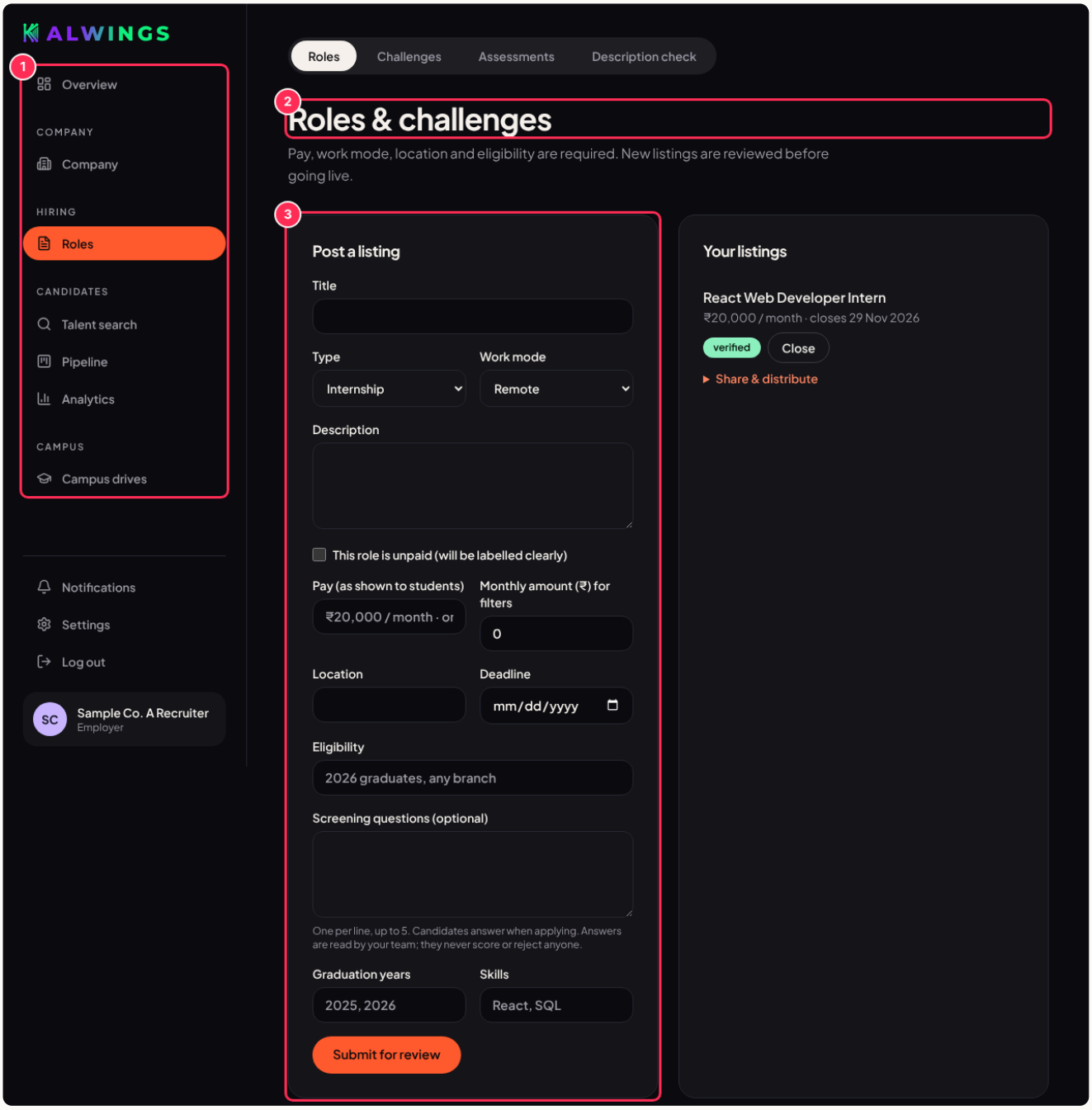


Figure 37. Employer: Roles

1. Navigation
2. Page title
3. Post a listing and see current listings

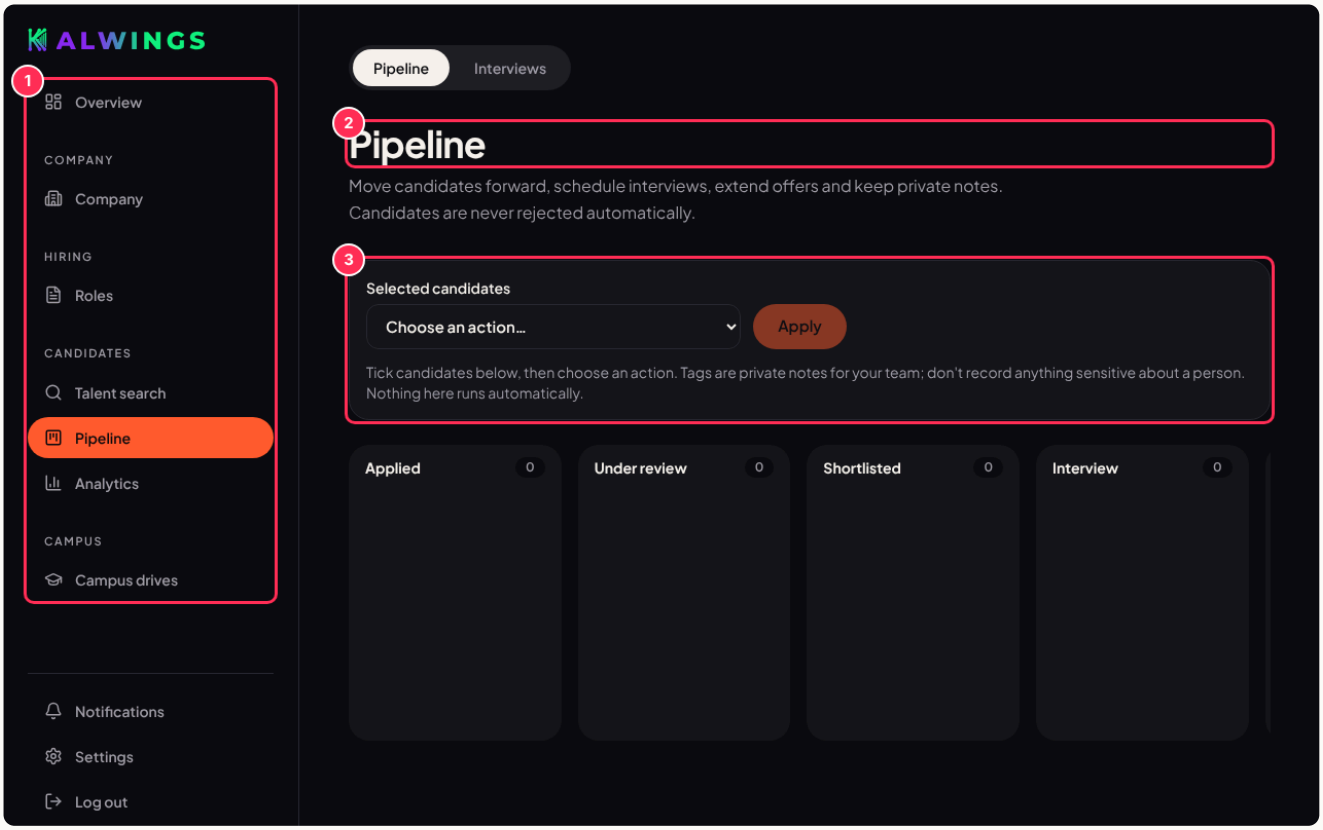


Figure 38. Employer: Pipeline

1. Navigation
2. Page title
3. Move candidates through stages; a person decides every move

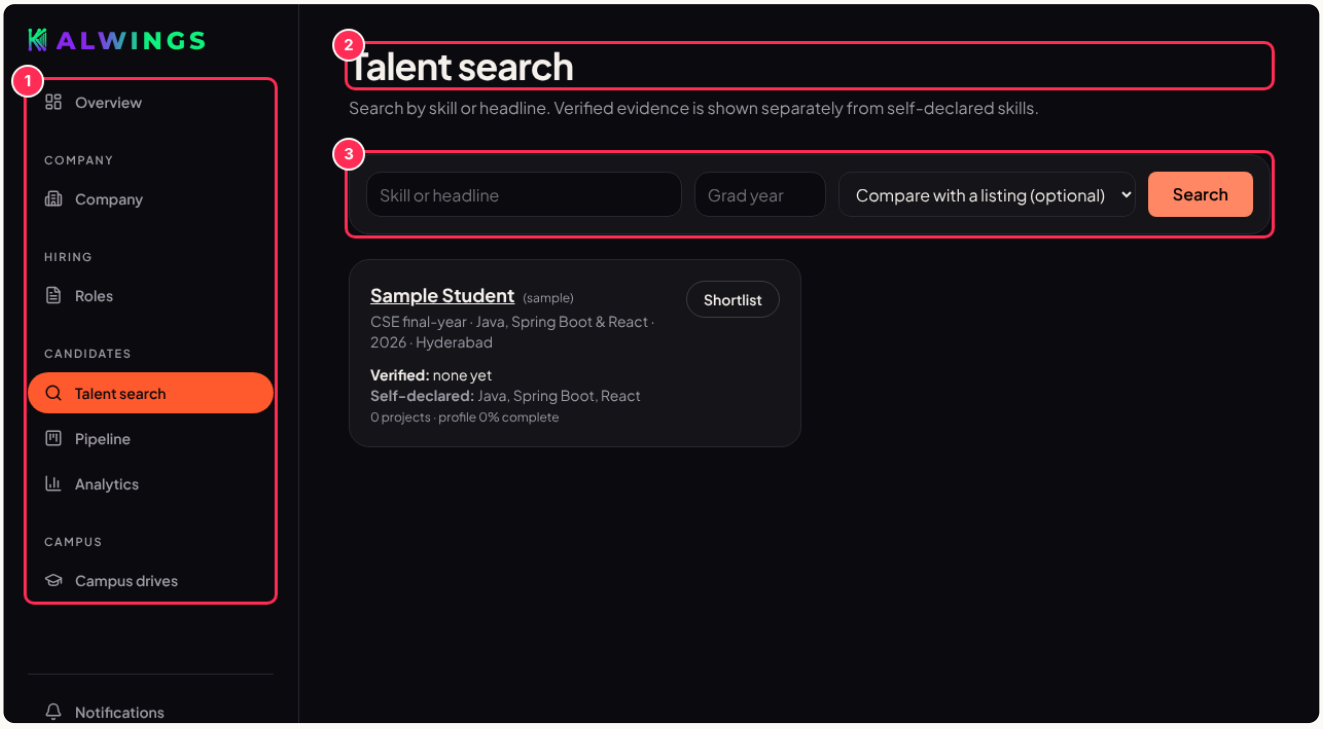


Figure 39. Employer: Talent search

1. Navigation
2. Page title
3. Search by skill; only verified companies can use it

College admin

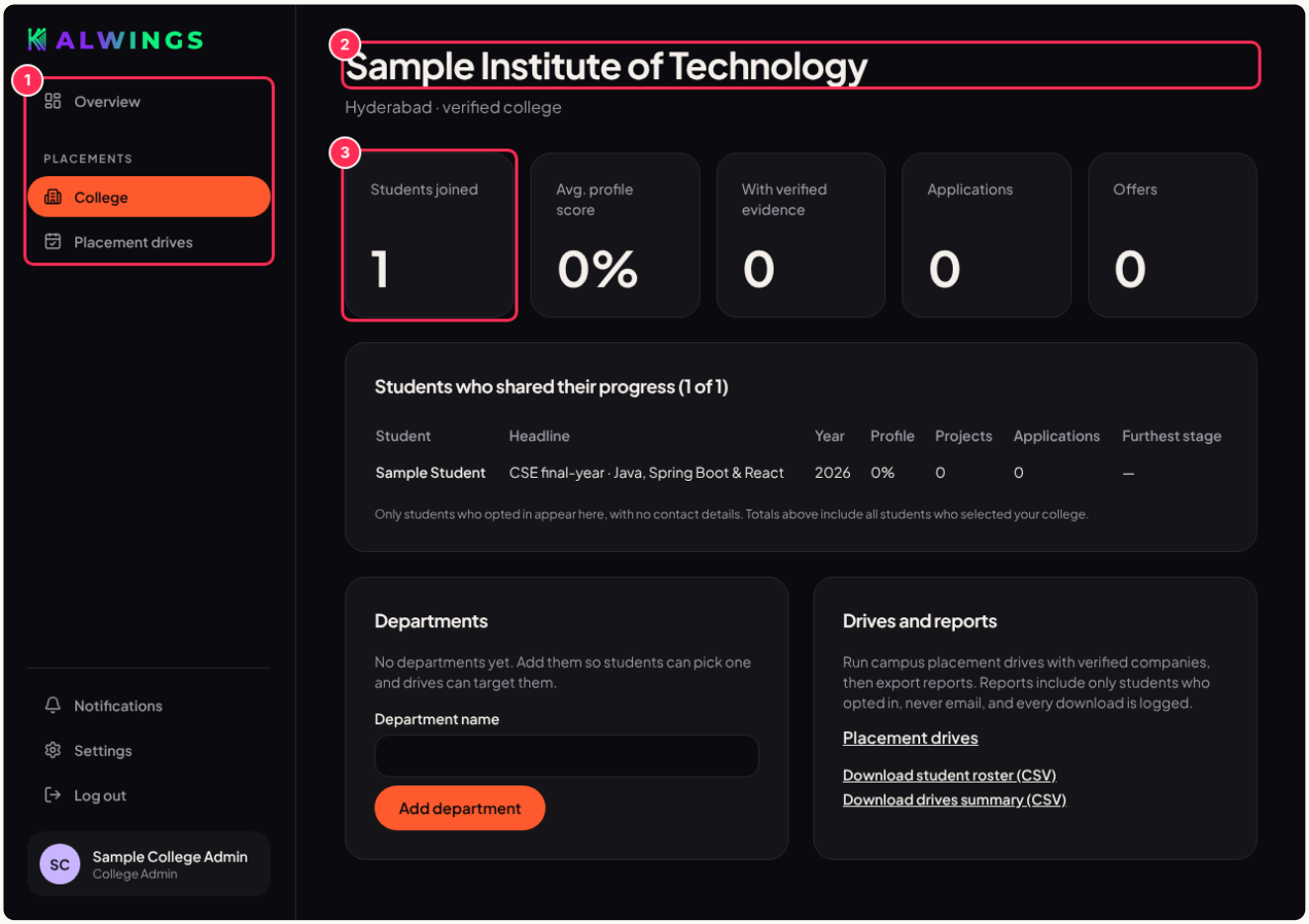


Figure 40. College dashboard

1. Navigation
2. College name
3. Students who chose to share progress, departments, reports

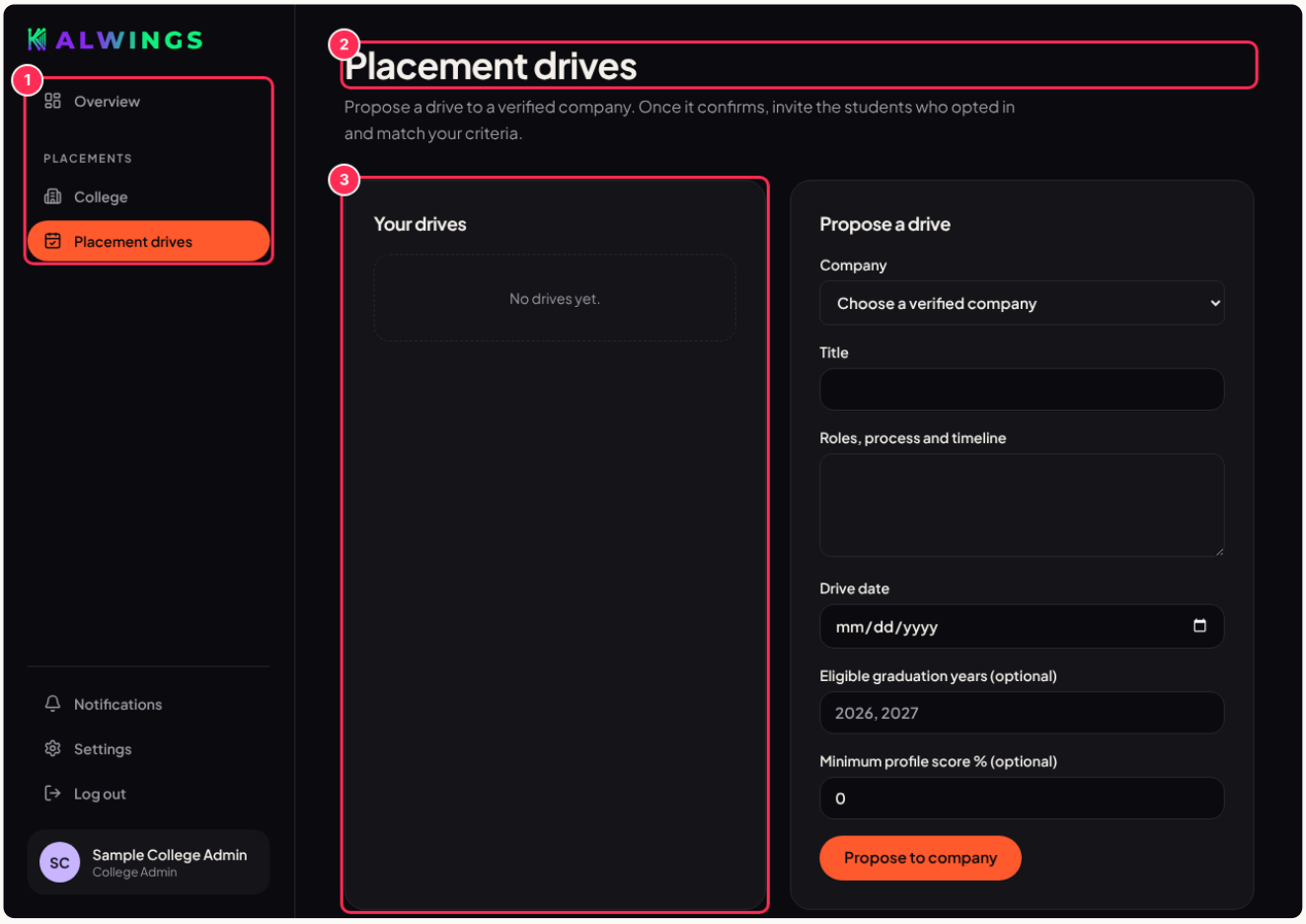


Figure 41. Placement drives

1. Navigation
2. Page title
3. Propose drives to verified companies

On a phone

Everything works at phone width. The menu becomes a scrolling strip at the top.

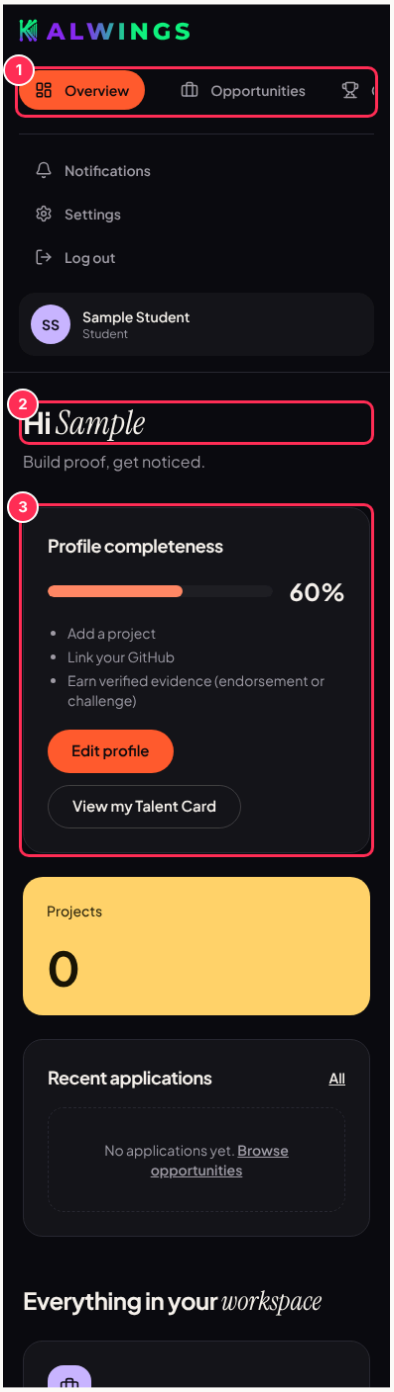


Figure 42. Student home on a phone (390 px wide)

1. Navigation strip
2. Greeting
3. Cards stack in one column

A18 Troubleshooting for staff

Problem	Likely cause	What to do
“I never got my sign-in code”	Spam folder, typo, rate limit, or email not configured	Check spam, retry after a minute, confirm the email, check Launch readiness (A3, A15)
A company says it cannot post	Not verified, or at its plan limit	Verify it (A5); check usage and plan (A12)

Problem	Likely cause	What to do
A mentor does not appear	Status is Pending or Rejected	Edit the mentor and set Verified (A8)
A student cannot find their college	College missing or not verified	Add it or set Verified (A7)
Bulk upload says “Missing column”	Header row changed or misspelled	Download the template again; keep row 1 as is (A11)
Bulk upload says “already exists”	Email or company is in the system	Expected: it is skipped, nothing doubled
An employer sees a limit message	Plan limit reached	Close old listings, or switch the plan (A12)
New sign-ups for a role are refused	Sign-up for that role is closed	Site settings (A14)
A user is stuck on the wrong screens	Wrong role	Change role (A9)
The whole site shows an error	Database or server problem	Tell the developer; give the time. They check /api/health (B16)

A19 Glossary for staff

Term	Meaning
Verified	A person at Kalwings checked it. Only admins can set this.
Pending / Rejected	Waiting for a decision / decided no
Listing	A job, internship, apprenticeship, campus hiring, freelance project, or challenge
Challenge	A real task open to everyone, scored by a person against a rubric
Assessment	A timed test assigned to a candidate (written, or coding where enabled)
Pipeline	An employer’s board of candidates by stage
Plan	A set of limits and a price shown to employers
Sample	Fake demo data. Tagged “Sample”. Never present it as real.
Audit log	The permanent record of admin actions
Suspended	Account blocked and hidden, but not deleted; reversible

PART B

Developer reference

For the developers who keep Kalwings running. Written as a senior developer's handover to a junior: how the code is organised, how to run it, how to change it safely, and the rules that keep users' data safe. Read B1 to B5 first; use the rest as reference.

B1 What you are working on

Kalwings (app.kalwings.com) is a career platform for freshers: students build a profile with real projects, employers post roles and challenges and run a hiring pipeline, colleges track consenting students and run placement drives, mentors give office hours, and platform admins verify, moderate and configure. The marketing site is separate (kalwings.com). This app starts at sign-in.

Stack

Layer	Choice	Notes
Framework	Next.js 15 (App Router), React 19, TypeScript	Server components by default; mutations are **server actions**; a few route handlers for files, uploads, exports
Styling	Tailwind CSS with CSS-variable tokens	Dark by default, light in Settings; tokens in `src/app/globals.css`
Data	Prisma 6 on MySQL 8 or MariaDB 10.11	Hostinger runs either; CI tests both
Validation	zod	Every action validates its input on the server
Auth	Email one-time codes (+ optional password, Google)	Signed JWT in an httpOnly cookie via `jose`
Email	SMTP via nodemailer	Hostinger mailbox; no third-party email API
Spreadsheets	`read-excel-file`	Bulk upload (never the vulnerable `xlsx` npm package)
Tests	tsx unit tests, Playwright e2e/a11y/access	Run against a dev server and a seeded database
Hosting	Hostinger Node.js app, auto-deploy from GitHub `main`	Migrations run at server start

Request flow in one paragraph

A browser requests a page. A server component calls `requirePageRole(...)`, which reads the `kw_session` cookie, verifies the JWT, then **re-reads the user from the database** (so suspensions and role changes apply on the very next request) and redirects if the role is wrong. The page queries Prisma directly and renders. When a form is submitted, a **server action** runs: it calls `requireActionRole(...)` first, validates with zod, writes via Prisma, writes an audit entry for sensitive changes, calls `revalidatePath`, and returns a small result object that the form displays.

Folder map

Path	What lives there
`src/app/`	Routes. `dashboard/*` is the signed-in app; `api/*` route handlers; `actions/*` server actions; public pages (login, join, opportunities, mentors, careers, talent, help, contact, legal)
`src/app/actions/`	All mutations, one file per area: `admin, console, employer, student, mentor, assessments, community, drives, auth, account, common, contact, college`
`src/lib/`	Server logic: `session, db, audit, email, plans, settings, bulk, validation, matching, nlsearch, migrate, rate-limit, modules` and more
`src/components/`	`ui/` (Button, Badge, Serif, CompanyLogo, brand), `dash/` (forms, sidebar, panels, uploads), `pages/`, `home/`
`prisma/`	`schema.prisma`, `migrations/` (hand-reviewed SQL), `seed.ts` (sample data only)

Path	What lives there
`scripts/`	`embed-migrations.mjs`, `gen-db-docs.mjs`, `gen-api-docs.mjs`, `backup-db.mjs`, `make-admin.ts`, export scripts
`e2e/`	`platform.mjs` (journeys), `a11y.mjs`, `access.mjs`, `unit.ts`, `ratelimit.ts`, `production.mjs`, `degraded.mjs`, mock mail and sandbox
`docs/`	Generated and written docs (`DATABASE.md`, `API.md`, `OPERATIONS.md`, `SECURITY.md`, `QUALITY.md`, `FEATURES.md`) and this manual

B2 Set up your machine (step by step)

You need Node 20 or newer (CI uses 22), Git, and a local MySQL 8 or MariaDB 10.6+.

1. Clone the repository and install: `git clone https://github.com/Sajeed25/kalwings.git`, `cd kalwings`, `npm install` (this also runs `prisma generate`).
2. Start a database. With Docker: `docker run -d -p 3306:3306 -e MARIADB_ROOT_PASSWORD=root -e MARIADB_DATABASE=kalwings mariadb:10.11`.
3. Copy the environment file: `cp .env.example .env`. Set `DATABASE_URL=mysql://root:root@localhost:3306/kalwings`, a 32+ character `AUTH_SECRET` (`openssl rand -base64 48`) and `NEXT_PUBLIC_SITE_URL=http://localhost:3000`. Leave SMTP empty locally: sign-in codes then appear on screen.
4. Create the tables: `npx prisma migrate dev` (needs permission to create a shadow database) or, if you only need to apply existing migrations, `npx prisma migrate deploy`.
5. Load demo data: `npm run db:seed`. This creates **sample** mentors, companies, listings, one sample admin, student, college admin, mentor and employer. Every record is flagged or named “Sample”.
6. Run the app: `npm run dev` and open http://localhost:3000.
7. Sign in as `admin@sample.kalwings.test` (or `student@…`, `college@…`, `sample-mentor-1@…`, `sample-co-a@…`). The one-time code is printed on the page in development.

> **Careful. Never run `npm run db:seed` against production.** If it ever ran there, run `npm run db:unseed`. Sample sign-in addresses end in `@sample.kalwings.test` and cannot receive mail, so they are inert, but they must not exist on a live site.

Make yourself a real admin locally

Put your email in `ADMIN_EMAILS` and sign in, or sign in once and run `DATABASE_URL=… npm run make-admin -- you@example.com`.

Everyday commands

Command	What it does
`npm run dev`	Dev server on port 3000
`npm run lint` / `npm run typecheck`	ESLint (next lint) / `tsc --noEmit`
`npm run test:unit`	18 pure-logic tests (no browser, no server)
`npm run build`	Embeds migrations (prebuild) then builds Next
`npm run db:seed` / `db:unseed`	Add or remove SAMPLE data
`npm run docs:db` / `docs:api`	Regenerate `docs/DATABASE.md` and `docs/API.md`

Command	What it does
`npm run backup -- ./backups 14`	gzip dump, keep newest 14
`npm run test:e2e | test:access | test:a11y`	Browser suites against a dev server (B15)

B3 Configuration reference

All configuration is environment variables. On Hostinger they are set in hPanel. Changing `NEXT_PUBLIC_SITE_URL` needs a redeploy because it is baked in at build time. Never commit secrets; rotate any secret that was pasted anywhere.

Variable	Required	Purpose
`DATABASE_URL`	yes	mysql://user:pass@host:3306/db?connection_limit=5 (URL-encode special characters in the password)
`AUTH_SECRET`	yes	32+ random characters. Changing it signs everyone out
`NEXT_PUBLIC_SITE_URL`	yes	https://app.kalwings.com, no trailing slash
`SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, EMAIL_FROM`	yes in production	Hostinger mailbox (smtp.hostinger.com, 465). Without it nobody can sign in. SMTP_USER must be the full address; EMAIL_FROM must be that mailbox or an alias
`ADMIN_EMAILS`	first admin	Comma-separated; these emails become platform admins when they sign in
`CONTACT_TO`	no	Inbox for contact-form emails (default EMAIL_FROM); messages are always stored in the database too
`SANDBOX_URL, SANDBOX_TOKEN`	no	Isolated Judge0-compatible runner; unset = coding assessments off
`GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET`	no	Google sign-in (untested against real credentials)
`MOBILE_LOGIN`	no	Leave unset. Mobile OTP is hidden and refused until an SMS provider exists
`AUTO_MIGRATE`	no	Default on: applies pending migrations at server start. `false` disables

What the server logs at start-up (check the Runtime log on Hostinger): `[startup] database: …`, `[startup] email: …` (host, user, password length, never the password), and `[migrate] ok` or `[migrate] FAILED`. Missing or malformed variables are reported there with a plain explanation.

B4 Authentication, sessions and roles

How sign-in works

1. The user submits an email. `requestOtp` (in `actions/auth.ts`) rate-limits per identifier and per IP, checks whether sign-up is open for the chosen role when the account does not exist (`signupClosedMessage`), creates a 6-digit code (hashed in `OtpCode`, expires in 10 minutes) and emails it.
2. `submitOtp` checks the code (5 attempts), creates the user if joining (student profile created for students), and calls `createSession`.
3. `createSession` signs a JWT (`HS256`, 14 days) with the user id as subject and sets `kw_session` (httpOnly, sameSite=lax, secure in production). A non-sensitive `kw_in` hint cookie lets static pages show “Dashboard”; it is never used for authorisation.

4 On every request `getUser()` verifies the JWT and re-reads the user. A suspended user returns `null`, so suspension is instant.

Authorisation: the two helpers you must use

```
// pages: redirect when signed out or wrong role
const user = await requirePageRole("PLATFORM_ADMIN");

// server actions and route handlers: throw instead of redirecting
const a = await requireActionRole("PLATFORM_ADMIN");   // FIRST line of EVERY action
```

Platform rule. A server action is a public HTTP endpoint. Hiding a button is not security. Every action must call `requireActionRole` before anything else, and every query for a user's own data must be scoped to that user in the query itself (`where: { id, userId: user.id }`), not checked afterwards.

Roles

Roles are the Prisma enum `Role`: STUDENT, MENTOR, EMPLOYER, COLLEGE_ADMIN, PLATFORM_ADMIN. Role-specific data hangs off `User` as optional one-to-one rows (`student` , `mentor` , `employer` (EmployerMember), `college` (CollegeAdmin)). A role change updates `User.role` only and keeps those rows. Only students require their profile row to exist (several student pages assume it), so `applyRole` in `actions/console.ts` creates an empty one when needed.

Admin bootstrap: `ADMIN_EMAILS` is read on sign-in by `src/lib/admin-bootstrap.ts`. Admin creation of other accounts is in `createUser` (`actions/admin.ts`).

Rate limits

`rateLimit(key, limit, windowMs)` in `src/lib/rate-limit.ts` is a database-backed fixed window using one atomic upsert, so it survives restarts and is shared by all processes. It **fails open** if the database errors. Examples: sign-in codes per identifier and per IP, uploads 20/hour, bulk upload 60/hour, notification emails 10/hour per user. In tests, late sign-ins use their own `x-forwarded-for` header to avoid the per-IP code limit (20/hour).

B5 The data model

Prisma schema: `prisma/schema.prisma`. Generated reference of every model and column: `docs/DATABASE.md` (`npm run docs:db`). Models (50): `User` · `StudentProfile` · `PassportEntry` · `Project` · `SkillEvidence` · `MentorProfile` · `OfficeHourSlot` · `MentorSession` · `Endorsement` · `Company` · `EmployerMember` · `Opportunity` · `Application` · `Challenge` · `Submission` · `Assessment` · `AssessmentAttempt` · `Group` · `GroupMember` · `Post` · `Comment` · `Block` · `Department` · `Drive` · `DriveInvite` · `Interview` · `ApplicationEvent` · `CandidateTag` · `JobQuestion` · `ApplicationAnswer` · `InterviewFeedback` · `MessageTemplate` · `Offer` · `CompanyInvite` · `SavedOpportunity` · `College` · `CollegeAdmin` · `Notification` · `Report` · `AuditLog` · `OtpCode` · `Message` · `Shortlist` · `ContactMessage` · `RateLimit` · `ErrorLog` · `Upload` · `PlanConfig` · `SiteSetting` · `RoleChangeRequest`.

Enums (23): `Role` · `VerificationStatus` · `OpportunityType` · `WorkMode` · `ApplicationStage` · `ChallengeKind` · `ChallengeStyle` · `SubmissionStatus` · `AssessmentKind` · `AttemptStatus` · `GroupKind` · `GroupRole` · `PostKind` · `DriveStatus` · `InviteStatus` · `DriveOutcome` · `InterviewMode` · `InterviewStatus` · `OfferStatus` · `SessionStatus` · `Visibility` · `EntryKind` · `FeedbackRecommendation`.

Models added for the admin console

Model	Purpose	Key fields
`PlanConfig`	Employer plans, editable by admins. One source of truth for the usage panel, the admin Plans page and server-side limit enforcement	`key` (id, e.g. STARTER), `label`, `priceText`, `description`, `features` (Json lines), `listings, challenges, assessments, team`, `highlight, active, position`
`SiteSetting`	Key/value settings	`key` (`announcement`, `supportEmail`, `signup.STUDENT` ...), `value` ("on"/"off" for sign-up)
`RoleChangeRequest`	A user asking an admin to change their role	`userId, fromRole, requestedRole, reason, status` (PENDING / APPROVED / DECLINED), `decidedById, decisionNote, decidedAt`

`Company.plan` is a **string** equal to a `PlanConfig.key` (it used to be an enum; changing it to a string is what lets admins create plans). A company whose plan key has no row falls back to the smallest plan so nothing is ever unlimited by accident (`getPlan` in `src/lib/plans.ts`).

Conventions

- Ids are `cuid()` strings. Timestamps are UTC `DateTime`; the UI formats in IST (`Asia/Kolkata`).
- JSON columns hold lists (`tags`, `skillsClaimed`, `features`). Read them through `asList()` in `src/lib/lists.ts` so a bad value never crashes a page.
- Sample data is flagged with `isSample` (companies, mentors, students) or named "Sample ..." (colleges). Real queries that count "real" records exclude them.
- Contact details (email, phone) live on `User` and are never selected by public, Passport, roster, drive or report queries. Always use an explicit `select` when returning user-related data to a page another person can see.
- Deleting a user cascades to their profile rows and role requests. No admin screen deletes users; only `deleteMyAccount` does, for the signed-in person.

B6 Database migrations (the careful part)

Migrations are plain SQL under `prisma/migrations/<timestamp>_<name>/migration.sql`. They are **forward-only and additive**: never edit a migration that has been merged; fix forward with a new one. Hostinger deploys only the built app (no `prisma/` folder, no Prisma CLI), so the SQL is **embedded into the build** and applied by the server at start.

How the pieces fit

- `scripts/embed-migrations.mjs` (runs as `prebuild`) reads every migration folder and writes `src/lib/migrations.generated.ts`. **Commit that file.** CI fails if it is stale.
- `src/lib/migrate.ts` runs from `src/instrumentation.ts` at server start (unless `AUTO_MIGRATE=false`): it takes a MySQL advisory lock so two instances cannot race, creates `_prisma_migrations` if missing, and applies pending migrations in order, recording them with Prisma's own checksum so `prisma migrate status` still agrees.
- CI applies migrations to MySQL 8 and MariaDB 10.11, and on MySQL 8 also checks that `schema.prisma` and the migrations produce the same schema (drift check).

Step by step: change the schema

1. Edit `prisma/schema.prisma`. Keep changes additive (new tables or nullable/defaulted columns).
2. Generate the SQL from your **local database that is currently at the previous migration state**: `npx prisma migrate diff --from-url "$DATABASE_URL" --to-schema-datamodel prisma/schema.prisma --script`.

3. Create `prisma/migrations/<YYYYMMDDHHMMSS>_short_name/migration.sql` and paste the output. The timestamp must sort after every existing folder.
4. **MariaDB noise:** MariaDB reports JSON columns as longtext, so the diff may include unrelated `MODIFY` statements for JSON columns. Delete those lines. If the diff misses something (for example changing a column type with a default), write the `ALTER TABLE` by hand and read it twice.
5. Embed and apply: `node scripts/embed-migrations.mjs`, then `npx prisma migrate deploy`, then `npx prisma generate`.
6. Prove there is no drift: run the diff from step 2 again; it should print an empty migration (ignoring MariaDB JSON noise).
7. Regenerate docs: `npm run docs:db && npm run docs:api`. Commit schema, migration folder, `migrations.generated.ts` and docs together.

```
-- prisma/migrations/20261020100000_role_change_requests/migration.sql (abridged)
CREATE TABLE `RoleChangeRequest` (
    `id` VARCHAR(191) NOT NULL,
    `userId` VARCHAR(191) NOT NULL,
    `requestedRole` ENUM('STUDENT','MENTOR','EMPLOYER','COLLEGE_ADMIN','PLATFORM_ADMIN') NOT NULL,
    `status` VARCHAR(12) NOT NULL DEFAULT 'PENDING',
    PRIMARY KEY (`id`)
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
ALTER TABLE `RoleChangeRequest` ADD CONSTRAINT `RoleChangeRequest_userId_fkey`
  FOREIGN KEY (`userId`) REFERENCES `User`(`id`) ON DELETE CASCADE ON UPDATE CASCADE;
```

Careful. Never `DROP` or rename a column in the same release that stops using it. Deploy the code that no longer reads it first, then remove it in a later migration. Seed data must never be inserted by a migration except fixed reference rows (the three default plans are one such case).

Note. Local gotchas: dropping the database under a running dev server breaks it, so restart `npm run dev` after recreating the database. If `migrate dev` complains about the shadow database, use the `migrate diff` flow above.

B7 Server actions: the pattern to copy

Every mutation is a server action in `src/app/actions/*.ts` (file starts with `"use server"`). They return a small object the form shows: `{ ok?, error?, fieldErrors? }` (`Result` in `_shared.ts`). The admin console adds `note?` for a confirmation sentence (`NoteResult`).

```
"use server";
import { revalidatePath } from "next/cache";
import { z } from "zod";
import { db } from "@/lib/db";
import { requireActionRole } from "@/lib/session";
import { audit } from "@/lib/audit";
import { fail, obj, type Result } from "./_shared";

const schema = z.object({ name: z.string().trim().min(2, "Enter the name").max(160) });

export async function saveThing(_: Result, fd: FormData): Promise<Result> {
  const a = await requireActionRole("PLATFORM_ADMIN");       // 1. who is calling?
  const r = schema.safeParse(obj(fd));                       // 2. validate everything
  if (!r.success) return fail(r.error);                      //    first message goes to the user
  const dup = await db.thing.findFirst({ where: { name: r.data.name } });
  if (dup) return { error: "That name already exists." };    // 3. friendly business-rule errors
  const t = await db.thing.create({ data: r.data });         // 4. write
  await audit(a.id, "thing.created", `${r.data.name} (${t.id})`.slice(0, 190)); // 5. audit sensitive
changes
  revalidatePath("/dashboard", "layout");                    // 6. refresh pages
  return { ok: true };
}
```

Rules

- **Return errors, do not throw**, for anything a user can cause. Throwing is for “this should never happen” or for button actions that take only an id.
- Plain-language messages. They are shown verbatim to staff and users.
- Cap every string (`.max()`) and slice values written to `AuditLog.target` to 190 characters.
- Admin actions never delete users. Deleting configuration rows (a plan) must refuse when something still uses it (see `deletePlan`).
- Email is best effort and must never fail the action: wrap sending in try/catch (see `emailAccountReady`, which never throws).
- Notify affected people with `notify(userId, title, body)`; it saves an in-app notification and emails when configured, capped at 10 emails/hour/user.

Forms on the client

- `ActionForm` (`src/components/dash/action-form.tsx`): the original wrapper with `Field`, `Area` helpers.
- `ConsoleForm` (`src/components/dash/console-form.tsx`): used by the admin console; takes `action`, an accessible `label` (this becomes the form’s `aria-label`, which tests use to find it), `submit` text, and optional `resetOnSuccess`. It shows the error in a `role="alert"` and the confirmation in `role="status"`.
- Both submit manually with `startTransition` instead of `action={…}`: React 19 resets uncontrolled forms after an action, which lost users’ input when an error came back.
- `ActionButton` (id-only actions such as Verify, Close) wraps a button in a form and supports a `confirm` message.

B8 How-to recipes

Recipe 1: add a new admin page

1. Create `src/app/dashboard/<name>/page.tsx`. First line of the component: `await requirePageRole("PLATFORM_ADMIN")`.
2. Use `PageTitle`, `Panel`, `Empty`, `Status` from `@/components/dash/ui`. Follow an existing page (`colleges/page.tsx` is the smallest full example).

3. Add one entry to the right section in `src/lib/modules.ts`: `{ label, icon, href, desc }`. The sidebar, the "Everything in your workspace" grid and the tab bars all derive from it. Pick an `icon` from the existing `IconKey` list or add one in `src/components/dash/icons.tsx`.
4. Add the route to `e2e/access.mjs` (`PAGES` map: role list allowed) and a check to `e2e/platform.mjs`.
5. Do not restyle anything. Reuse existing classes.

Recipe 2: add a new setting an admin can change

1. Choose a key. Settings are rows in `SiteSetting`; defaults live in `getSettings()` in `src/lib/settings.ts` (so a missing row still works).
2. Add the field to the `Settings` type and parse it in `getSettings()`.
3. Add the input to `src/app/dashboard/site/page.tsx` and the zod field and upsert row in `saveSettings` (`actions/console.ts`).
4. Read it with `await getSettings()` wherever it is used.

Recipe 3: add or change a plan limit

1. Add the column to `PlanConfig` (migration, B6).
2. Extend `PlanRow` and `FALLBACK` in `src/lib/plans.ts`, the `Meter` type and `usage()`.
3. Enforce with `limitError(companyId, planKey, meter)` in the action that creates the thing (see how `postOpportunity` does it).
4. Add the input to `PlanFields` in `dashboard/plans/page.tsx` and to `planSchema` in `savePlan`.

Recipe 4: add a bulk-import type

1. Add the kind to `BulkKind` and a `SPECS` entry (`columns`, `required`, `sample`, `help`) in `src/lib/bulk.ts`.
2. Write a processor `async function thing(r, ctx): Promise<Out>` modelled on `student`: validate, de-duplicate with `ctx.seen`, check the database, and only write when `ctx.commit` is true. Return `good`, `skip` or `err`.
3. Register it in `PROCESS`.
4. Add the kind to the allow-list in `src/app/api/admin/bulk/route.ts`, a `<BulkUpload kind=…>` panel in `dashboard/bulk/page.tsx`, and a test.

Recipe 5: add an environment variable

1. Read it only on the server (`process.env.NAME`). Anything prefixed `NEXT_PUBLIC_` is baked into the browser bundle at build time: never put secrets there.
2. Document it in `.env.example`, `docs/OPERATIONS.md` and B3 of this manual.
3. If production cannot work without it, validate in `src/lib/env.ts` so the start-up log says so clearly.
4. If an admin should know about it, add a line to `dashboard/launch/page.tsx`.

Recipe 6: add a new role-gated API route

```
// src/app/api/admin/thing/route.ts
export async function POST(req: Request) {
  try { await assertSameOrigin(); } catch { return NextResponse.json({ error: "Bad origin" }, { status: 403
}); } // CSRF
  const user = await getUser();
  if (!user) return NextResponse.json({ error: "Please sign in" }, { status: 401 });
  if (user.role !== "PLATFORM_ADMIN") return NextResponse.json({ error: "Not allowed" }, { status: 403 });
  if (!(await rateLimit(`thing:${user.id}`, 60, 60 * 60 * 1000))) return NextResponse.json({ error: "Too
many" }, { status: 429 });
  // …validate, do the work, audit…
}
```

Server actions already enforce same-origin; route handlers do not, so call `assertSameOrigin()` in every POST. Add the route to `e2e/access.mjs`.

B9 The admin console, file by file

File	Responsibility
`src/lib/plans.ts`	`allPlans()`, `getPlan(key)` (falls back to the smallest plan), `usage(companyId)`, `limitError()`. Server-only
`src/lib/settings.ts`	`getSettings()`, `SIGNUP_ROLES`, `signupClosedMessage(role)`. Defaults when a row is missing; logs and returns defaults if the table is unreachable
`src/app/actions/console.ts`	`savePlan, deletePlan, saveSettings, saveCompany, postListingFor, adminSetListingClosed, saveCollege, saveMentor, changeUserRole, requestRoleChange, cancelRoleRequest, decideRoleRequest`. All admin-only except the user-side role-request actions; all audit-logged
`src/app/actions/admin.ts`	Older admin actions: verify/reject company, college, mentor; moderate listing; reports; suspend/unsuspend; `setPlan`; `createUser`
`src/lib/account-email.ts`	`emailAccountReady({ name, email, extra })`: tells a person an admin set up their account; returns a sentence for the admin; never throws
`src/lib/bulk.ts`	Spreadsheet reading, per-row validation and import (B10)

File	Responsibility
`src/app/api/admin/bulk/route.ts`, `…/template/route.ts`	Upload endpoint (preview or import) and CSV template download
`src/app/api/upload/route.ts`	Image upload. Platform admins may upload a company logo (`companyId`) or a mentor photo (`mentorId`) on someone's behalf
`src/app/dashboard/{companies,colleges,mentors,users,plans,site,bulk,launch}`	The admin pages
`src/components/dash/{console-form,bulk-upload,announcement,image-upload}.tsx`	Shared UI for those pages

Behaviours worth knowing

- **Plans.** `savePlan` upserts by key; features are stored as up to 12 lines of up to 120 characters. It refuses to switch off the last active plan. `deletePlan` refuses while companies use the plan or when it is the only plan. `setPlan` validates against `PlanConfig` and notifies the company's team.
- **Admin-posted listings** (`postListingFor`) are created with `moderated: "VERIFIED"`, require a VERIFIED company and bypass plan limits (the admin is the moderator). Challenges are refused because they need a brief and rubric.
- **Companies**: names unique (case-insensitive collation), website must be http(s), career slug validated by `slugProblem` and unique. Verifying sets `verifiedAt`.
- **Mentors**: `saveMentor` creates a User (role MENTOR) with a verified MentorProfile and emails them via `emailAccountReady` with a sentence telling them to update their profile. A mentor's own `saveMentorProfile` cannot change status.
- **Sign-up toggles** are enforced in three places: `requestOtp` (before emailing a code), `submitOtp` (at user creation) and the Google callback. Existing users are never blocked.
- **Roles**: `applyRole` refuses own-role changes, same-role changes and demoting the last active admin, runs in a transaction, audits, and notifies.

B10 Bulk import internals

Flow: browser → `POST /api/admin/bulk` (multipart: `kind`, `file`, `mode=preview|import`, optional `sendEmails=1`) → `readTable` → `runBulk` → JSON report. It is a route handler, not a server action, because server actions have a small default body limit.

1. **Guards:** same-origin, signed in, PLATFORM_ADMIN, rate limit 60/hour. The route returns 400 for an unknown kind or a file that is not .xlsx or .csv.
2. `readTable`**:** rejects empty files and files over 2 MB. `.csv` is parsed by a small RFC-4180 parser (quotes, escaped quotes, CRLF, BOM). `.xlsx` is read with `read-excel-file/node` (`readSheet`, first sheet). Cell values become trimmed strings; dates become ISO dates; trailing empty rows are dropped.
3. `runBulk`**:** needs a header row plus data; caps at 1,000 rows; normalises headers (`lowercase`, spaces and dashes to underscores); checks required columns; loads the verified colleges or the plans once; then processes **row by row**, each in its own try/catch so one failure never aborts the batch.
4. **Per-row processors** (`student`, `company`, `mentor`) validate, de-duplicate within the file (`ctx.seen`), skip existing records, and write only when `ctx.commit`. Preview and import run the same code, so the preview is a faithful dry run.

5. **Emails** are sent after the loop, at most 200 (`BULK_MAX_EMAILS`), through `emailAccountReady`; the report says how many were sent.
6. **Audit:** one `bulk.import` entry per committed import with counts and the file name.

> **Note.** The upload is processed in memory and never stored. Users created by bulk import get no password; they sign in with an emailed code like everyone else.

Report shape: `{ kind, committed, rows: [{ n, label, status: "ok"|"skipped"|"error", message }], counts: { ok, skipped, error }, emailsSent }`.

Why not the usual `xlsx` package?

The version of `xlsx` published to npm has known vulnerabilities and is no longer maintained there. `read-excel-file` reads .xlsx only (no macros, no formulas executed), which is exactly what we need. `write-excel-file` is a dev dependency used only by tests to build sample workbooks.

B11 Email

- `src/lib/email.ts` sends over SMTP with nodemailer. `emailConfigured()` tells you if the variables exist. Without SMTP, sign-in codes cannot be delivered in production.
- Sign-in codes: `actions/auth.ts`. Notifications: `notify()` in `src/lib/audit.ts`. Account-ready messages: `src/lib/account-email.ts`.
- Hostinger only allows the mailbox itself (or an alias) as the sender: `EMAIL_FROM` must match `SMTP_USER`. The start-up log warns when they differ.
- In tests, `e2e/mock-mail.mjs` is a tiny SMTP server that captures messages.
- Unsubscribe links are signed (`src/lib/unsub.ts`); users can turn off notification emails in one click.

B12 Uploads and files

- Images only: PNG, JPEG, WebP, verified by magic bytes in `src/lib/image.ts` (`sniffImage`); SVG is refused. Maximum 1 MB (`MAX_IMAGE_BYTES`); the browser shrinks images first (`ImageUpload` draws them on a canvas and exports JPEG).
- Stored in the `Upload` table (data in the database, so backups cover them). Served by `GET /api/files/[id]` via an unguessable id with long caching: a capability URL. Do not upload anything sensitive.
- Ownership: `POST /api/upload` decides what an upload replaces by role. Students: own avatar and project images; mentors: own avatar; employers: own company logo; platform admins: any company logo or mentor photo. Old files are deleted only when the same owner uploaded them.

B13 UI conventions and the design system

> **Platform rule. Do not restyle existing screens.** The product look is approved. New pages reuse existing components and classes. **Do not change the logo** (`src/components/ui/brand.tsx`, `public/brand/`).

Topic	Convention
Theme	Dark by default; light is chosen in Settings → Appearance (stored in localStorage). Never add a theme toggle to the header
Tokens	CSS variables in `src/app/globals.css`, mapped to Tailwind colours in `tailwind.config.ts`: `bg, surface, muted, ink, ink-soft, line, primary, on-primary, accent, spot, lilac, sun, mint, danger`. Use tokens, never raw hex

Topic	Convention
Colour	Warm near-black / warm white, one hot orange (#FF5B2E) for actions; soft lilac, sun and mint “sticker” colours always with dark text
Fonts	Exactly two: Plus Jakarta Sans for everything and Instrument Serif italic for one emphasised word in a heading (`Serif` component). Code editors use the browser’s monospace
Shapes	Pill buttons, 20px cards (`.card`), rounded inputs with an orange focus ring, segmented-pill tabs, icon sidebar with an orange active pill
Primary button text	Dark text on orange (white on orange fails contrast)
Components	`Button` , `Badge` , `CompanyLogo` (monogram tile until a logo exists), `Serif` , `PageTitle` , `Panel` , `Status` , `Empty` , `ActionForm` , `ConsoleForm`
Navigation	One list: `src/lib/modules.ts` . Sidebar, section tabs and the workspace grid read it
Accessibility	Every form has an accessible name (`aria-label` or label); errors use `role="alert"` , confirmations `role="status"` ; scrollable regions are keyboard reachable; no horizontal scroll at 375 px
Copy	Plain language, no jargon, no “guaranteed”, no invented numbers. Pay, mode, location and eligibility always shown

Trust rules built into the product (never weaken)

- Pay, work mode, location and eligibility are always shown; unpaid roles are labelled unpaid; paid roles must state an amount.
- Student contact details are private by default and never returned by public, Passport, roster, drive or report queries.
- No “guaranteed job” language; match scores are advisory; nobody is auto-rejected; a person decides each application.
- Only the admin verification action can set VERIFIED on a profile entry, company, college or mentor.
- No invented statistics, testimonials, awards or logos. Sample data is always tagged “Sample”.
- Consent: college visibility and shared progress are opt-in and revocable.

B14 Security rules for contributors

The full checklist, with what is done and what is not, is `docs/SECURITY.md` . The rules you need every day:

1. **Authorise on the server, every time** (B4). Scope queries to the owner.
2. **Validate with zod**; cap lengths; never trust hidden fields.
3. **No raw SQL** except where already used (the rate limiter). Prisma parameterises everything else.
4. **Never render user HTML.** React escapes. The only `dangerouslySetInnerHTML` uses are JSON-LD (with `<` escaped) and a server-made QR SVG.
5. **External URLs**: accept http(s) only; use `rel="noopener noreferrer nofollow"` for user-supplied links.
6. **CSV exports** neutralise spreadsheet formulas (see the college report).
7. **Secrets** only in environment variables. Never log them. Never put them in `NEXT_PUBLIC_*` .
8. **Security headers** (CSP, HSTS, frame, nosniff, referrer, permissions) are set in `next.config.mjs` . The CSP forbids remote scripts and fonts, so do not add a CDN.

9. **Code execution** for coding assessments goes only to an isolated Judge0-compatible service (`SANDBOX_URL`); never run user code on the app server.
10. **Admin actions are audited** (`audit()`), and nothing an admin does deletes a user.

B15 Testing

Suite	Command	What it covers	Needs
Unit	`npm run test:unit`	18 tests of pure logic: matching, natural-language search parser, profile review, JD check, password hashing	Nothing
End-to-end	`npm run test:e2e`	About 280 checks of real journeys across all roles, including the admin console, bulk upload and role changes	Dev server + seeded empty-ish DB
Access matrix	`npm run test:access`	Every protected page and API as every role and anonymous: allowed or blocked	Dev server + seed
Accessibility	`npm run test:a11y`	axe-core WCAG A/AA on every page and role at desktop and 375 px, no horizontal scroll	Dev server + seed
Production smoke	`npm run test:e2e:prod`	CSP, PWA/offline, Open Graph, contact form on a production build	Production build
Rate limiter	`npx tsx e2e/ratelimit.ts fill|check`	Atomicity and survival across process restarts (runs in CI)	DB
Database down	`node e2e/degraded.mjs`	Public pages degrade gracefully when the DB is unreachable	Dev server

Run the end-to-end suite step by step

1. Create a **clean** database: `DROP DATABASE IF EXISTS kalwings_e2e; CREATE DATABASE kalwings_e2e;` (a dirty database causes false failures).
2. In your shell: `export DATABASE_URL=mysql://…/kalwings_e2e`, then `npx prisma migrate deploy` and `npm run db:seed`.
3. Start the dev server on port 3000 in the same environment: `npm run dev -- -p 3000` (restart it whenever you recreate the database).
4. If Playwright's own browser is not installed, point at one: `export CHROMIUM_PATH=/path/to/chrome`. Do not run `playwright install` on a managed machine.
5. Run `node e2e/platform.mjs`. It prints `PASS …` per check and `ALL PASSED` or `N FAILURE(S)` at the end. A full run takes about 8 minutes.

Writing a new e2e check

1. Find the matching block in `e2e/platform.mjs` (they are grouped by feature with `// ---` headings). Sessions per role already exist (`student, mentor, employer, admin, anon`).
2. Find forms by **aria-label** (`admin.locator('form[aria-label="Add a plan"]')`) and sections by their `aria-label`. This is why every form needs an accessible name.
3. Assert with `ok(condition, 'what it proves')`.
4. Use unique names with the run's `tag` so reruns do not collide.

5. After a mutation that re-renders, wait for the new state (`waitForSelector`), not a fixed delay.

Known flake patterns and the fixes we use

Symptom	Cause	Fix
Password form not found	Hydration not finished	Use the `openPasswordForm` helper (retries)
Message not seen after click	Server action still running	`waitUntil: "networkidle"` or wait for `p[role=status]`
Lockout loop flaky	Alert not rendered yet	Wait for the alert each iteration
Late sign-in refused	Per-IP code limit (20/hour) used up	New context with its own `x-forwarded-for`
Required field blocks resubmit	A form that resets on success clears required inputs	Refill every required field before the next submit
Success text vanishes	Page revalidates and swaps the form for another state	Assert the new state instead (for example “Waiting for an admin”)

Never do these to get a test green

Never skip, disable, loosen or quarantine a failing test, and never push an empty commit to re-run CI. Find the cause. A failure that passes on a second run is still a defect until you can explain it.

B16 CI/CD, deploy, health and backups

CI (GitHub Actions: `.github/workflows/ci.yml`)

Runs on every pull request and on pushes to main, on two databases in parallel (`verify (mysql-8)` and `verify (mariadb-10.11)`). Steps: install from the lockfile; `prisma validate`; apply all migrations; (MySQL only) schema-vs-migrations drift check; rate limiter test; embedded-migrations freshness; lint; typecheck; unit tests; build; start the server and check `/api/health`. The browser suites run locally, not in CI.

Deploy

1. Open a pull request from a feature branch. Wait for both CI jobs to be green.
2. Merge into `main` (squash). Hostinger pulls, runs `npm run build` and starts the app with `npm start`.
3. On start the server applies pending migrations and logs `[migrate] ok`.
4. Open `/api/health`: expect `{"ok":true}`. Open Dashboard → Launch readiness.
5. **Rollback:** revert the PR on GitHub (auto-deploys) or redeploy the previous commit from hPanel. Migrations do not roll back: fix forward.

Health and monitoring

- `GET /api/health` returns `{"ok":true}` or a safe failure category: `unreachable, auth_failed, unknown_database, bad_database_url, prisma_engine, too_many_connections`. Point an uptime monitor at it (1-minute check).
- Unhandled server errors are stored (no personal data) and listed at Dashboard → Server errors.
- If the database is down, public pages still serve; sign-in and dashboards show a friendly error.

Backups and recovery

- `DATABASE_URL=… npm run backup -- ./backups 14` writes a gzip dump and keeps the newest 14. Schedule it daily and copy files off the server. Enable Hostinger’s own database backups too.

- Restore into a **new, empty** database first: `gunzip -c backups/FILE.sql.gz | mysql -h HOST -u USER -p DBNAME`, then confirm `npx prisma migrate status`, then switch `DATABASE_URL`. Test a restore before launch and quarterly.
- Uploaded images live in the database, so they are covered.

Routine maintenance

- Rotate secrets every 6 to 12 months or after any exposure: new `AUTH_SECRET` (signs everyone out), mailbox password, database password.
- Delete old rate-limit and error rows monthly (not scheduled yet): `DELETE FROM RateLimit WHERE resetAt < NOW()`; `DELETE FROM ErrorLog WHERE createdAt < NOW() - INTERVAL 30 DAY`.
- Keep audit logs at least 12 months.
- Review Dependabot pull requests weekly; run `npm audit` before launches.

Incident checklist

1. Check `/api/health` and the runtime log.
2. If the database is unreachable the public site still serves; fix the connection.
3. Check recent deploys; roll back if the problem started with one.
4. Check Dashboard → Server errors.
5. Suspected data exposure: suspend affected accounts, rotate secrets, preserve logs, notify affected users as the law requires.

B17 Working with Git and pull requests

1. Branch from an up-to-date `main`: `git fetch origin main && git checkout -b feature/short-name origin/main`.
2. Make small, reviewable changes. Keep unrelated refactors out.
3. Before you push, run what CI runs: `npm run lint && npm run typecheck && npm run test:unit && npm run build`, and the e2e suite for anything user-facing.
4. Write a commit message that says what and why in plain words.
5. Push and open a pull request. Describe what changed, what did not (for example “no schema change”), and how you tested. Mention any migration.
6. Wait for both CI jobs. Fix failures properly (B15). Merge only when you have approval; **ask before merging** and merge with squash.
7. Stacked PRs merge bottom-up. If two PRs conflict, merge `main` into your branch (do not rewrite shared history), resolve, retest.

> **Careful.** Never commit secrets, `.env`, database dumps or real people’s data. Never push sample-seeding changes that touch real users. The unseed command only ever removes sample data.

B18 Troubleshooting for developers

Symptom	Likely cause	Fix
Server fails at start: “AUTH_SECRET must be set”	Missing or short secret	Set a 32+ character `AUTH_SECRET`

Symptom	Likely cause	Fix
`/api/health` shows `auth_failed` / `unknown_database`	Wrong DATABASE_URL user, password or name	Fix the URL; URL-encode special characters; read the `[startup] database` line
`[migrate] FAILED`	A migration cannot apply (permissions, conflicting object)	Read the logged statement; fix forward with a new migration; never edit an applied one
CI: embedded migrations are stale	Changed `prisma/migrations` without regenerating	`node scripts/embed-migrations.mjs` and commit `migrations.generated.ts`
CI: schema drift on MySQL	Edited `schema.prisma` without a migration	Add the migration (B6)
Dev server errors after recreating the DB	Server holds old connections	Restart `npm run dev`
"Port 3000 already in use"	Old dev server	Stop it (`fuser -k 3000/tcp` on Linux) then restart
E2E: many unrelated failures	Dirty database	Recreate the e2e database and reseed
No sign-in code appears in dev	SMTP variables set locally	Unset them; dev shows the code on screen when no provider is configured
Client build error about server-only code	A client component imports a server-only module (for example via a shared file)	Split the file: keep types or constants in a plain module; server code behind server components
Admin cannot see new menu item	Item not in `modules.ts` or wrong section/role	Add it to the correct role array
Uploaded file rejected	Over 1 MB, not PNG/JPEG/WebP, or wrong owner	Resize; check role rules in B12

B19 Glossary

Term	Meaning
Server action	A server function a form calls directly; a public HTTP endpoint, so it must authorise itself
Revalidate	Tell Next.js to refresh cached pages after a change (`revalidatePath`)
Embedded migration	Migration SQL compiled into the build and applied at server start
Drift	The database schema differs from what the migrations produce
Capability URL	An unguessable URL that works without login (uploaded images)
Sample data	Fake demo records created by the seed; always tagged "Sample"
Moderated	The Opportunity field that says whether a listing is approved (VERIFIED) or waiting
Meter	One of the four plan limits: listings, challenges, assessments, team

B20 Appendices (generated from the source tree)

Appendix 1: Navigation by role

Source: `src/lib/modules.ts`. Roles map to the pages each role sees in the sidebar.

Role	Label	Route
student	Opportunities	`/opportunities`
student	Challenges	`/challenges`
student	Campus drives	`/dashboard/drives`
student	Saved roles	`/dashboard/saved`
student	Applications	`/dashboard/applications`
student	Profile	`/dashboard/profile`
student	Career copilot	`/dashboard/copilot`
student	Mentor sessions	`/dashboard/mentoring`
student	Community	`/community`
employer	Company	`/dashboard/company`
employer	Roles	`/dashboard/roles`
employer	Talent search	`/dashboard/talent`
employer	Pipeline	`/dashboard/pipeline`
employer	Analytics	`/dashboard/analytics`
employer	Campus drives	`/dashboard/drives`
mentor	Mentor profile	`/dashboard/mentoring`
mentor	Community	`/community`
college	College	`/dashboard/college`
college	Placement drives	`/dashboard/drives`
admin	Listing moderation	`/dashboard/moderation`
admin	Profile verification	`/dashboard/credentials`
admin	Abuse reports	`/dashboard/reports`
admin	Companies	`/dashboard/companies`
admin	Colleges	`/dashboard/colleges`
admin	Mentors	`/dashboard/mentors`
admin	Users	`/dashboard/users`
admin	Bulk upload	`/dashboard/bulk`
admin	Employer plans	`/dashboard/plans`
admin	Site settings	`/dashboard/site`
admin	Insights	`/dashboard/insights`
admin	Contact inbox	`/dashboard/inbox`
admin	Launch readiness	`/dashboard/launch`
admin	Audit log	`/dashboard/audit`
admin	Server errors	`/dashboard/errors`

Appendix 2: Server actions by file

Source: `src/app/actions/*.ts`. Every function here must start with `requireActionRole` or be an intentionally public action (sign-in, contact form).

File	Exported actions
`account.ts`	`deleteMyAccount` `setPassword` `removePassword`
`admin.ts`	`reviewCompany` `reviewCollege` `reviewMentor` `moderateOpportunity` `resolveReport` `markContactHandled` `suspendUser` `unsuspendUser` `reviewEntry` `setPlan` `createUser`
`assessments.ts`	`createAssessment` `closeAssessment` `runSamples` `submitAttempt` `submitWritten` `reviewWritten`
`auth.ts`	`requestOtp` `submitOtp` `loginWithPassword` `logout`
`college.ts`	`createCollege`
`common.ts`	`sendMessage` `reportContent` `setEmailNotifications`
`community.ts`	`createGroup` `joinCollegeGroup` `joinGroup` `leaveGroup` `createPost` `createComment` `setHidden` `deleteOwnContent` `blockUser` `unblockUser` `archiveGroup`
`console.ts`	`savePlan` `deletePlan` `saveSettings` `saveCompany` `postListingFor` `adminSetListingClosed` `saveCollege` `saveMentor` `changeUserRole` `requestRoleChange` `cancelRoleRequest` `decideRoleRequest`
`contact.ts`	`sendContact`
`drives.ts`	`addDepartment` `deleteDepartment` `createDrive` `respondDrive` `closeDrive` `inviteEligible` `respondInvite` `setOutcome` `setDepartment`
`employer.ts`	`createCompany` `postOpportunity` `closeOpportunity` `reopenOpportunity` `setStage` `scheduleInterview` `setInterviewStatus` `extendOffer` `markHired` `inviteMember` `acceptInvite` `removeMember` `cancelInvite` `saveStageLabels` `saveNotes` `toggleShortlist` `createChallenge` `reviewSubmission` `saveCareerSlug` `saveInterviewFeedback` `saveTemplate` `deleteTemplate` `bulkUpdate` `removeTag`
`mentor.ts`	`saveMentorProfile` `addSlot` `deleteSlot` `respondToSession` `giveEndorsement`
`student.ts`	`saveProfile` `addProject` `deleteProject` `saveCollege` `toggleContactVisible` `applyTo` `applyWithAnswers` `withdrawApplication` `toggleSave` `requestSession` `cancelSession` `markNotificationsRead` `saveUsername` `savePrivacy` `addEntry` `deleteEntry` `requestEntryVerification` `respondOffer` `submitWork` `withdrawSubmission`

Appendix 3: HTTP route handlers

Route	Methods
`/api/account/export`	GET
`/api/admin/bulk/template`	GET
`/api/admin/bulk`	POST
`/api/auth/google/callback`	GET
`/api/auth/google`	GET
`/api/college/report`	GET
`/api/files/[id]`	GET
`/api/health`	GET
`/api/interviews/[id]/ics`	GET

Route	Methods
`/api/sessions/[id]/ics`	GET
`/api/upload`	POST

Appendix 4: Where to read more

Document	What it holds
`README.md`	Status of every area, setup, where to replace placeholders
`DEPLOYMENT.md`	GitHub → Hostinger step by step, including the VPS option
`docs/DATABASE.md`	Generated: every model, column and index (`npm run docs:db`)
`docs/API.md`	Generated: server actions and routes (`npm run docs:api`)
`docs/OPERATIONS.md`	Environment, deploy, health, backups, routine tasks, incidents, retention, release gates
`docs/SECURITY.md`	Checklist with Done / Partly / Not done
`docs/QUALITY.md`	Test evidence and known gaps
`docs/FEATURES.md`	What each role can do and how overlapping features were merged
`docs/ROADMAP.md` , `docs/PRD.md`	Phases and product requirements

Appendix 5: Known limitations and open items

- **Not built:** SMS / mobile sign-in, online payments and invoicing, LLM features, file uploads for challenge submissions, a deployed code-execution sandbox, Sentry/analytics.
- **Untested against real services:** Google sign-in, a real Judge0 sandbox. Email is tested against a mock server and, in production, by Hostinger SMTP.
- **Wording:** product names in `modules.ts` are plain placeholders until the kalwings.com wording is supplied; a few older headings still say “Talent Card” or “Talent Passport”.
- **Logo:** the mark is redrawn in code from a supplied image; replace with the original file in `public/brand/` and `brand.tsx` when available (do not otherwise change the logo).
- **Data retention:** automated deletion of old rate-limit, OTP and error rows is documented but not scheduled.
- **Bulk upload:** templates are CSV (open in Excel); .xlsx and .csv are accepted; no .xls; 1,000 rows and 2 MB per file.
- **Accessibility:** automated axe checks pass; no manual screen-reader pass has been done.